

Apuntes completos teoría de la información

UNIDAD Nº1 - INFORMACIÓN Y ENTROPÍA

1.1. Historia

La Teoría de la Información (TI) adquiere su base formal en 1948, cuando el matemático e ingeniero Claude E. Shannon publicó su artículo *"Una teoría matemática de la comunicación"*. Shannon demostró que la cantidad de información puede medirse matemáticamente con total independencia del significado o connotación semántica del mensaje, y probó que es posible transmitir datos sin errores siempre y cuando la velocidad de transmisión sea menor a la capacidad del canal. Se apoyó fuertemente en el trabajo de Ralph Hartley (1928), quien fue pionero al proponer medir la información utilizando el logaritmo de las probabilidades.

1.2. Definición: teoría de la Información

La Teoría de la Información es la disciplina que estudia la transmisión, compresión y control de errores de la información de manera eficiente a través de canales de comunicación. Nació debido a la complejidad de los sistemas y el abarrotamiento de los sistemas de comunicación. Aplica procedimientos matemáticos provenientes de la termodinámica, la física y el análisis armónico.

Permite:

- Calcular la capacidad máxima o ideal de los canales para determinar si están bien aprovechados
- Analizar estadísticamente las distintas fuentes de información para seleccionar el medio más adecuado
- Adaptar las características de la transmisión a la capacidad discriminadora del destinatario
- Comparar y evaluar distintos sistemas de modulación y codificación en cuanto a su eficiencia para combatir ruidos y errores
- Estudiar y evaluar nuevos medios de comunicación

1.3. Mensaje

Serie temporal de sucesos variables y medibles que puede ser representada y almacenada en forma de ordenamiento espacial.

Los mensajes se dividen en continuos (sistemas analógicos, como el sonido de la voz) o discretos (sistemas digitales, constituidos por un conjunto numérico limitado como letras o signos de un alfabeto).

1.4. Comunicación de ideas

Al comunicarnos intervienen varios aspectos (físicos, formales, semánticos y pragmáticos). Sin embargo, la Teoría de la Información circunscribe su estudio estrictamente a la estructuración estadística de las señales, ignorando por completo el nivel semántico (el significado) y la intención humana detrás de ellas.

1.5. Noción de canal

El "canal" engloba el sistema físico de comunicación por el que transitan los mensajes. Debe contar con dos o más estados posibles que el emisor controla y el receptor es capaz de discriminar, estableciéndose previamente un repertorio o alfabeto de símbolos conocido en ambos extremos para codificar la transmisión.

1.6. Incertidumbre e información

Cuando el receptor desconoce qué símbolo de la serie va a llegar a continuación, se dice que está en un estado de ignorancia o **incertidumbre**. En la TI, se define a la **información** como todo aquello que tiene el poder de disminuir o eliminar el rango de incertidumbre del receptor.

1.7. Medida de la información

Para poder medirla, se compara la incertidumbre del receptor antes y después de que llegue el mensaje.

A nivel matemático, la medida de la información debe cumplir que: aumenta cuantas más alternativas posibles existan, alcanza su máximo cuando todos los símbolos son equiprobables, se vuelve cero cuando hay completa certeza, y sus valores individuales gozan de la propiedad aditiva.

En resumen, propiedades de la medida de información:

- Aumenta con el número de posibilidades
- Alcanza un valor máximo para alternativas equiprobables
- Se anula cuando hay certeza
- Goza de propiedad aditiva

1.8. Probabilidad e información

La información es inversamente proporcional a la probabilidad, un evento con certeza absoluta no aporta información, mientras que un suceso improbable aporta mucha.

Sea E_i un suceso que puede presentarse con probabilidad p_i . Cuando E_i tiene lugar decimos que hemos recibido:

$$I_i = \log_2 \left(\frac{1}{p_i} \right) = -\log_2 p_i \quad \text{"unidades de informacion"}$$

La elección de la base del logaritmo que interviene en la definición es la que determina la unidad de información:

- Para la base 2, la unidad correspondiente es el *bit* (la que usaremos siempre a partir de ahora)
- Para la base e (logaritmo natural), la unidad correspondiente es el *nat*
- Para la base 10, la unidad de información es el Hartley

Notemos que si $P(E_i) = 1/2$, será $I(E_i) = 1 \text{ bit}$. Es decir, un bit es la cantidad de información obtenida al especificar una de dos posibles alternativas igualmente probables.

1.9. Fuentes de información

Es el mecanismo, aparato o proceso que emite señales o genera los mensajes. Desde el punto de vista estadístico, una fuente discreta se caracteriza matemáticamente por su "alfabeto" (el conjunto

finito de símbolos o eventos que puede emitir) y por la distribución de probabilidad de dichos símbolos.

La fuente de memoria nula es el tipo de fuente es la más sencilla, ya que los símbolos emitidos son estadísticamente independientes, está compuesta por, un alfabeto finito y fijo, $S = \{s_1, s_2, \dots, s_q\}$ y las probabilidades de aparición de cada símbolo $P(s_1), P(s_2), \dots, P(s_q)$

Luego hay fuentes más complejas donde la probabilidad de aparición de un símbolo depende de los símbolos que aparecieron anteriormente

1.10. Entropía. Fórmula de Shannon

Esta es un promedio ponderado de la información suministrada por la fuente.

La entropía de una fuente A con n símbolos, con probabilidades $p_1, p_2, \dots, p_n$, es:

$$H(A) = \sum_{i=1}^n p_i * I_i = - \sum_{i=1}^n p_i * \log_2(p_i)$$

Esta se interpreta como: la fuente A tiene un promedio de informacion de $H(A)$ bits, por símbolo que la fuente genera.

$$0 \leq H(A) \leq \log_2 n$$

El caso de $H(A) = 0$ ocurre cuando la fuente es cierta, es decir uno solo de los símbolos tiene una probabilidad de 1, por lo tanto, no hay incertidumbre sobre lo que hará la fuente y por lo tanto los símbolos emitidos por la fuente no aportan información.

El caso de $H(A) = \log_2 n$ se da cuando la incertidumbre de lo que hará la fuente es máxima, esto ocurre cuando todos los símbolos son equiprobables.

La diferencia entre el rendimiento máximo y el real se conoce como **redundancia** (el uso de más símbolos de los estrictamente necesarios para transmitir la misma información).

$$R = 1 - \frac{H(A)}{H_{max}} = 1 - \frac{H(A)}{\log_2 n}$$

Una fuente con **alta redundancia**: Tiene símbolos muy predecibles.

Una fuente con **baja redundancia**: Se acerca a la distribución uniforme.

Ejemplo:

- Alfabeto de 26 letras equiprobables: $H_{max} = \log_2 26 \approx 4.7$ bits/símbolo
- En el texto en castellano la entropía real suele ser entre 1 y 1.5 bits/símbolo
- Redundancia: $R = 1 - \frac{1.5}{4.7} \approx 68\%$

Significa que casi **dos tercios del espacio de codificación** no aportan información nueva, porque el idioma tiene patrones y repeticiones.

Los algoritmos de compresión funcionan eliminando esta redundancia para transmitir la misma cantidad de información con menor cantidad de símbolos.

UNIDAD N°2 - CANAL DE INFORMACION

2.1. Introducción

Mientras que el estudio de las fuentes de información se centra en eliminar la redundancia y optimizar la codificación de los símbolos, el abordaje del canal de información nos enfrenta al problema físico del ruido y la posibilidad de que se cometan errores en la transmisión. En esta etapa se utiliza la entropía no solo para comprimir datos, sino como herramienta para medir matemáticamente cuánta información *real* y confiable fluye desde un extremo al otro del sistema a pesar de dichas distorsiones.

2.2. Canal de Información

Como mencionamos antes, el canal se refiere al sistema físico de comunicación por el que transitan los mensajes, definido por sus estados y alfabeto. La definición matemática es: un canal de información es un modelo estadístico del medio a través del cual pasa una señal que transporta información, descrito por:

- Un conjunto $\{a_1, a_2, \dots, a_q\}$ de estados o símbolos de entrada
- Un conjunto $\{b_1, b_2, \dots, b_s\}$ de estados o símbolos de salida
- Un conjunto de probabilidades condicionales

Esto se representa mediante una **matriz de transición** $P(B|A)$ de dimensiones $q \times s$:

$$P(B/A) = \begin{bmatrix} P(b_1|a_1) & P(b_2|a_1) & \cdots & P(b_s|a_1) \\ P(b_1|a_2) & P(b_2|a_2) & \cdots & P(b_s|a_2) \\ \vdots & \vdots & \ddots & \vdots \\ P(b_1|a_q) & P(b_2|a_q) & \cdots & P(b_s|a_q) \end{bmatrix}$$

$P(b_1|a_1)$ se lee como “probabilidad de recibir b_1 dado que se envió a_1 ”

Propiedades de la matriz:

- La i -ésima fila corresponde al i -ésimo símbolo de entrada a_i
- La j -ésima columna corresponde al j -ésimo símbolo de salida b_j
- La suma de los elementos de una fila es siempre 1 (eso hace que para cada entrada debe haber al menos una salida del alfabeto B)

2.3. Relaciones probabilísticas en el canal

Las probabilidades en los extremos del canal están interconectadas a través de la matriz de transición. Sabiendo de estadística podemos encontrar varias relaciones:

Por ej.: si conocemos las probabilidades a priori con las que la fuente emite cada símbolo ($P(a_i)$), podemos calcular la probabilidad total de recibir cada símbolo de salida b_j .

Probabilidad de salida: La probabilidad de que aparezca un símbolo de salida b_j se calcula como:

$$P(b_j) = \sum_{i=1}^q P(b_j \cap a_i) = \sum_{i=1}^q P(a_i) * P(b_j|a_i)$$

Probabilidad conjunta: La probabilidad de aparición en simultaneo del par entrada-salida (a_i, b_j) :

$$P(a_i, b_j) = P(b_j \cap a_i) = P(a_i) * P(b_j|a_i) = P(b_j) * P(a_i|b_j)$$

Probabilidades hacia atrás: Dado un símbolo de salida, cuál símbolo de entrada lo produjo

$$P(a_i|b_j) = \frac{P(a_i \cap b_j)}{P(b_j)} = \frac{P(a_i) \cdot P(b_j|a_i)}{P(b_j)}$$

2.4. Canal uniforme y canal simétrico

Canal uniforme: Un canal se dice uniforme cuando todas las filas de la matriz de probabilidades de transición son permutaciones de la primera fila. Ej.:

$$\begin{bmatrix} 0.9 & 0.05 & 0.05 \\ 0.05 & 0.9 & 0.05 \\ 0.05 & 0.9 & 0.05 \end{bmatrix}$$

Canal simétrico: Subconjunto de los uniformes, exige simetría tanto en filas como en columnas. Lo que implica que el ruido afecte exactamente de la misma forma a cualquier símbolo (a diferencia del ejemplo anterior). Esto lleva a que las probabilidades se centran en la diagonal principal de la matriz de probabilidades, y el error para cada símbolo es uniforme para los demás símbolos, ej.:

$$\begin{bmatrix} 0.9 & 0.05 & 0.05 \\ 0.05 & 0.9 & 0.05 \\ 0.05 & 0.05 & 0.9 \end{bmatrix}$$

2.5. Canal binario simétrico (BSC)

El canal binario simétrico (BSC), es un caso particular de canal simétrico, es el canal más estudiado debido a su simplicidad y aplicabilidad práctica.

Canal sencillo con dos símbolos de entrada (0 y 1) y dos de salida (0 y 1). Con probabilidad de error p , siendo $1 - p$ la probabilidad de transmisión correcta

Matriz del BSC: $P(B|A) = \begin{bmatrix} P(0/0) & P(0/1) \\ P(1/0) & P(1/1) \end{bmatrix} = \begin{bmatrix} 1-p & p \\ p & 1-p \end{bmatrix}$

2.6. Sistemas de entropías

Al conectar la fuente con el receptor a través del canal, definimos distintas medidas de incertidumbre:

- $H(A)$: entropía de la fuente a priori (antes de la transmisión)
- $H(B)$: entropía de la fuente después de la transmisión
- $H(A|B)$: conocida como equivocación del canal o entropía condicional, mide la incertidumbre que aun le queda al receptor sobre el símbolo original enviado (A) después de haber leído el símbolo de salida (B). Es la información promedio perdida durante la transmisión.

$$H(A|B) = \sum_{i=1}^q \sum_{j=1}^s P(a_i, b_j) * I(a_i|b_j)$$

- $H(B|A)$: Representa la incertidumbre sobre qué símbolo se recibirá en la salida dado que sabemos qué símbolo se envió en la entrada. Es la **información promedio del ruido**. Representa la cantidad de información inútil que se añade en el canal de forma aleatoria, por lo que el receptor recibe símbolos que no fueron los que el emisor pretendía enviar. Es una medida de cuanto ruido mete el canal.

$$H(B|A) = \sum_{i=1}^q \sum_{j=1}^s P(a_i, b_j) * I(b_j|a_i)$$

- $H(A, B)$: Mide la incertidumbre de que ocurra el suceso simultáneo de que aparezca el símbolo a en la entrada y el símbolo b en la salida. Representa la información promedio total necesaria para describir simultáneamente el suceso de la entrada (A) y la salida (B). Es la información promedio que se envía más la información promedio añadida por el ruido (la cual no es útil).

$$H(A, B) = H(A) + H(B|A) = H(B) + H(A|B) = \sum_{i=1}^q \sum_{j=1}^s P(a_i, b_j) * I(a_i; b_j)$$

Relación fundamental: $I(A, B) = H(A) + H(B) - H(A, B)$

Cuando A y B son independientes:

- $H(A, B) = H(A) + H(B)$
- $I(A, B) = 0$

2.7. Información mutua

La **información mutua promedio** es un concepto de la teoría de la información que mide cuánta información comparten dos variables aleatorias. En otras palabras, indica cuánto saber una variable reduce la incertidumbre sobre la otra.

Se define como la cantidad real de información que la observación de los símbolos de salida (B) nos proporciona sobre los símbolos de entrada (A).

$$I(A; B) = \sum_{i=1}^q \sum_{j=1}^s P(a_i, b_j) * \log_2 \frac{P(a_i, b_j)}{P(a_i) * P(b_j)}$$

$$I(A; B) = H(A) + H(B) - H(A, B) = H(A) - H(A|B)$$

Es la información de la fuente de entrada menos la información perdida durante la transmisión.

Propiedades:

- $I(A; B) \geq 0$ (no negativa)
- $I(A; B) \leq \min(H(A), H(B)) \leq \min(\log_2 q, \log_2 s)$
- $I(A; B) = 0$ si y sólo si A y B son independientes
- $I(A; B) = I(B; A)$ (simetría)

Información mutua puntual (también conocida como mutua de sucesos/símbolos individuales):

$$I(a_i; b_j) = I(a_i \cap b_j) = \log_2 P(a_i \cap b_j) = \log_2 \frac{P(a_i | b_j)}{P(a_i)} = \log_2 \frac{P(a_i, b_j)}{P(a_i)P(b_j)}$$

Es una magnitud estrictamente **local** (no es una medida de información). Evalúa un par de realizaciones particulares:

- Si $P(a_i|b_j) = P(a_i)$: La observación de b_j no da información sobre a_i : $I(a_i; b_j) = 0$
- Si $P(a_i|b_j) > P(a_i)$: Ganancia de información positiva
- Si $P(a_i|b_j) < P(a_i)$: Pérdida de información (indica que hay mucho error en este símbolo)

Propiedades:

- Simetría: $I(a_i; b_j) = I(b_j; a_i)$
- $I(a_i; b_j) \leq \min(I(a_i), I(b_j))$
- $I(a_i; b_j) = 0$ si y solo si $P(a_i|b_j) = P(a_i)$ (son estadísticamente independientes)

Información mutua parcial

Mide la cantidad de información que un símbolo de entrada específico a_i deposita, en promedio, sobre todo el alfabeto de salida B o viceversa.

$$I(a_i, B) = \sum_{j=1}^s P(b_j|a_i) * \log_2 \frac{P(b_j|a_i)}{P(a_i)}$$

Es la esperanza matemática de la información puntual calculada sobre todas las posibles salidas para una entrada fija a_i . Debe ser positiva.

2.8. Capacidad de un canal

Máximo de información (bits/símbolo) transmisible por el canal, es el valor *máximo* que puede alcanzar la Información mutua promedio $I(A, B)$ respecto a las probabilidades de entrada $P(a)$.

$$C = \max_{P(a)} I(A; B)$$

Representa el límite superior o la **tasa máxima a la que se puede transmitir información de forma confiable**, independientemente de la codificación usada.

Dado que esta es el valor máximo de información mutua promedio, sigue sus mismas propiedades.

2.8.1. Capacidad de un canal uniforme

En un canal uniforme, como las filas de la matriz de transición son permutaciones, la situación es la misma para cada símbolo así que el cálculo se simplifica porque el ruido es constante para cualquier entrada, lo que hace que la entropía del ruido (entropía condicional) es constante a cualquier entrada.

$$C_{unif} = \log_2 r + \sum_B P(b|a_i) * \log_2 P(b|a_i)$$

La capacidad del canal se alcanza cuando la distribución de probabilidad de los símbolos de entrada es uniforme; es decir, **cuando todos los símbolos de entrada son equiprobables**.

2.8.2. Capacidad del BSC

Para un BSC con probabilidad de error p , la capacidad se alcanza cuando la entrada es equiprobable $P(0) = P(1) = 0.5$ (recordar que un BSC es un caso particular del uniforme).

$$C_{BSC} = 1 - H(p) = 1 + p * \log_2(p) + (1 - p) * \log_2(1 - p)$$

Gráfica de capacidad del BSC:

- Cuando $p = 0$: $C = 1 \text{ bit}$ (canal perfecto, sin errores)
- Cuando $p = 0.5$: $C = 0 \text{ bits}$ (canal inútil, ruido máximo)
- Cuando $p = 1$: $C = 1 \text{ bit}$ (canal perfecto con inversión)

2.8.3. Capacidad de un canal no uniforme

Es mucho más difícil de encontrar dado que distintos símbolos podrían tener distintas tasas de error, pasa a ser un problema de optimización complejo, del tipo maximización con restricciones.

UNIDAD Nº3 - DISCRETIZACIÓN TEMPORAL Y EN AMPLITUD (mejorar)

3.1. Introducción a las señales

En un sistema de comunicación, los estados sucesivos que adopta un canal (que puede ser un medio de propagación de una señal eléctrica) se conocen como señales.

Es decir, una **señal** es cualquier magnitud física que varía en el tiempo y porta información. (ondas de radio, señales de eléctricas, señales de luz), matemáticamente es una función:

$$x: \mathbb{R} \rightarrow \mathbb{R}, \quad t \mapsto x(t)$$

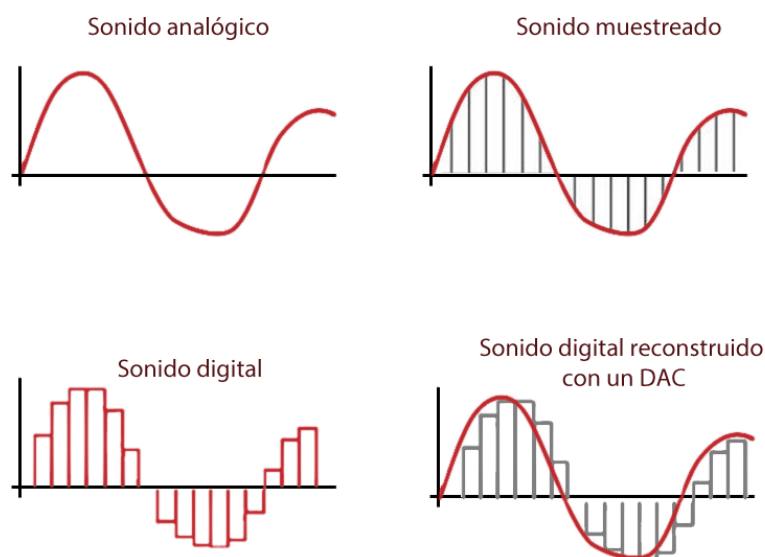
Sea x una señal, $x(t)$ el valor de la señal en el momento de tiempo t .

Entrando en muy pocos detalles, el caso más común es señales eléctricas, de las cuales se varia su tensión para transmitir información, también llamada voltaje, simbolizada V .

3.1.1. Señal analógica vs señal digital

Las diferencias principales entre ambos tipos de señales radican en cómo representan la información:

- **Señal analógica:** Es una señal de carácter continuo. Se caracteriza por poseer infinitos valores posibles de amplitud en cada instante de tiempo. Es decir, fluye de manera ininterrumpida tanto en el dominio del tiempo como en el de la amplitud.
- **Señal digital:** Es una señal de carácter **discreto** que solo puede tomar un **número finito de valores predeterminados**.



3.2. Digitalización

La digitalización convierte una señal analógica $x(t)$ en digital (una secuencia de números binarios). Esto permite almacenar, transmitir y procesar la señal con computadoras, pero **siempre introduce pérdida de información**, una vez digitalizada, no se puede recuperar la señal analógica original perfectamente.

Es un proceso muy común debido a que en el mundo real todo lo medible es analógico, todos los sensores funcionan de forma analógica en su nivel más básico (pueden realizar un proceso de digitalización interno).

Este proceso tiene 3 pasos obligatorios y secuenciales: muestreo, cuantificación, codificación.

3.2.1. Muestreo (sampling)

Sea $x(t)$ una señal analógica. El muestreo consiste en evaluar (medir) la señal x en instantes de tiempo equidistantes $n * T_s$:

$$x[n] = x(n * T_s), \quad n = 0, 1, 2 \dots$$

Donde:

- T_s es el periodo de muestreo (segundos entre muestras consecutivas)
- $f_s = \frac{1}{T_s}$ es la frecuencia de muestreo (muestras por segundo)

Este paso introduce el error de medición, que depende del instrumento usado para realizarlas. Y además se pierde todo lo que ocurre entre los instantes de muestreo, si la señal varía muy rápido, las muestras pueden no capturar esa variación.

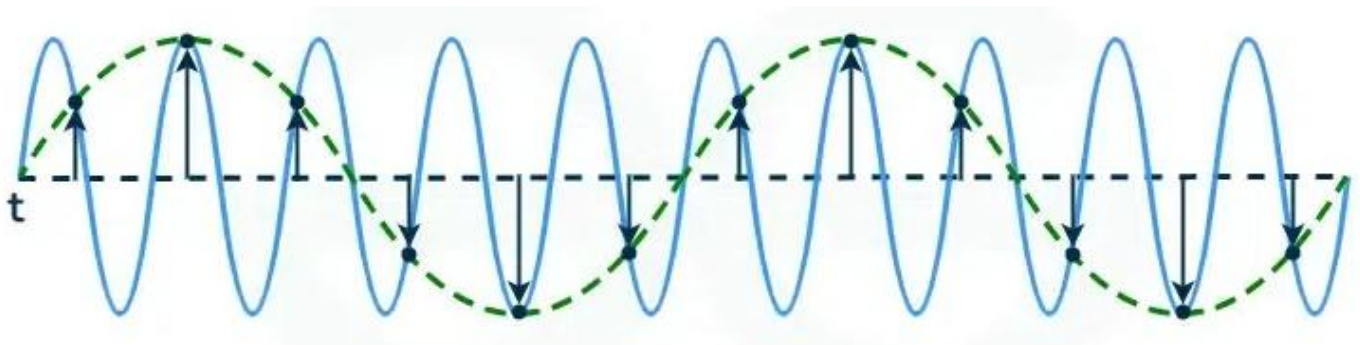
Teorema del muestreo de Nyquist-Shannon

Para reconstruir una señal $x(t)$ sin pérdida, la frecuencia de muestreo debe respetar: $f_s \geq 2B$, donde B es el ancho de banda máximo (o dicho de otra forma, la máxima frecuencia presente en la señal analógica), esta frecuencia $f_N = 2B$ es llamada frecuencia de Nyquist.

Dicho de otra forma: se debe muestrear al menos el doble de rápido, que el cambio más rápido que la señal analógica puede presentar (siendo la duración de este $\frac{1}{B}$, el periodo de muestreo mínimo para captarlo sería $\frac{1}{2B}$).

Aliasing (solapamiento espectral):

Si $f_s < 2B$ las componentes de alta frecuencia se "pliegan" sobre las de baja frecuencia, creando frecuencias falsas que no existían en la señal original, ej.:



La señal azul es la original, y la verde la reconstruida en base a las muestras.

Filtro anti aliasing

3.2.2. Cuantificación

Asigna cada valor continuo de amplitud $x[n]$ a uno de L niveles discretos predefinidos. Pasa de infinitos valores posibles a un conjunto finito.

$$x_q[n] = Q\{x[n]\}$$

Donde Q es la función de cuantización.

Cuantizador uniforme: es el tipo más simple (cada nivel tiene el mismo tamaño). El rango total $[V_{min}, V_{max}]$ se divide en L intervalos iguales de ancho Δ :

$$\Delta = \frac{V_{max} - V_{min}}{L}$$

Cada intervalo k -esimo ($k = 0, 1, \dots, L - 1$) tiene:

- Limite inferior: $V_{min} + \Delta * k$
- Limite superior: $V_{min} + \Delta * (k + 1)$
- Valor representativo (el valor en el medio del intervalo): $V_{min} + \Delta * \left(k + \frac{1}{2}\right)$

Se asigna cada muestra $x[n]$ al nivel cuyo valor representativo sea mas cercano (o lo que es lo mismo que este dentro de los limites).

Cabe destacar aquí que la cantidad de niveles se determina según la cantidad de bits disponibles para la digitalización, es decir $L = 2^n$ donde n es la cantidad de bits disponible. A mayor cantidad de bits mas intervalos y mejor cuantización.

Tipos de cuantizacion

Tipo	Descripción	Ventaja	Desventaja
Uniforme	Es el tipo más sencillo y consiste en dividir la señal utilizando niveles de cuantificación distribuidos exactamente de la misma manera o con un paso constante.	Simple de implementar	Malo para señales débiles
No uniforme	Asigna niveles de cuantificación que no están distribuidos uniformemente, adaptándose a la señal para asignar más resolución a las amplitudes más frecuentes. Generalmente se implementa mediante la cuantificación logarítmica, con niveles más densos cerca del 0.	Mejor error para señales débiles	Mayor complejidad

Error de cuantización

El error de cuantización es la diferencia entre el valor real y el valor cuantizado:

$$e_q[n] = x[n] - x_q[n]$$

Para un cuantizador uniforme con redondeo al nivel más cercano:

$$-\frac{\Delta}{2} \leq e_q \leq \frac{\Delta}{2}$$

El error es intrínseco e inevitable: es el precio de representar infinitos valores con un conjunto finito.

3.2.3. Codificación

Convierte cada nivel cuantizado $x_q[n]$ en una palabra binaria de n bits. Es una etapa sin pérdida: no agrega error.

Algunos códigos

Tipo	Uso	Ventaja
Binario natural	General	Simple, el k -ésimo nivel se codifica convirtiendo el número k a su equivalente binario de n bits
Gray	Conversión A/D	Solo 1 bit cambia entre niveles adyacentes, lo que da menos errores de transición
Complemento a 2	Señales con signo	Permite representar negativos
Signo-magnitud	Señales con signo	Intuitivo

Ej., para $n = 3$ y el rango $[0v, 4v]$: $\Delta = \frac{4v-0v}{2^3} = 0.5$

k	V_{min}	$x_q[k]$	V_{max}	Código binario	Código Gray
0	0	0.25	0.5	000	000
1	0.5	0.75	1	001	001
2	1	1.25	1.5	010	011
3	1.5	1.75	2	011	010
4	2	2.25	2.5	100	110
5	2.5	2.75	3	101	111
6	3	3.25	3.5	110	101
7	3.5	3.75	4	111	100

Tasa de bits resultante

Después de los 3 pasos, la tasa de bits (bits por segundo) del flujo digital es:

$$R = f_s * n * c$$

Donde:

- f_s = frecuencia de muestreo (Hz)
- n = bits por muestra
- c = número de canales (muchas aplicaciones en vida real usan múltiples señales a la vez y al digitalizar se digitalizan en un mismo proceso)

Ejemplos:

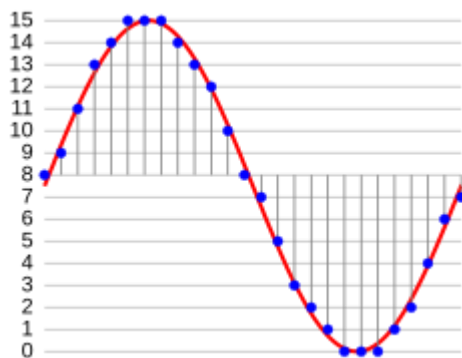
Aplicación	f_s (kHz)	n (bits)	canales	R (kbps)
Telefonía	8	8	1	64
Telefonía DPCM	8	4	1	32
Audio CD	44.1	16	2	1411

Audio DVD	96	24	6	13824
Audio estudio	192	32	2	12288

3.3. Codificación PCM, DPCM, ADPCM, Modulación delta. (mejorar)

3.3.1. Codificación PCM

La modulación en impulsos codificados o pulsecode modulation (PCM), es el codificador de forma de onda temporal más simple y de uso más extendido. Funciona muestreando una señal (generalmente de audio) a una velocidad constante y cuantificando directamente la amplitud de cada muestra obtenida.



En la grafica se puede ver la señal analógica (marcada en rojo) y luego se puede ver como a un intervalo fijo tomamos muestras de la amplitud (los puntos azules), además se puede ver que estas muestras no siempre coinciden con la señal analógica, dado que están limitadas a los valores que se pueden representar con 4 bits (del 0 al 15).

3.3.2. Codificación DPCM

La modulación diferencial por impulsos codificados o differential pulse code modulation (DPCM), es una técnica utilizada cuando la señal a codificar (como la voz humana) presenta un alto grado de correlación o similitud entre muestras adyacentes. En lugar de codificar el valor absoluto de cada muestra, el DPCM calcula y codifica únicamente la diferencia entre muestras consecutivas. Como el rango dinámico de estas diferencias es significativamente menor que el de la señal original, se necesitan menos niveles de cuantificación y, en consecuencia, menos bits por muestra.

3.3.3. Codificación ADPCM

La modulación diferencial adaptativa por impulsos codificados o adaptive differential pulse-code modulation (ADPCM), consiste en una evolución del DPCM donde el cuantificador pasa a ser adaptativo. Esto significa que el paso de cuantificación no es constante, sino que se agranda o achica dinámicamente dependiendo del nivel de la propia señal diferencial a cuantificar. El objetivo es reducir el error de cuantificación usando los mismos bits, o reducir los bits necesarios manteniendo la calidad. La adaptación se realiza mediante dos estrategias: *feedforward* (los niveles se estiman en el codificador y se envían como información extra al decodificador, aumentando el tráfico) o *backward* (se calculan a partir de la señal pasada que ya tienen ambos extremos, sin usar bits extra, aunque es más sensible a errores de transmisión). A veces, el filtro predictor también es adaptativo.

3.3.4. Codificación delta

DPCM de 1 bit que codifica si la señal sube o baja.

UNIDAD Nº4 - COMPACTADORES

4.1. Conceptos básicos

La compresión busca convertir un flujo de datos de entrada (como un archivo) en otro de menor tamaño. Esto consigue principalmente: reducir el almacenamiento necesario y reducir el tiempo de transmisión.

El principio central de la compresión es **evitar la redundancia** presente en los datos. Esto se logra mediante un proceso de codificación inteligente donde se asignan códigos más cortos a los mensajes, símbolos o frases que aparecen con mayor frecuencia en la fuente.

Para lograrlo, empleamos un **código**:

- **Definición:** Es una correspondencia biunívoca entre un conjunto de símbolos (o secuencias de símbolos) generados por una fuente, llamado alfabeto fuente, y secuencias de símbolos pertenecientes a un alfabeto código (palabras código).
- En términos prácticos, funciona como un diccionario de conversión (*palabras entrada* ↔ *palabras código*) donde el objetivo es que las palabras de código/salida sean, en promedio, más pequeñas que las de entrada para lograr la compresión (“en promedio” por qué es común que se expanda el tamaño de las palabras menos probables).

Filosofías generales de codificación

Para tratar distintos tipos de datos (como texto, imágenes o sonido), se plantean tres filosofías generales de codificación:

1. **RLE (Codificación de Longitud del Recorrido):** Es sumamente útil para secuencias con caracteres repetitivos o áreas uniformes (como en imágenes). Funciona agrupando cadenas de píxeles o caracteres idénticos y reemplazándolos por un contador de repeticiones y el valor del símbolo. Ej: aaabccddddd, se puede comprimir como 3abcc4d.
2. **Métodos estadísticos:** Se basan puramente en las probabilidades de aparición de los símbolos para definir el código óptimo. Algoritmos como Huffman y Shannon-Fano entran en esta categoría, asignando sistemáticamente palabras más cortas a los símbolos con mayor probabilidad.
3. **Métodos de diccionario:** Consisten en la sustitución de cadenas de caracteres por punteros o índices que hacen referencia a un diccionario de frases. Este diccionario se va construyendo dinámica o estáticamente con las secuencias ya leídas (algoritmos de la familia LZ, como LZ77, LZ78 o LZW).

Clasificación de los Compresores

Además, los programas encargados de realizar este proceso, llamados **compresores**, se pueden clasificar según dos criterios fundamentales:

A. Según su Adaptabilidad:

- **Adaptativos (Dinámicos):** Modifican su comportamiento y su modelo de codificación de forma dinámica dependiendo de los datos que van leyendo (por ejemplo, el algoritmo LZW o Huffman dinámico).
- **No adaptativos (Estáticos):** Utilizan un modelo y un diccionario fijo que no cambia durante el proceso de compresión.

B. Según la Pérdida de Datos:

- **Sin pérdidas (Lossless):** El proceso es totalmente reversible; al descomprimir, se recupera una copia bit a bit exactamente idéntica al archivo original. Es obligatorio en archivos de texto o datos ejecutables.
- **Con pérdidas (Lossy):** El algoritmo no permite recuperar con exactitud los valores de cada muestra original, por lo que no se recupera el archivo exacto. Sin embargo, hay entornos donde esta pérdida de información es aceptable, como en audio o video, ya que se eliminan frecuencias o detalles imperceptibles para el ojo u oído humanos, logrando tasas de compresión muchísimo mayores (como ocurre en JPEG o MP3).

El respaldo matemático

Todas estas filosofías y compresores están regidos por las métricas y leyes matemáticas de la Teoría de la Información que mencionamos anteriormente:

- **Razón de compresión:** Es la proporción entre el tamaño del archivo resultante y el original. Para que exista compresión real, este valor debe ser menor a 1.

$$\text{Razon de compresion} = \frac{\text{Tamaño de salida}}{\text{Tamaño de entrada}}$$

- **Factor de compresión:** Es el valor inverso a la razón de compresión y señala cuántas veces se redujo el archivo. Debe ser mayor a 1.

$$\text{Factor de compresion} = \frac{\text{Tamaño de entrada}}{\text{Tamaño de salida}}$$

- **Entropía $H(S)$:** es el límite matemático absoluto de la compresión, el teorema de Shannon demuestra que la longitud media (L) de las palabras de nuestro código compresor jamás podrá ser inferior a la entropía de la fuente. Si lo fuera, inevitablemente es una compresión con pérdidas.

4.2. Clasificación de códigos (formatear mejor)

- Según su longitud
 - **Códigos de bloque (longitud fija):** Son aquellos en los que todas las palabras código (de salida) tienen la misma longitud fija y predeterminada. Tiene la ventaja de que su lectura es muy sencilla, pero no es muy eficiente para comprimir información. (Ej.: código ASCII de 8 bits)
 - **Códigos no bloque (longitud variable):** Son aquellos en los que no todas las palabras código tienen la misma longitud. Su objetivo principal es asignar codificaciones más cortas a los símbolos con mayor probabilidad de ocurrencia en la fuente, logrando así reducir la longitud promedio y compactar el tamaño de la información.

- **Según la unicidad de sus símbolos**

- **Códigos singulares:** Ocurre cuando a más de un símbolo de la fuente se le asigna exactamente la misma palabra código. Esto genera una ambigüedad total y los hace inútiles para la transmisión fiel. Dado que al recuperar el texto original, cuando leemos la palabra código no sabemos cuál de los símbolos originales sería el correcto. (El termino “singular” viene del algebra lineal, una transformación es **singular** cuando "destruye" o pierde información, haciendo imposible invertir el proceso para recuperar los datos originales).
- **Códigos no singulares:** Es aquel donde todas las palabras código son distintas entre sí. Es decir, cualquier secuencia finita del código corresponde, a lo sumo, a un solo símbolo de la fuente.
 - Aunque un código sea no singular, puede presentar problemas. Si transmitimos una secuencia continua de palabras, la combinación de estas podría decodificarse de múltiples formas válidas, generando confusión. Por ello, se necesita una restricción mayor.

- **Según la capacidad de decodificar secuencias continuas**

Para resolver el problema de las secuencias continuas, se define una nueva subclase dentro de los códigos no singulares:

- **Código Unívoco (Unívocamente decodificable):** Es un código donde, para cada secuencia codificada recibida, existe una y solo una posible descomposición o traducción al alfabeto fuente original. Garantiza que el mensaje completo se interpretará sin ninguna ambigüedad.

- **Según el momento en el que pueden traducirse (Subclase de los códigos unívocos)**

- **Códigos instantáneos/prefijo:** Son un tipo de código unívoco donde es posible decodificar cada palabra de una secuencia inmediatamente, sin necesidad de esperar y conocer los símbolos que la suceden.

La condición esencial para que un código sea instantáneo es que cumpla con la regla de que ninguna palabra código sea prefijo de otra.
- **Códigos no instantáneos:** Aunque permiten una lectura unívoca, el receptor no puede traducir el símbolo de inmediato y experimenta una demora en la decodificación, ya que necesita leer bits o delimitadores posteriores para descartar posibilidades y saber dónde finaliza una palabra código.

4.3. Desigualdad de Kraft (1949)

(revisar, creo que falta clasificación de código prefijo para explicar esto)

Es la regla de oro al construir nuestros códigos (ya sean estadísticos o de diccionario) para asegurarnos de que la correspondencia entrada salida genere secuencias que puedan ser leídas y decodificadas de forma única y sin ambigüedad en el receptor. (Código instantáneo/prefijo)

Todo código instantáneo base r deberá satisfacer, respecto de las longitudes individuales $l_1, l_2, \dots, l_q$ de cada palabra código la siguiente expresión:

$$\sum_{i=1}^q r^{-l_i} \leq 1$$

Significado:

- Si $\sum r^{-l_i} = 1$: El código es instantáneo y el árbol de prefijos está lleno
- Si $\sum r^{-l_i} < 1$: El código es instantáneo y aun queda espacio en el árbol de prefijos
- Si $\sum r^{-l_i} > 1$: No existe código instantáneo con esas longitudes

Si el código es instantáneo $\rightarrow$ la desigualdad se cumple

Si la desigualdad se cumple $\rightarrow$ existe un código instantáneo con esas longitudes

4.4. Desigualdad de McMillan (1956)

Es la misma que la de Kraft, pero con otra aplicación, se usa como una condición necesaria pero no suficiente para la obtención de un código unívocamente decodificable. Es decir: Dice que **todo código unívocamente decodificable debe cumplir Kraft**.

Si un código no cumple Kraft, automáticamente **no puede ser unívoco**.

Aquí **sí puede ocurrir** que una palabra sea prefijo de otra, siempre que el conjunto completo siga siendo decodificable sin ambigüedad.

4.5. Longitud promedio de un código

Media ponderada de las longitudes l_i de las palabras código en base a su probabilidad de aparición p_i :

$$\bar{L} = \sum_{i=1}^q p_i \cdot l_i$$

4.6. Eficiencia y redundancia del código

Eficiencia: $\eta = \frac{H(X)}{\bar{L}}$

Redundancia: $R = 1 - \eta = 1 - \frac{H(X)}{\bar{L}}$

Primer Teorema de Shannon (Codificación de Fuente):

Para cualquier código unívocamente decodificable L existe un código tal que: $H(X) \leq \bar{L} < H(X) + 1$

En términos de eficiencia: $\frac{H(X)}{H(X)+1} < \eta \leq 1$

Código Óptimo: Código que minimiza la longitud promedio $\bar{L}$ para una fuente dada. Por ej, el código de Huffman es óptimo.

4.7. Código de Huffman base r (quitar y cambiar por código de ejemplo básico y fijo, tal vez antes de lo de longitud promedio para usarlo como ejemplo)

4.8. Extensión de orden n (revisar, mover?)

La extensión de un código consiste en agrupar los símbolos de una fuente de información en bloques o secuencias de longitud n para codificarlos en conjunto, en lugar de hacerlo símbolo por símbolo. Si la fuente original posee un alfabeto de q símbolos, la nueva fuente extendida (su extensión de orden n) pasará a tener un alfabeto de q^n símbolos distintos.

Ej: fuente A original de alfabeto $\{a, b\}$ con probabilidad 0.5 para cada símbolo, su extensión A^2 de orden 2 sería: $\{aa, ab, ba, bb\}$ con probabilidad 0.25 para cada símbolo.

El propósito fundamental de trabajar con extensiones es mejorar la eficiencia de la codificación y acercarse al límite teórico ideal. Según el teorema de codificación de Shannon, un código compacto (como el de Huffman estándar) sufre de un pequeño desperdicio por redondeo, pudiendo requerir hasta 1 bit extra por encima de la entropía de la fuente. Al agrupar n símbolos, esa sobrecarga se diluye sobre el bloque completo, haciendo que el error por símbolo original se reduzca a $\frac{1}{n}$.

Entropía: $H(X^n) = n * H(X)$

Longitud promedio por símbolo original: $\bar{L}_n = \frac{\bar{L}_{total}}{n}$

Límite cuando $n \rightarrow \infty$: $\lim_{n \rightarrow \infty} \bar{L}_n = H(X)$

Esto significa que podemos acercarnos arbitrariamente a la entropía agrupando símbolos.

UNIDAD Nº5 - FUENTES DEPENDIENTES, CÓDIGOS DIVERSOS

5.1. Introducción.

Hasta este punto de la teoría, las fuentes de información suelen modelarse asumiendo que los símbolos se emiten de forma independiente (fuentes de memoria nula). Sin embargo, en la mayoría de las aplicaciones prácticas, como el texto en un lenguaje natural, esta suposición es demasiado limitada. La probabilidad de que aparezca un símbolo determinado depende en gran medida de los símbolos o secuencias de símbolos que lo precedieron. Para describir estadísticamente este tipo de secuencias con dependencias, se introduce el concepto de fuentes con memoria.

Para ejemplificar esto de forma sencilla: en la sintaxis del lenguaje castellano, dada la aparición de la letra "q", podemos afirmar con casi total seguridad que a continuación vendrá la letra "u".

Matemáticamente, esto significa que la probabilidad condicional $P(u|q) \approx 1$. Esta restricción en el empleo de los símbolos provoca que la aparición de un símbolo sea mucho menos sorpresiva y, en consecuencia, la cantidad media de información generada sea menor a la que tendríamos si los símbolos fueran totalmente independientes. Lo que nos permitiría comprimir en gran manera sin perder información.

5.2. Procesos o fuente de MARKOV

Un **proceso o fuente de Markov** es un modelo estadístico para fuentes con memoria. En una fuente de Markov de orden m , la probabilidad de emisión de un símbolo cualquiera viene determinada exclusivamente por los m símbolos que lo preceden.

$$P(X|X_{m-1}, X_{m-2}, \dots, X_0)$$

Están compuestas por:

- Un alfabeto A , con q símbolos
- Un conjunto de estados Σ , de tamaño q^m , dado que ahora un estado σ se describe como una cadena de los últimos m símbolos $(X_{m-1}, X_{m-2}, \dots, X_0)$
- Un set de transiciones entre estados: $\sigma_i \rightarrow \sigma_j$
- Dos sets de probabilidades
 - El primer es el set que define las probabilidades condicionales de las transiciones $P(X|\sigma_i)$ (o lo que es lo mismo $P(X|X_{m-1}, X_{m-2}, \dots, X_0)$), pero esto requiere conocer los m símbolos anteriores
 - El segundo es el set **probabilidades iniciales** es el que se utiliza cuando no se conocen los m símbolos anteriores, contiene la probabilidad de que cada estado σ_i aparezca (es decir la probabilidad de que aparezca la secuencia de símbolos $X_{m-1}, X_{m-2}, \dots, X_0$)

La matriz de transición

El comportamiento estadístico y las transiciones entre estos q^m estados se describen completamente mediante una **matriz de transición**, de forma similar a la de un **canal**. En esta matriz cuadrada, cada elemento $P(S_j|S_i)$ representa la probabilidad condicional de que el sistema pase del estado S_j dado que actualmente se encuentra en el estado S_i .

Esta matriz sigue cumpliendo la condición de que estando en un estado S_i , el sistema debe transicionar a alguno de los estados posibles. Por ende, la suma de los elementos de cualquier fila de la matriz debe dar 1.

$$\sum_{j=1}^{q^m} P(S_j|S_i) = 1$$

5.3. Proceso de MARKOV ergódico

La **ergodicidad** y la **distribución estacionaria** nos dicen cómo se comporta el sistema cuando se lo deja evolucionar durante un tiempo prolongado.

Ergodicidad: Un sistema es ergódico si, a lo largo del tiempo, explora todo su espacio de estados accesible de manera uniforme.

En términos simples, significa que observar una trayectoria suficientemente larga de un sistema es equivalente a analizar todas sus posibles configuraciones.

Esto implica que **el promedio temporal de una magnitud** (observando un sistema durante mucho tiempo) es igual al **promedio de conjunto** (considerando muchas realizaciones independientes del sistema).

Características de la no-ergodicidad

- **Dependencia del estado inicial:** El resultado depende fuertemente del estado inicial del sistema.
- **estados inaccesibles:** El sistema puede quedar atrapado en una región del espacio de estados y nunca explorar otras.
- **Promedios distintos:** El promedio temporal de una variable no coincide con el promedio estadístico calculado sobre todas las realizaciones posibles.

5.4. Distribución estacionaria de un proceso de Márkov ergódico

La consecuencia más importante de que un proceso de Markov sea ergódico es que el sistema, tras un número elevado de transiciones, **alcanza el equilibrio estadístico**.

Cuando esto ocurre, la probabilidad de encontrar a la fuente en un estado determinado deja de variar con el tiempo y **se vuelve totalmente independiente de la distribución inicial** con la que arrancó el sistema.

A este conjunto único de probabilidades a largo plazo se le llama **distribución estacionaria**.

Métodos para obtener la matriz estacionaria

Método asintótico o iterativo

La manera más sencilla, consiste en multiplicar la matriz de transición por sí misma (P^2), elevándola a una potencia n lo suficientemente grande (P^n). Al tratarse de una fuente ergódica, al incrementar el exponente, los valores convergen hacia una matriz límite y dejan de admitir modificaciones. El proceso se detiene cuando iterar no produce cambios significativos, es decir $P^n \approx P^{n-1}$.

Cada valor de la matriz resultante indica la probabilidad de transicionar de un estado original σ_i a un estado destino σ_j en exactamente n pasos.

Método analítico

Se basa en plantear algebraicamente el comportamiento de equilibrio del sistema. Se genera un sistema de ecuaciones lineales de la forma:

$$P(S_j) = \sum_{i=1}^{q^m} P(S_i) * P(S_j|S_i)$$

Sin embargo, este sistema de ecuaciones por sí solo no tiene una solución única. Para poder resolverlo y encontrar los valores exactos, es obligatorio añadir la **condición de normalización** de las probabilidades (la certeza de que la fuente tiene que estar en alguno de los estados):

$$\sum_{i=1}^{q^m} P(S_i) = 1$$

Al resolver el sistema por cualquier técnica matemática de resolución de ecuaciones, se obtiene el vector estacionario exacto.

Método de algebra lineal

Desde el punto de vista del álgebra de matrices, una distribución invariante o estacionaria es, por definición, un **vector propio (eigenvector)** de la matriz de probabilidades de transición asociado al **valor propio (eigenvalue) de magnitud 1**. Puesto que todas las filas de la matriz de transición suman 1, la matriz siempre admite este autovalor. Al calcular el vector propio normalizado (de forma que sus componentes sumen 1) asociado a dicho autovalor, se halla directamente la distribución estacionaria de la fuente.

5.3. Codificación eficiente de un proceso de MARKOV.

Para lograr una codificación eficiente en un proceso de Markov, la estrategia central consiste en abandonar el uso de un único diccionario o código general para todo el alfabeto, y en su lugar **generar un código distinto para cada estado de la fuente**.

Dado que en una fuente de Markov la aparición de un símbolo depende fuertemente del estado anterior en el que se encontraba la fuente, las probabilidades de emisión cambian constantemente según el contexto. Aprovechar estas probabilidades condicionales permite asignar palabras código más cortas a las transiciones más probables desde cada estado específico, en lugar de basarse en la probabilidad global promedio del símbolo.

Procedimiento de Codificación

1. **Múltiples diccionarios:** Se considera la fuente como un proceso de Markov de orden m y se construye un código independiente (por ejemplo, utilizando el algoritmo estadístico de Huffman) para las transiciones posibles desde *cada uno* de los estados. Si el modelo tiene varios estados posibles, se generará una tabla de codificación diferente para cada uno de ellos.
2. **Arranque (Estado Inicial):** Como la compresión requiere conocer el estado actual para elegir el diccionario correcto, el primer símbolo del mensaje (que define el estado inicial) debe codificarse obligatoriamente mediante un formato de bloque conocido (de longitud fija) o un código base, ya que aún no hay historia previa registrada.
3. **Transiciones:** A partir del estado actual ya establecido, se lee el siguiente símbolo y se codifica utilizando *exclusivamente* el diccionario correspondiente a la transición desde el estado en el que se está. Una vez emitido el código, la fuente avanza, el nuevo símbolo pasa a formar parte de la memoria del sistema estableciendo el nuevo estado, y el proceso se repite con el diccionario de ese nuevo estado.

Ecuaciones y definiciones matemáticas

Para medir matemáticamente la eficiencia de este método frente a los códigos de memoria nula, se definen las siguientes magnitudes:

- **Longitud Promedio por Estado ($L_M(S_i)$):** Para un estado particular S_i , la longitud media de codificación se obtiene calculando la sumatoria de las longitudes $l(S_j)$ de las palabras código asignadas a cada posible símbolo futuro S_j , ponderadas por la **probabilidad condicional** de dicha transición $P(S_j|S_i)$:

$$L_M(S_i) = \sum_{j=1}^q l(S_j) * P(S_j|S_i)$$

- Longitud promedio total de markov (L_M): La longitud promedio final de todo el proceso

$$L_M = \sum_{i=1}^{q^m} P(S_i) * L_M(S_i)$$

- Entropía de markov $H(S^M)$: Cota inferior de entropía para una fuente de markov.

$$H(S^M) = - \sum_{j=1}^{q^m} P(S_{j1}, S_{j2}, \dots, S_{jm}) * \sum_{i=1}^q P(S_i | S_{j1}, S_{j2}, \dots, S_{jm}) * \log_2 P(S_i | S_{j1}, S_{j2}, \dots, S_{jm})$$

Conclusión fundamental

Puesto que conocer el estado previo siempre aporta información y reduce la incertidumbre frente al siguiente evento, **la entropía de una fuente modelada por Markov es siempre menor o igual a la entropía de esa misma fuente tratada como independiente** ($H(S^M) \leq H(S)$)

Por consiguiente, la codificación eficiente que aprovecha los estados de Markov logra que su longitud promedio L_M se ubique por debajo de la cota teórica impuesta por $H(S)$, consiguiendo reducir drásticamente el tamaño del archivo final en comparación con lo que se lograría al asignar un único diccionario estático para todos los símbolos.

La desventaja es que como el receptor debe conocer los diccionarios hay mas información que transmitir antes de que pueda descomprimir.

5.4. Extensión de un proceso de MARKOV. Codificación Predictiva.

Al igual que ocurre con las fuentes de memoria nula, es posible agrupar los símbolos generados por una fuente de Markov en bloques para mejorar la eficiencia de la codificación.

- **Definición formal:** Sea S una fuente de información de Markov de orden m , con un alfabeto de q símbolos discretos y un conjunto de probabilidades condicionales conocidas. La **extensión de orden n** de esta fuente, denotada como S^n , es una nueva fuente de Markov que agrupa las salidas en bloques n de símbolos.
- **Nuevo Alfabeto y Orden:** Esta nueva fuente extendida tendrá un alfabeto mucho mayor, compuesto por símbolos distintos (cada nuevo símbolo representa una secuencia de símbolos originales). El orden de memoria de esta nueva fuente extendida se redefine como un valor $\lceil n/m \rceil$, donde $\lceil \cdot \rceil$ es decir, es el menor número entero que sea igual o superior al cociente.
- **Entropía de la extensión:** El cálculo matemático de las probabilidades conjuntas demuestra que la entropía de la extensión de orden n de una fuente de Markov de primer orden (y extensible a orden n) es exactamente veces la entropía de la fuente original:

$$H(S^n) = nH(S)$$

- **Aplicación práctica (Teorema de Shannon):** El propósito fundamental de extender una fuente de Markov es **aumentar la eficiencia del código**. Al aplicar el Primer Teorema de Shannon a procesos dependientes, se demuestra que si codificamos bloques de longitud n en lugar de símbolos individuales, la longitud media por símbolo original ($\frac{L_n}{n}$) puede acercarse a la cota teórica impuesta por la entropía condicional de Markov $H(S)$ tanto como se desee, siempre que se elija un n suficientemente grande.

Codificación predictiva (a esto no se refería el profe, mover a unidad 3 y corregir?)

La codificación predictiva es una técnica que aprovecha directamente la memoria y la altísima dependencia estadística (correlación) que existe entre muestras adyacentes en ciertas fuentes continuas o discretizadas, como las señales de voz, audio o imágenes. En lugar de codificar el valor absoluto de cada muestra, el sistema intenta adivinar el siguiente valor basándose en el historial de los estados anteriores.

- **El Módulo Predictor:** Es un sistema matemático que, al recibir un historial de la señal en su entrada, proporciona a la salida una estimación o **predicción** de cuál será la muestra siguiente de la señal.
- **Modulación Diferencial por Impulsos Codificados (DPCM):** Es la implementación clásica de la codificación predictiva. Puesto que las muestras adyacentes son muy similares, el codificador DPCM resta el valor real de la muestra actual frente al valor estimado por el predictor. Al canal de comunicaciones se transmite únicamente la diferencia o error de predicción, denotado como $e[n]$, en lugar de la muestra completa.
 - **Ventaja matemática:** El rango dinámico de la señal diferencia $e[n]$ es drásticamente más pequeño que el rango de la señal original $x[n]$. Al ser un valor pequeño, se necesitan muchos menos niveles de cuantificación para codificar $e[n]$ con el mismo error, lo que reduce radicalmente el número de bits por muestra transmitidos.
- **Modulación Diferencial Adaptativa (ADPCM):** Es una evolución del DPCM donde el paso de cuantificación utilizado para medir el residuo $e[n]$ no es fijo, sino que es **variable y se adapta dinámicamente** a la propia señal. Esto permite reducir aún más el error de cuantificación o el número de bits por muestra requeridos.
- **Codificación por Predicción Lineal (Vocoder LPC):** Es un codificador predictivo paramétrico específico para la voz humana. En lugar de transmitir la forma de onda, el algoritmo LPC modela el tracto vocal humano mediante un filtro lineal. En este modelo, la señal de voz actual $s[n]$ se aproxima matemáticamente como una combinación lineal de las muestras anteriores más una señal de error $e[n]$:

$$s[n] \approx \sum_{i=1}^p a_i * s[n - i] + e[n]$$

El codificador calcula y transmite los coeficientes a_i (que minimizan el error de predicción $e[n]$), junto con parámetros de la fuente como la altura tonal (pitch) y la sonoridad, logrando tasas de compresión bajísimas (por ejemplo, 2,4 kbps).

5.5. Código GRAY. Codificación y decodificación. (mover?)

El código Gray es un método de codificación posicional en el cual **los números sucesivos cambian en una única posición (un solo bit)**. Este código no fue diseñado con fines de compresión de datos, sino para resolver un problema físico crucial en la adquisición y medición de datos: la conversión de señales analógicas a digitales.

Imagina un dispositivo analógico, como una rueda giratoria segmentada que lee valores mediante escobillas de contacto. Si la rueda usara codificación binaria directa, al pasar del sector 011 (3 en decimal) al 100 (4 en decimal), los tres bits deben cambiar simultáneamente. Si las escobillas no

están perfectamente alineadas y leen exactamente en el momento de la transición, el sistema podría registrar resultados con un error gigantesco (como 111 o 000). Al utilizar el código Gray, puesto que de un sector al siguiente solo varía un bit, la lectura en el límite entre dos sectores **garantiza que el error nunca será mayor a una posición** (es decir, el sistema leerá o bien el sector anterior o el siguiente, sin resultados disparatados).

Construcción (Codificación) La construcción de este código se puede realizar de forma inductiva (reflexiva). Para un código de un dígito, se tienen los estados 0 y 1. Para pasar a dos dígitos, se escriben estos símbolos, debajo se escriben en orden inverso (reflejado), y se les añade un 0 a los originales y un 1 a los reflejados. Esta progresión garantiza que, en la transición de la primera mitad a la segunda, solo cambie el dígito inicial (el que se acaba de añadir), manteniendo la regla de un solo cambio por salto. Ej.:

Caso base	Paso intermedio	Código gray de longitud 2	Paso intermedio	Código gray de longitud 3
0	0	00	00	000
1	1	01	01	001
	1	11	11	011
	0	10	10	010
			10	110
			11	111
			01	101
			00	100

Además, hay una manera de obtener el valor decimal original sin recurrir a una tabla que contenga todos los valores, aquí hay un ejemplo para código gray de 3 bits.

$$Bit\ 2\ salida = Bit\ 2$$

$$Bit\ 1\ salida = Bit\ 1 \oplus Bit\ 2\ salida$$

$$Bit\ 0\ salida = Bit\ 0 \oplus Bit\ 1\ salida$$

El bit más significativo siempre se deja intacto y luego cada bit se calcula sucesivamente haciendo un XOR entre sí mismo y el bit anterior. A simple vista se puede ver como invertir la posición siguiente a cada 1 presente de forma sucesiva.

Código binario	Código gray de longitud 3	Aplicando inversión
000	000	000
001	001	001
011	011	011 → 010
010	010	010 → 011
110	110	110 → 100
111	111	111 → 101
101	101	101 → 111 → 110
100	100	100 → 110 → 111

5.6. Codification Prediction by Partial Matching (PPM)

La codificación por **Prediction by Partial Matching (PPM)** o Predicción por Coincidencia Parcial es un método de compresión estadística que predice la probabilidad de que aparezca el siguiente

carácter analizando los caracteres que lo precedieron (su contexto) a través de una colección de modelos de cadenas de Markov (también llamados n -gramas) de distintos ordenes.

Proceso para Codificar (Comprimir) vs. Decodificar (Descomprimir)

Paso	Codificación	Decodificación
1. Identificar el contexto	Examina los últimos n caracteres procesados (contexto tamaño n).	Mira los últimos caracteres que ya ha logrado decodificar (mismo contexto n)
2. Evaluar frecuencias	Busca el carácter en la tabla del contexto actual. - Frecuencia > 0: Pasa la probabilidad al codificador aritmético para emitir los bits del carácter. - Frecuencia = 0: Emite un carácter de escape <i>ESC</i>	Intenta interpretar los bits recibidos usando las probabilidades del contexto actual. Si decodifica el carácter real: El proceso de búsqueda termina con éxito. Si decodifica un ESC: Sabe que el codificador no pudo predecir el carácter en este nivel.
3. Mecanismo de escape	Si emitió un ESC, reduce el contexto a uno más pequeño ($n - 1$) y vuelve al Paso 2.	Si leyó un ESC, reduce su contexto a $n - 1$ e intenta decodificar nuevamente con la tabla de ese nivel inferior.
4. Finalización y actualización	Una vez escrito el carácter en el flujo, actualiza las frecuencias en todos sus modelos de n -gramas.	Una vez recuperado el carácter real, lo escribe en la salida y actualiza sus tablas de frecuencias exactamente igual que el codificador.

Detalle: PPM podría usar cualquier codificación simple de forma interna, pero usa codificación aritmética porque permite aproximar la longitud del mensaje comprimido al límite teórico de la entropía de manera casi perfecta.

¿Qué es un Contexto y cómo varían sus Órdenes?

En PPM, el contexto se refiere simplemente a los caracteres que se encuentran inmediatamente antes de la letra que estamos intentando comprimir. El orden (n) del contexto es el número de caracteres que miramos hacia atrás.

Los contextos almacenan contadores sobre los caracteres vistos en base a ese contexto, ejemplo de un contexto de orden 2:

$$ab: \{a: 5, b: 2, ESC: 1\}$$

Los contadores del contexto nos permiten obtener las probabilidades actuales de forma simple:

$$Total\ de\ conteos = 8$$

$$P(a|ab) = \frac{5}{8} \quad P(b|ab) = \frac{2}{8} \quad p(ESC|ab) = \frac{1}{8}$$

El contador de *ESC* (es el mecanismo de escape) no es siempre 1, esto depende de que versión de PPM estamos tratando, en PPM-A si se mantiene siempre en 1, en PPM-B es un contador normal (sería igual a la cantidad de símbolos distintos de ESC en el contexto pues tienen que pasar por el para ser agregados al contexto) y el PPM-C es el mas utilizado donde funciona como contador pero cambia la forma de calcular las probabilidades en base a los contadores.

En una codificación PPM, el n debe ser mayor o igual a 1, si no lo fuera, no sería mejor que una codificación simple. Además este siempre tiene contextos orden 0 y orden -1 .

El contexto de orden 0 funciona llevando un contador para la aparición individual de cada letra similar a una codificación normal, ej: $\{a: 30, b: 27, ESC: 1\}$

El contexto de orden -1 es el "fondo de seguridad" del sistema, tiene una forma fija, la cual contiene todos los símbolos que pueden aparecer con una probabilidad (o lo que es lo mismo, contadores) uniforme:

$$\{a: 1, b: 1, c: 1, d: 1\}$$

Ejemplo corto de codificación

Para codificar siempre se miran primero los n caracteres anteriores, siguiendo el ejemplo anterior, supongamos que estamos en el contexto ab (orden 2):

- Si recibimos un símbolo a , en ese caso se le pasa al codificador aritmético la probabilidad $\frac{5}{8}$ para su codificación, y después se actualiza el contexto: $ab: \{a: 6, b: 2, ESC: 1\}$
- Si recibimos un símbolo b , se codifica la probabilidad $\frac{2}{8}$, y se actualiza el contexto $ab: \{a: 5, b: 3, ESC: 1\}$
- Si recibimos un símbolo c , nunca vista en este contexto, no hay ninguna probabilidad para pasarle al codificador aritmético, lo cual rompería la codificación, aquí se recurre al mecanismo de escape, y se emite un ESC , haciendo que ahora la letra c deba codificarse bajo el contexto b (orden 1), en caso de que no este presente ahí se usara el nivel 0 y si tampoco esta en el nivel 0, en el nivel -1, una vez se termina la codificación en niveles inferiores, se actualizan todos los contextos por los cuales se paso, agregando el símbolo.

UNIDAD Nº6 - ENTROPIA Y CODIFICACION

6.1. Entropía y codificación.

La entropía (H) representa el límite inferior teórico de la longitud promedio (L) de un código para una fuente dada: $\bar{L} \geq H$. Lo que nos dice que no es posible comprimir la información de una fuente por debajo de su información promedio sin sufrir una **pérdida de información**.

Un código se considera óptimo o 100% eficiente cuando su longitud promedio es exactamente igual a la entropía $H = L$.

6.2. Compactadores estáticos.

Son compactadores que para funcionar necesitan la información de las probabilidades de cada símbolo de antemano.

6.2.1. Códigos de HUFFMAN

Los códigos de Huffman son un método estadístico de compresión de datos sin pérdida que genera códigos de longitud variable, instantáneos y óptimos (minimiza la longitud promedio del código lo más posible para una fuente)

El algoritmo para construirlos diseña un árbol binario "de abajo hacia arriba" (desde las hojas hasta la raíz) siguiendo estos sencillos pasos:

1. Se ordena el alfabeto de símbolos en forma decreciente según sus probabilidades de ocurrencia.
2. Se toman los r (la r se refiere a la base del código objetivo) símbolos menos probables y se combinan para crear un nuevo nodo o "símbolo" virtual, cuya probabilidad es la suma de las probabilidades de los r originales.
3. Este proceso de asociar las r probabilidades menores se repite una y otra vez, reduciendo el alfabeto en cada paso, hasta que solo queda un único nodo con una probabilidad total de 1 (la raíz del árbol).
4. Finalmente, se asignan los símbolos (por ejemplo, con $r = 2$ y el alfabeto binario, se asigna, 0 a la rama izquierda y 1 a la derecha) y se leen los caminos desde la raíz hacia abajo para obtener el código definitivo de cada símbolo original.

Ejemplo:

Dada la fuente $P(s_1) = 0.2$ $P(s_2) = 0.4$ $P(s_3) = 0.1$ $P(s_4) = 0.1$ $P(s_5) = 0.2$

Primera parte: AGRUPAMIENTO (Se lee hacia abajo)	Segunda parte: ASIGNACIÓN (Se construye hacia arriba)
$s_2(0.4)$ $s_1(0.2)$ $s_5(0.2)$ $s_3(0.1)$ $s_4(0.1)$	$s_2 = 1$ $s_1 = 01$ $s_5 = 000$ $s_3 = 0010$ $s_4 = 0011$
$s_2(0.4)$ $s_1(0.2)$ $s_5(0.2)$ $[s_3 + s_4](0.2)$	$s_2 = 1$ $s_1 = 01$ $s_5 = 000$ $[s_3 + s_4] = 001$
$s_2(0.4)$ $[s_5 + s_{3,4}](0.4)$ $s_1(0.2)$	$s_2 = 1$ $[s_5 + s_{3,4}] = 00$ $s_1 = 01$
$[s_1 + s_{5,3,4}](0.6)$ $s_2(0.4)$	$[s_1 + s_{5,3,4}] = 0$ $s_2 = 1$

Nota: Hay momentos en los cuales podríamos haber agrupado de varias formas distintas, pero son todas equivalentes dado que tienen las mismas probabilidades, esto variaría la forma final del código de Huffman, pero los distintos códigos tendrían la misma longitud promedio lo que los hace equivalentes.

6.2.2. Codificación de SHANNON.

Concepto: Este método determina matemáticamente la longitud l_i de cada palabra código en función de la probabilidad individual del símbolo, mediante la desigualdad:

$$\log_r \left(\frac{1}{p_i} \right) \leq l_i \leq \log_r \left(\frac{1}{p_i} \right) + 1$$

Es considerado el método estadístico más sencillo de implementar.

Procedimiento paso a paso:

1. Ordenar y calcular longitudes: Se ordenan los símbolos de mayor a menor según su probabilidad de aparición. Se calcula la longitud l_i que tendrá la palabra código para cada símbolo i mediante la desigualdad.
2. Función Acumulativa (FA): Se calcula una probabilidad acumulada para cada símbolo, sumando las probabilidades de los símbolos anteriores en la lista (el primer símbolo tiene un FA de 0).
3. Conversión a binario: Se convierte el valor fraccionario de la FA a la base de codificación (binario).
4. Asignación del código: Finalmente, para obtener la palabra código definitiva, se toman exactamente los primeros l_i dígitos fraccionarios de la FA expresada en binario.

Ejemplo:

Símbolo	P	l_i	FA	FA en base 2	Código
S_1	0.5	1	0	0.0000	0
S_2	0.25	2	0.5	0.1000	10
S_3	0.15	3	0.75	0.1100	110
S_4	0.10	4	0.90	0.1110	1110

6.2.3. Codificación de FANO.

Se basa en la premisa de que los símbolos o grupos de símbolos que son equiprobables deberían corresponderse con palabras código de igual longitud. Para lograrlo, divide recursivamente el alfabeto en grupos de peso estadístico similar.

Procedimiento paso a paso:

1. Ordenar: Al igual que en Shannon, los símbolos se ordenan en forma decreciente de acuerdo con sus probabilidades.
2. División equilibrada: Se divide o "corta" la lista completa de símbolos en dos mitades (o grupos) buscando que la suma de las probabilidades de cada mitad sea lo más parecida posible, es decir, que ambas se acerquen o igualen a 0.5.
3. Asignación de bits: A todos los símbolos de la primera mitad se les asigna un 0 como primer dígito de su código, y a los de la segunda mitad se les asigna un 1.
4. Recursividad: Se continúa este proceso de dividir cada subgrupo por la mitad (tratando de equilibrar las probabilidades) y asignando nuevos ceros y unos. El proceso finaliza cuando ya no quedan subgrupos por dividir, es decir, cada subgrupo contiene un único símbolo.

Ejemplo:

Inicio (Alfabeto)	$s_1(0.45), s_2(0.25), s_3(0.20), s_4(0.10)$
1ra Iteración	$S_1(0.45) = 0$ $[S_2, S_3, S_4](0.55) = 1$
2da Iteración	$S_1(0.45) = 0$ $S_2(0.25) = 10$ $[S_3, S_4](0.3) = 11$
3ra Iteración	$S_1(0.45) = 0$ $S_2(0.25) = 10$ $S_3(0.2) = 110$ $S_4(0.1) = 111$

6.2.4. Comparación de las tres metodologías anteriores.

Ambos métodos generan códigos instantáneos (libres de prefijos), pero **no siempre garantizan la compactación óptima perfecta** como sí lo logra el algoritmo de Huffman

En la teoría de la información, el rendimiento de estos métodos se relaciona de manera estricta mediante sus longitudes medias (L). La relación matemática es la siguiente:

$$L_{Huffman} \leq L_{Fano} \leq L_{Shannon}$$

Esto indica que Huffman ofrece la mayor eficiencia de compresión, seguido por Fano y, por último, Shannon. Además, se destaca como ventaja que ni el algoritmo de Fano ni el de Huffman expanden jamás el tamaño del archivo original.

6.3. Compactadores dinámicos

A diferencia de los métodos estadísticos estáticos, los compactadores dinámicos (o en línea/adaptativos) no requieren conocer las probabilidades de los símbolos de la fuente de antemano.

En estas codificaciones, tanto el compresor como el descompresor comienzan a trabajar con un modelo inicial (como un árbol vacío) y van modificando las asignaciones de código "sobre la marcha" a medida que los símbolos son leídos y procesados. La gran ventaja de esto es que **no es necesario transmitir la tabla de códigos** al receptor, ya que ambos extremos actualizan sus estructuras simultáneamente y se mantienen sincronizados como espejos. La desventaja es la complejidad.

6.3.1. Codificación HUFFMAN dinámico

Es la metodología de compresión dinámica más estudiada y eficiente. En lugar de reconstruir por completo el árbol de Huffman con cada nuevo carácter, el algoritmo se basa en tres premisas clave y un criterio de balanceo:

- **Actualización de Pesos (Premisa 1):** Tanto el transmisor como el receptor mantienen un árbol en el que cada hoja representa un símbolo con su respectivo 'peso' (la cantidad de veces que ha aparecido). Este peso se incrementa en una unidad cada vez que el símbolo es procesado.
- **El Símbolo Imaginario o NYT (Premisa 3):** Para poder manejar la aparición de caracteres nuevos, el árbol se inicializa con un nodo especial llamado símbolo imaginario (o Not Yet Transmitted - NYT).
 - Si el carácter leído es nuevo: se emite el código de la trayectoria hacia el nodo imaginario, seguido del símbolo literal. Luego, el nodo imaginario se divide (se "bifurca") para crear un nuevo nodo de símbolo imaginario (hijo izquierdo) y la hoja del carácter nuevo con peso inicial 1 (hijo derecho).
 - Si el carácter ya existe en el árbol: se emite su código binario actual y simplemente se incrementa su peso.
- **Sibling Property (Premisa 2 y balanceo):** Para garantizar que el árbol siga siendo un árbol de Huffman óptimo de longitud mínima, los nodos de mayor peso deben estar lo más cerca posible de la raíz. Para mantener esta propiedad, se recorre el árbol en una lista ordenada de nivel a nivel, de izquierda a derecha y de abajo hacia arriba. Si al incrementar un peso se detecta que un nodo de nivel profundo supera en peso a un nodo más cercano a la raíz, se realiza una **reestructuración o intercambio de nodos** (y sus subárboles si los tuvieran) para restablecer la propiedad de hermanos (*sibling*).

6.3.2. Codificación de SHANNON y FANO dinámico

A diferencia de Huffman, estos métodos no cuentan con una regla de actualización física y local estructurada (como la propiedad de hermanos o *Sibling Property* de un árbol) que permita mantener la estructura óptima mediante pequeños movimientos de piezas, por lo cual la codificación es recalculada por cada nuevo carácter.

6.3.3. Codificación LEMPEL-ZIV. LEMPEL-ZIV-WELCH

Las codificaciones Lempel-Ziv (LZ) y su variante Lempel-Ziv-Welch (LZW) pertenecen a la familia de los métodos de diccionario y son algoritmos de compresión sin pérdida de carácter universal.

A diferencia de los métodos estadísticos (como Huffman o Shannon-Fano), los métodos LZ no necesitan conocer las probabilidades previas de los símbolos de la fuente. En su lugar, logran la compresión reemplazando secuencias de caracteres repetidas en el mensaje por "punteros" a apariciones previas de esas mismas secuencias, construyendo un diccionario de forma dinámica durante la codificación y la decodificación.

6.4.1. LZ77 (ventana corrediza)

Propuesto en 1977, este método no utiliza un diccionario independiente, sino que emplea el propio texto ya procesado como diccionario mediante una ventana corrediza.

- Estructura: La ventana se divide en dos partes: un buffer de búsqueda (contiene el texto que ya se acaba de codificar) y un buffer de mirar más allá (contiene el texto que está por codificarse).
- Codificación: El algoritmo escanea hacia atrás en el buffer de búsqueda tratando de encontrar la coincidencia más larga con los símbolos que están en el buffer de mirar más allá.
- Salida: Produce una marca o "token" de tres campos: (desplazamiento, longitud, siguiente_símbolo). Si no encuentra ninguna coincidencia, emite el token (0, 0, símbolo_no_coincidente).
- Aplicación: Las variantes de este método son la base de los formatos ZIP y GZIP

Ej.: mensaje a comprimir: sir sid

Paso	Buffer de Búsqueda (Diccionario previo)	Buffer de mirar más allá (A procesar)	Coincidencia hacia atrás	Salida (Marca generada)
1	(vacío)	sir sid	No hay. Siguiente es 's'.	(0, 0, 's')
2	s	ir sid	No hay. Siguiente es 'i'.	(0, 0, 'i')
3	si	r sid	No hay. Siguiente es 'r'.	(0, 0, 'r')
4	sir	sid	No hay. Siguiente es ' ' (espacio).	(0, 0, ' ')
5	sir	sid	Hay coincidencia para "si". Está a 4 posiciones atrás, la coincidencia tiene 2 de longitud, y el símbolo que le sigue es 'd'.	(4, 2, 'd')
El proceso ya termino, pues el (4,2) codifico el "si" y la 'd' es agregada después				

6.4.2. LZ78

Esta variación elimina por completo el concepto de ventana deslizante y buffers de búsqueda de tamaño fijo, funciona construyendo dinámicamente un **diccionario explícito** en forma de árbol (llamado estructura **trie**) a medida que lee los datos. El diccionario se inicializa únicamente con la cadena vacía (índice 0).

El compresor va leyendo caracteres hasta formar la subcadena más corta que **no** esté en el diccionario. En ese momento, emite un **par (índice, carácter)**.

- **Índice:** El número de entrada en el diccionario que corresponde a la coincidencia más larga que sirvió como prefijo.
- **Carácter:** El símbolo nuevo que hizo fallar la búsqueda en el diccionario (símbolo de innovación).

Inmediatamente después de emitir el par, esa nueva subcadena completa se agrega al diccionario con un nuevo índice.

Codificación LZW

Es una evolución del LZ78, la cual elimina el segundo campo de salida, el "carácter". Para lograrlo, tanto el codificador como el decodificador inician con un diccionario prellenado con todos los símbolos base (por ejemplo, las posiciones 0 a 255 se llenan con los caracteres ASCII).

El codificador lee símbolos y los acumula en una cadena I . Mientras I + el siguiente carácter (x) exista en el diccionario, sigue acumulando. Cuando $I + x$ **no** está en el diccionario, emite el índice de I , guarda la nueva frase $I + x$ en el diccionario, y reinicia la cadena I para que sea solo x .

Ejemplo:

Mensaje a comprimir: sir sid

Diccionario inicial (ASCII): 's' = 115, 'i' = 105, 'r' = 114, ' ' = 32. Las nuevas frases se guardarán a partir del índice 256

Paso / Lectura	Cadena Acumulada (I)	Evaluación ($I + x$)	Salida (Código Emitido)	Nueva entrada añadida al Diccionario
1 (Lee 's')	s	Lee 'i'. "si" no está en el diccionario.	115 (índice de 's')	256 = si
2 (Lee 'i')	i	Lee 'r'. "ir" no está en el diccionario.	105 (índice de 'i')	257 = ir
3 (Lee 'r')	r	Lee ' '. "r " no está en el diccionario.	114 (índice de 'r')	258 = r
4 (Lee ' ')		Lee 's'. " s" no está en el diccionario.	32 (índice de ' ')	259 = s

5 (Lee 's')	s	Lee 'i'. "si" SÍ ESTÁ en el dicc (es la 256).	<i>(No emite nada aún, sigue acumulando)</i>	<i>(Ninguna)</i>
6 (Lee 'i')	si	Lee 'd'. "sid" no está en el diccionario.	256 (índice de 'si')	260 = sid

6.4.3. Comparación entre metodologías dinámicas de compactación.

Los algoritmos dinámicos **Shannon, Fano y Huffman codifican carácter por carácter**, por lo que su tasa de compresión está limitada por la entropía de primer orden de la fuente ($H(S)$). En cambio, **LZ77, LZ78 y LZW capturan de manera natural las dependencias estadísticas de largo alcance** de la fuente al agrupar caracteres en frases de longitud creciente, logrando tasas de compresión significativamente mejores en textos largos (ej. ~55% de compresión para LZ frente a ~43% para Huffman en inglés estándar).

Bajo cualquier escenario dinámico, la longitud promedio de codificación de los mensajes mantiene la estricta relación matemática:

$$L_{Huffman} \leq L_{Fano} \leq L_{Shannon}$$

El árbol adaptativo de Huffman siempre garantizará el código más compacto (menor longitud promedio de palabra) de entre los métodos dinámicos de caracteres individuales.

En términos de velocidad:

- Fano y Shannon dinámicos son computacionalmente costosos por el reordenamiento continuo de tablas de frecuencia globales tras procesar cada byte.
- Huffman dinámico optimiza este procesamiento mediante balanceos locales del árbol, pero sigue siendo más lento que los métodos de diccionario.
- LZW es sumamente rápido al procesar índices numéricos directos. No obstante, el diccionario de LZW se adapta con relativa lentitud al inicio de textos muy cortos, requiriendo que la cadena sea lo suficientemente larga para que las subcadenas repetidas en el diccionario se vuelvan complejas y brinden ventajas.

6.4. Comparación entre métodos estáticos y dinámicos.

Criterio	Estáticos	Semi-adaptativos	Dinámicos/adaptativos
Origen del modelo estadístico	Histórico y predeterminado. Utiliza estadísticas previas de un idioma o tipo de archivo estándar.	Específico del archivo. Se calcula analizando exhaustivamente el mensaje antes de comprimir	Local y en tiempo real. Se aprende y calcula directamente del flujo de datos sobre la marcha.
Lecturas al archivo o fuente de información	Una sola pasada. Se lee y se codifica directamente con la tabla fija.	Dos pasadas. Primera para contar frecuencias y segunda para codificar.	Una sola pasada. Se lee y codifica a la vez.

¿Transmite cabecera? (diccionario / árbol)	No. El codificador y decodificador comparten previamente el mismo modelo inalterable.	Sí. Es obligatorio adjuntar el árbol o tabla al inicio del archivo comprimido.	No. Ambos extremos del canal reconstruyen la estructura en perfecta sincronía paso a paso.
Costo en el canal (ancho de banda)	Bajo (sin cabecera), pero ineficiente si los datos no se ajustan al modelo estándar.	Alto impacto por el peso (<i>overhead</i>) que introduce la cabecera en el archivo.	Nulo. No hay desperdicio de bits en cabeceras o diccionarios.
Complejidad de cómputo	Muy Baja. Solo requiere búsquedas rápidas en una tabla inalterable.	Media. Requiere contar frecuencias, ordenar y estructurar la codificación una vez por archivo.	Muy Alta. Exige reestructurar árboles o reordenar y recalcular tablas con cada carácter procesado.
Eficiencia de compresión	Baja. Se degrada rápidamente si el archivo se desvía de la estadística supuesta.	Alta en datos puros, pero penalizada severamente en archivos cortos por la cabecera.	Óptima. Se adapta con precisión matemática a los cambios y características locales del flujo.
Escenario de aplicación ideal	Dispositivos de hardware con CPU muy limitada y datos de comportamiento previsible.	Almacenamiento local en un solo paso de archivos de tamaño masivo.	Comunicaciones en línea, transmisiones en tiempo real y flujos interactivos de datos.

6.5. Entropía en los procesos de MARKOV. (mover?)

6.6. Compactación mediante procesos de MARKOV. (ppm repetido?)

6.7. Codificación aritmética.

La **codificación aritmética** es una técnica de compresión estadística sin pérdidas que supera las limitaciones de Huffman al **codificar un mensaje completo como un único número real dentro del intervalo $[0, 1)$** , prescindiendo de la restricción de que cada carácter deba traducirse en un número entero de bits.

Se puede ver como asignar un numero real a todas las cadenas posibles de una fuente, donde a medida que se lee el siguiente carácter el numero real pasa a ser cada vez mas especifico, esto es mas fácil de ver con un ejemplo:

Alfabeto: $\{a, b, c\}$

$$P(a) = 0.3 \quad P(b) = 0.5 \quad P(c) = 0.2$$

Intervalos de probabilidades:

- Para el símbolo a : $[0.0, 0.3)$ (Limite acumulado inferior $Q_a = 0.0$, superior $R_a = 0.3$)
- Para el símbolo b : $[0.3, 0.8)$ (Limite acumulado inferior $Q_b = 0.3$, superior $R_b = 0.8$)
- Para el símbolo c : $[0.8, 1)$ (Limite acumulado inferior $Q_c = 0.8$, superior $R_c = 1$)

Cadena a codificar: abbca

Nota:

$$\Delta = V - U$$

$$U_{nuevo} = U + \Delta * Q_{simbolo\ leido}$$

$$V_{nuevo} = U + \Delta * R_{simbolo\ leido}$$

Paso	Símbolo leído	Operación	Límite inferior (U)	Límite superior (V)	Amplitud o rango (Δ)
Inicio	-	$U = 0.0$ $V = 1.0$	0.0	1.0	1.0
1	a	$U_{nuevo} = U + \Delta * Q_a = 0.0 + 1.0 * 0.0 = 0.0$ $V_{nuevo} = U + \Delta * R_a = 0.0 + 1.0 * 0.3 = 0.3$	0.0	0.3	0.3
2	b	$U_{nuevo} = U + \Delta * Q_b = 0.0 + 0.3 * 0.3 = 0.09$ $V_{nuevo} = U + \Delta * R_b = 0.0 + 0.3 * 0.8 = 0.24$	0.09	0.24	0.15
3	b	$U_{nuevo} = 0.09 + 0.15 * 0.3 = 0.18$ $V_{nuevo} = 0.09 + 0.15 * 0.8 = 0.21$	0.135	0.21	0.075
4	c	$U_{nuevo} = 0.135 + 0.075 * 0.8 = 0.195$ $V_{nuevo} = 0.135 + 0.075 * 1 = 0.21$	0.195	0.21	0.015
5	a	$U_{nuevo} = 0.195 + 0.015 * 0.0 = 0.195$ $V_{nuevo} = 0.195 + 0.015 * 0.3 = 0.1995$	0.195	0.1995	0.0045

Al terminar el proceso de codificación, el intervalo numérico único que identifica al mensaje "abbca" con estas nuevas probabilidades es $[0.195, 0.1995)$.

Para transmitir el menor número posible de bits, buscamos el valor binario más corto que caiga dentro de este estrecho rango:

La expansión en base 2 de los límites del intervalo es:

- $0.195_{10} = 0.0011000111101 \dots_2$
- $0.1995_{10} = 0.0011001100010 \dots_2$

Buscando un binario intermedio, encontramos 0.0011001_2 que equivale exactamente a 0.1953125_{10} en base 10.

Retirando el prefijo decimal "0.", el código binario final transmitido es **0011001**.

Descompresión

Para comprender el proceso inverso (la descompresión o decodificación), utilizaremos exactamente los mismos datos del ejercicio anterior. El receptor recibe únicamente el código binario **0011001**, el cual interpreta como la fracción decimal $c = 0.1953125$

Para decodificar, el receptor utiliza el mismo modelo de probabilidad que el emisor:

Alfabeto e Intervalos Acumulados:

- $a: [0.0, 0.3)$
- $b: [0.3, 0.8)$
- $c: [0.8, 1.0)$

El receptor decodifica los símbolos uno a uno. Para saber qué símbolo corresponde en cada iteración, toma el valor actual de su secuencia (*val*) y busca en qué intervalo de probabilidad acumulada cae. Luego, para el siguiente paso, **re-escala** el valor usando la fórmula:

$$val_{nuevo} = \frac{val - Q_{simbolo}}{P_{simbolo}}$$

Paso	Valor a evaluar (<i>val</i>)	Intervalo en el que cae	Símbolo decodificado	Operación de reescalado para el siguiente paso
1	0.19531254	[0.0, 0.3)	<i>a</i>	$\frac{0.19531254 - 0.0}{0.3} = 0.6510416 \dots$
2	0.6510416 ...	[0.3, 0.8)	<i>b</i>	$\frac{0.6510416 \dots - 0.3}{0.5} = 0.7020833 \dots$
3	0.7020833 ...	[0.3, 0.8)	<i>b</i>	$\frac{0.7020833 \dots - 0.3}{0.5} = 0.8041666 \dots$
4	0.8041666 ...	[0.8, 1)	<i>c</i>	$\frac{0.8041666 \dots - 0.8}{0.2} = 0.0208333 \dots$
5	0.0208333 ...	[0.0, 0.3)	<i>a</i>	$\frac{0.0208333 \dots - 0.0}{0.3} = 0.069444 \dots$

El receptor ha recuperado con total exactitud la secuencia original: **abbca**

¿Cómo sabe específicamente el receptor cuándo cortar la cadena?

Este es un problema crítico de la codificación aritmética. Como se nota en el Paso 5, después de decodificar la última 'a', el resultado del re-escalado nos da 0.069444.... Si el receptor continuara el bucle, este valor volvería a caer en el intervalo de la letra 'a', luego se volvería a re-escalar y **seguiría decodificando letras de manera infinita** (por ejemplo, abbcaaaaa...).

Para evitar esto y saber exactamente cuándo detenerse, existen **dos soluciones estándar**:

Solución 1: La más directa, transmitir la longitud del mensaje (*N*) de antemano (*N* = 5 en el caso del ejemplo)

Solución 2: Utilizar un carácter especial de “Fin de transmisión” (EOF), consiste en agregar un símbolo ficticio de parada al alfabeto, para el ejemplo anterior, el alfabeto sería {*a, b, c, EOF*}, donde *EOF* tiene asignada una probabilidad muy pequeña, este se agrega artificialmente al final de la compresión y en la decodificación al detectarse simboliza el final del proceso

El problema de la solución 1 es que generalmente a *N* se le asignaran una cantidad de bits/bytes fija, y podría pasar que se supere el límite máximo de *N* en un caso extremo lo que haría que se pierda información. La solución 2 es mas compleja pero no impone ningún limite.

6.8. Transformaciones de fuente: Burrows-Wheeler

La **Transformación de Burrows-Wheeler (BWT)**, representa una de las técnicas más innovadoras de preprocesamiento de fuente empleada en compresores de alto rendimiento como bzip y bzip2.

A diferencia de los algoritmos de compresión tradicionales, **la transformación de Burrows-Wheeler no reduce el tamaño de los datos por sí misma**. En su lugar, actúa como una **transformación reversible de la fuente** que reordena los caracteres del texto de manera que símbolos idénticos o de contexto estadístico similar queden agrupados de forma contigua. Esto eleva enormemente la

redundancia local del mensaje, permitiendo que algoritmos simples de compresión alcancen mejores niveles de compresión en comparación al mensaje original.

¿Cómo funciona la Transformación Directa (Codificación)?

La BWT procesa el texto dividiéndolo en bloques de tamaño fijo. Para comprender el algoritmo de forma clara e intuitiva, supongamos que queremos codificar la palabra "**banana**".

Para evitar ambigüedades en la reconstrucción, es una práctica estándar añadir un carácter especial de fin de cadena (por ejemplo, \$) al final de la palabra, resultando en la cadena original: "banana\$". También se podría hacer transmitiendo el índice de la primera letra de la palabra.

Paso 1: Generar todas las rotaciones cíclicas

Se toman los N caracteres del bloque y se generan todas sus rotaciones cíclicas posibles (desplazamientos hacia la izquierda carácter por carácter), lo que forma una matriz de tamaño $N \times N$:

b	a	n	a	n	a	\$
a	n	a	n	a	\$	b
n	a	n	a	\$	b	a
a	n	a	\$	b	a	n
n	a	\$	b	a	n	a
a	\$	b	a	n	a	n
\$	b	a	n	a	n	a

Paso 2: Ordenamiento

Se ordenan alfabéticamente todas las filas de la matriz generada. El carácter especial \$ se define tradicionalmente como el menor elemento del alfabeto:

\$	b	a	n	a	n	a
a	\$	b	a	n	a	n
a	n	a	\$	b	a	n
a	n	a	n	a	\$	b
b	a	n	a	n	a	\$
n	a	\$	b	a	n	a
n	a	n	a	\$	b	a

Paso 3: Extraer la última columna (L)

El resultado de la transformación de Burrows-Wheeler es la secuencia formada por los caracteres de la **columna de más a la derecha (L)** de la matriz ordenada.

Salida de la BWT: "annb\$aa" (las letras idénticas quedan agrupadas, facilitando la compresión posterior).

Decodificación (transformación inversa)

El receptor reconstruye la palabra usando solo la columna final **L = "annb\$aa"**:

1. **Obtener F:** Ordenamos alfabéticamente la columna **L** para recrear la primera columna (**F**).

2. **Numerar ocurrencias:** Enumeramos secuencialmente las letras repetidas en ambas columnas de arriba a abajo:

- **Columna Final (L):** $L = [a_1, n_1, n_2, b_1, \$_1, a_2, a_3]$
- **Primera Columna (F):** $F = [\$_1, a_1, a_2, a_3, b_1, n_1, n_2]$

Seguir el camino (LF-Mapping):

Sabiendo que el carácter en **L** precede al de **F** en la misma fila, rastreamos hacia atrás desde **\$**:

1. **\$₁** (en fila 0 de F) apunta a **a₁** en L. (Cadena acumulada hacia atrás: a\$)
2. **a₁** (en fila 1 de F) apunta a **n₁** en L. (Cadena: na\$)
3. **n₁** (en fila 5 de F) apunta a **a₂** en L. (Cadena: ana\$)
4. **a₂** (en fila 2 of F) apunta a **n₂** in L. (Cadena: nana\$)
5. **n₂** (en fila 6 de F) apunta a **a₃** en L. (Cadena: anana\$)
6. **a₃** (en fila 3 de F) apunta a **b₁** en L. (Cadena: banana\$)
7. **b₁** (en fila 4 de F) apunta a **\$₁** (Volvimos al inicio, fin).

Leído al revés, recuperamos exactamente: "**banana\$**".

ANEXO – Campos de Galois

Definición

Un **Campo de Galois** (o cuerpo finito), denotado como $GF(q)$, es una estructura algebraica que consiste en un conjunto con un número finito de q elementos, sobre los cuales se han definido operaciones de suma, resta, multiplicación y división de tal forma que siempre dan como resultado otro elemento válido dentro del mismo conjunto.

Es similar a la aritmética modular pero extendida a polinomios, además tienen más restricciones, que les dan propiedades útiles. La principal es garantizar la existencia de inversos multiplicativos, esta es una propiedad muy necesaria para que las operaciones hechas se puedan deshacer y recuperar el numero original, son muy usados criptografía y control de error. Y la otra razón es para evitar desbordamientos, pero esto también se consigue con aritmética modular.

Propiedades y características principales

1. **Cantidad de elementos ($q = p^m$):** Un campo finito existe si y solo si el número total de sus elementos q (también llamado orden) es de la forma p^m , donde p es un número primo (la característica del campo) y m es un entero positivo (el grado de extensión). No existe ningún campo finito fuera de esta regla.

Esta restricción es lo que garantiza la existencia de los inversos multiplicativos (garantiza la existencia de a^{-1} para cualquier número a del conjunto, tal que $a * a^{-1} \equiv 1$ o el elemento neutro), esto es necesario para que las operaciones hechas se puedan deshacer y recuperar la información original en criptografía y control de error. Por el contrario, en la aritmética modular, se puede usar cualquier número como modulo y no garantiza la existencia de inversos multiplicativos, ej: $5 * x = 1 \pmod{10}$.

2. **Unicidad salvo isomorfismo:** Para cada valor q existe un único campo de Galois (salvo isomorfismos, lo que significa que pueden tener nombres diferentes para sus elementos, pero funcionan exactamente igual, es decir, son equivalentes)
3. **Estructura operacional:** Posee dos operaciones internas (suma y multiplicación) que cumplen con las propiedades formales de cuerpo conmutativo:
 1. El conjunto de los q elementos forma un grupo abeliano bajo la suma. Existe un único elemento neutro (0) y cada elemento posee su correspondiente opuesto aditivo. Para referencia, cuando $m = 1$, este conjunto es siempre $\{0, 1, 2, \dots, p - 1\}$.
 2. Si se excluye el cero, el conjunto de los $q - 1$ elementos restantes forma un **grupo cíclico abeliano** bajo la multiplicación. Contiene un elemento neutro (1) y garantiza que todo elemento no nulo posea un **inverso multiplicativo**. Es denominado el grupo multiplicativo del campo de Galois, simbolizado $GF(q)^*$.
 3. Ambas operaciones se vinculan mediante la propiedad distributiva: $(a + b) * c = (a * c) + (b * c)$

Nota: **grupo abeliano**, es un conjunto G con una operación binaria $\wedge$ (podría ser cualquiera, como $+$ o $*$) que cumple los axiomas de grupo y además la conmutatividad:

- Asociatividad: $(a \wedge b) \wedge c = a \wedge (b \wedge c)$
- Elemento identidad: $\exists e \in G \quad \forall a \in G: a \wedge e = e \wedge a = a$ (equivalente al 1 en multiplicación y 0 en suma)
- Elemento inverso: $\forall a \in G \quad \exists b \in G: a \wedge b = b \wedge a = e$ (garantizar la existencia de los inversos para todos los elementos, en la suma sería $-a$ y en la multiplicación a^{-1})
- Conmutatividad: $\forall a, b \in G: a \wedge b = b \wedge a$

Construcción de campos de Galois

- **Campos primos ($GF(p)$ con $m = 1$):** Corresponden directamente a la aritmética modular entera habitual módulo un número primo p . El conjunto de elementos es siempre $\{0, 1, 2, \dots, p - 1\}$
- **Campos extendidos ($GF(p^m)$ con $m > 1$):** Los elementos de $GF(p^m)$ son polinomios de grado menor a m , con coeficientes pertenecientes a $GF(p)$. Por ejemplo, en un campo basado en $GF(2)$, los elementos de $GF(2^m)$ son expresiones polinómicas cuyos coeficientes son binarios (0 o 1).
 1. La suma se efectúa sumando los coeficientes término a término, en campos binarios, esta operación es una suma XOR bit a bit. Como usamos en códigos cíclicos.
 2. La multiplicación se calcula multiplicando los polinomios de forma habitual y obteniendo el resto al dividir el resultado por un polinomio irreducible de grado m con coeficientes en $GF(p)$, que sea irreducible quiere decir que no pueda factorizarse como producto de polinomios de menor grado (es el equivalente a un número primo, pero en polinomios).

Nota: los campos de Galois se construyen en base a la cantidad de elementos que necesitamos, por ejemplo: el campo $GF(2^8)$ es muy utilizado pues tiene 256, perfecto para un byte, este es un campo dominante en telecomunicación y seguridad, se usa por ej. En el cifrado AES y en los códigos reed-solomon.

El elemento primitivo (raíz generadora)

El elemento primitivo α es un elemento fundamental en la teoría de campos finitos y se define como un generador del grupo multiplicativo $GF(q)^*$.

Las potencias sucesivas (mod q) de α : $\alpha^0, \alpha^1, \alpha^2, \dots, \alpha^{q-2}$ producen la totalidad de los $q - 1$ elementos no nulos del campo sin repetirse. Al llegar a la potencia α^{q-1} , el ciclo se completa volviendo a 1 ($\alpha^{q-1} = 1$)

Ejemplo en $GF(11)$ ($m = 1, p = 11$)

$$GF(q)^* = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$$

$$2^0 = 1 \quad 2^1 = 2 \quad 2^2 = 4 \quad 2^3 = 8 \quad 2^4 = 16 \equiv 5 \pmod{11}$$

$$2^5 = 32 \equiv 10 \pmod{11} \quad 2^6 = 64 \equiv 9 \pmod{11} \quad 2^7 = 128 \equiv 7 \pmod{11}$$

$$2^8 = 256 \equiv 3 \pmod{11} \quad 2^9 = 512 \equiv 6 \pmod{11} \quad 2^{10} = 1024 \equiv 1 \pmod{11}$$

Nota: si $m \neq 1$ en vez de $\text{mod } 11$ se debe usar un polinomio irreducible

Polinomio primitivo: Un polinomio irreducible de grado m que tiene a un elemento primitivo α como raíz se denomina polinomio primitivo.

La existencia del elemento primitivo permite trabajar con dos representaciones equivalentes para cada elemento del campo:

- Forma polinómica/vectorial: Expresa el elemento como una combinación lineal de potencias de α de grado menor a m : $a_0 + a_1\alpha + a_2\alpha^2 + \dots + a_{m-1}\alpha^{m-1}$ o mediante su vector binario de coeficientes. Esta forma es la idónea para realizar sumas.
- Forma exponencial/potencia: Expresa el elemento como una potencia de α . Es la forma óptima para efectuar multiplicaciones y divisiones, ya que multiplicar elementos se reduce a sumar sus exponentes modulo $q - 1$:

$$\alpha^i * \alpha^j = \alpha^{(i+j) \bmod (q-1)}$$

Aritmética en $GF(2^8)$ con ejemplos de cálculos

El campo $GF(2^8)$ es sumamente común y de gran importancia, ya que cada uno de sus elementos puede representarse exactamente con 8 bits (un byte). Por esto mismo, es la estructura matemática subyacente en el diseño de circuitos digitales y algoritmos criptográficos modernos, como el estándar de cifrado AES (Advanced Encryption Standard).

En este cuerpo, los coeficientes de los polinomios operan en el cuerpo base binario $GF(2)$. Esto tiene dos consecuencias fundamentales:

1. **La suma y la resta son la misma operación** (ya que $1 + 1 = 0$ y $-1 = 1$).
2. Ambas operaciones equivalen a la operación lógica **XOR** ($\oplus$) bit a bit.

Ejemplo de suma

$$(x^3 + x + 1) + (x^2 + 1) = x^3 + x^2 + x$$

A nivel digital, los polinomios se representan como un byte donde los coeficientes de grado 7 a 0 dan valor a los bits. La operación se realiza aplicando XOR bit a bit: $00001011 \oplus 00000101 = 00001110$

Ejemplo de multiplicación

Multipliquemos $(x^5 + x)$ por $(x^4 + x^3 + x^2 + 1)$ modulo el polinomio irreducible de AES $f(x) = x^8 + x^4 + x^3 + x + 1$.

Para multiplicar polinomios cuyo producto exceda el grado 7, se requiere reducir la expresión final por $f(x) = x^8 + x^4 + x^3 + x + 1$. Multipliquemos $(x^5 + x)$ por $(x^4 + x^3 + x^2 + 1)$:

- Paso A (Multiplicación distributiva ordinaria):**

$$(x^5 + x) * (x^4 + x^3 + x^2 + 1) = x^9 + x^8 + x^7 + x^5 + x^5 + x^4 + x^3 + x$$

- Paso B (Reducción de coeficientes en $GF(2)$):** Como estamos en coeficientes binarios, los términos repetidos de igual grado se cancelan ($x^5 + x^5 = 0$):

$$x^9 + x^8 + x^7 + x^4 + x^3 + x$$

- Paso C (Reducción modular usando el polinomio irreducible):** Dado que el polinomio irreducible $f(x) \equiv 0 \pmod{f(x)}$, podemos establecer que:

$$x^8 + x^4 + x^3 + x + 1 \equiv 0 \implies x^8 = x^4 + x^3 + x + 1$$

Equivalencia que necesitamos para reemplazar x^8 , para el término x^9 podemos multiplicar la identidad anterior por x :

$$x^9 = x * x^8 = x(x^4 + x^3 + x + 1) = x^5 + x^4 + x^2 + x$$

- Paso D (Sustitución y agrupación final):** Reemplazamos x^9 y x^8 en la ecuación obtenida en el Paso B:

$$(x^5 + x^4 + x^2 + x) + (x^4 + x^3 + x + 1) + x^7 + x^4 + x^3 + x$$

Agrupamos nuevamente eliminando los términos repetidos.

$$x^7 + x^5 + x^4 + x^2 + x + 1$$

- Resultado final de la multiplicación:** $x^7 + x^5 + x^4 + x^2 + x + 1$
 - Representación binaria:** 10110111

Ejemplo de división

Dividamos $I(x) = x^7 + x^2 + 1$ en $g(x) = x^3 + x + 1$

The image shows a handwritten polynomial division on a black background. The dividend is $x^7 + x^2 + 1$ and the divisor is $x^3 + x + 1$. The division steps are as follows:

$$\begin{array}{r} x^7 + x^2 + 1 \\ - (x^7 + x^5 + x^4) \\ \hline x^5 + x^4 + x^2 + 1 \\ - (x^5 + x^3 + x^2) \\ \hline x^4 + x^3 + 1 \\ - (x^4 + x^2 + x) \\ \hline x^3 + x^2 + x + 1 \\ - (x^3 + x + 1) \\ \hline x^2 \end{array}$$

A red checkmark is drawn at the bottom, indicating the division is complete.

Resultado final:

- **Cociente:** $x^4 + x^2 + x + 1$
- **Resto:** x^2
- **Significado:** $I(x) = g(x) * cociente + resto = (x^3 + x + 1) * (x^4 + x^2 + x + 1) + x^2$

Lo que se hace es restas sucesivas de la forma: $r_i(x) - g(x) * x^{b_i} = r_{i+1}(x)$

Donde $r_0(x) = I(x)$ es decir el residuo inicial es el dividendo, y $b_i = \deg(r_i(x)) - \deg(g(x))$ es decir la diferencia de grados entre el residuo actual y el divisor.

Estas restas sucesivas se realizan mientras $b_i \geq 0$ y $r_i \neq 0$, luego el r_i es la última operación es el resto de la división, y el cociente es la suma de todos los x^{b_i} . Aquí se puede ver la división con todas las partes marcadas:

The image shows a handwritten polynomial long division on a dark background. It starts with $I(x) = x^7 + x^2 + 1$ and $g(x) = x^3 + x + 1$. The division proceeds through four steps, each subtracting a multiple of $g(x)$ from the current remainder r_i to get r_{i+1} . The steps are: 1) $r_0 = I(x)$, subtract $g(x) * x^4$ to get r_1 ; 2) $r_1 = x^5 + x^4 + x^2 + 1$, subtract $g(x) * x^2$ to get r_2 ; 3) $r_2 = x^4 + x^3 + 1$, subtract $g(x) * x$ to get r_3 ; 4) $r_3 = x^3 + x^2 + x + 1$, subtract $g(x) * x^0$ to get $r_4 = x^2$, which is the final remainder. A red checkmark is drawn at the bottom.

$$\begin{array}{rcll} x^7 + x^2 + 1 & = r_0(x) = I(x) & & \\ - x^7 + x^5 + x^4 & = g(x) * x^4 \leftarrow b_0 = 4 & & \\ \hline x^5 + x^4 + x^2 + 1 & = r_1(x) & & \\ - x^5 + x^3 + x^2 & = g(x) * x^2 \leftarrow b_1 = 2 & & \\ \hline x^4 + x^3 + 1 & = r_2(x) & & \\ - x^4 + x^2 + x & = g(x) * x \leftarrow b_2 = 2 & & \\ \hline x^3 + x^2 + x + 1 & = r_3(x) & & \\ - x^3 + x + 1 & = g(x) * x^0 = g(x) \leftarrow b_3 = 1 & & \\ \hline x^2 & = r_4(x) = \text{resto final} & & \end{array}$$

Ejemplo de inverso multiplicativo

El inverso multiplicativo de un elemento b en un cuerpo de Galois es otro elemento b^{-1} tal que su producto es igual a la unidad (1) dentro de la aritmética de ese cuerpo.

- Puede haber elementos que son inversos multiplicativos a sí mismos
- Los elementos 1 y $p - 1$ son siempre sus propios inversos multiplicativos
- Los campos de Galois garantizan que para todos los elementos (exceptuando al 0) existe uno y solo un inverso multiplicativo. A diferencia de el algebra modular donde podría no haber inverso multiplicativo o podría haber varios para un solo elemento.

Ejemplo simple en $GF(5)$

El cuerpo de Galois de orden 5, que tiene los elementos $\{0,1,2,3,4\}$

Si quisiéramos hallar el inverso multiplicativo de 3, es decir un numero x tal que:

$$3 * x \equiv 1 \pmod{5}$$

Podemos probar multiplicando por los elementos del cuerpo:

$$3 * 1 = 3 \not\equiv 1 \pmod{5}$$

$$3 * 2 = 6 \equiv 1 \pmod{5}$$

Ejemplo en $GF(2^8)$

Para ilustrar el cálculo de un **inverso multiplicativo** en el cuerpo $GF(2^8)$ (el pilar fundamental para construir la S-Caja de sustitución de AES), utilizaremos el **Algoritmo Extendido de Euclides**.

Este algoritmo resuelve dos tareas:

- Encuentra el máximo común divisor entre los dos polinomios a y b
- Encuentra dos coeficientes (u y v) para expresar ese MCD como una suma combinada de los originales: $MCD(a, b) = u * a + v * b$

¿Por es importante esto ?, si a y b son coprimos (no comparten divisores comunes), su MCD es 1, el algoritmo queda así:

$$MCD(a, b) = u * a + v * b = 1$$

En nuestro caso a será el polinomio irreducible $f(x)$ y b el polinomio del cual buscamos el inverso multiplicativo, como a es el polinomio irreducible, sabemos que son coprimos (dado que el polinomio irreducible no puede ser dividido).

Con esto el termino $u * a$ será igual a 0, dado que operamos modulo $f(x)$ y $u * a$ será un múltiplo de $f(x)$. Esto deja:

$$v * b \equiv 1 \pmod{f(x)}$$

Entonces al aplicar el algoritmo de Euclides encontraríamos v que sería el inverso multiplicativo que buscamos del polinomio b . El algoritmo es así:

Paso (i)	Cociente (c_i)	Residuo (g_i)	Coeficiente u_i	Coeficiente v_i
0	—	$g_0 = a$	$u_0 = 1$	$v_0 = 0$
1	—	$g_1 = b$	$u_1 = 0$	$v_1 = 1$
2	$c_2 = g_0 \div g_1$	$g_2 = g_0 \pmod{g_1}$	$u_2 = u_0 - c_2 * u_1$	$v_2 = v_0 - c_2 * v_1$
3	$c_3 = g_1 \div g_2$	$g_3 = g_1 \pmod{g_2}$	$u_3 = u_1 - c_3 * u_2$	$v_3 = v_1 - c_3 * v_2$
...	...	...	...	...
i	$c_i = g_{i-2} \div g_{i-1}$	$g_i = g_{i-2} \pmod{g_{i-1}}$	$u_i = u_{i-2} - c_i * u_{i-1}$	$v_i = v_{i-2} - c_i * v_{i-1}$

El algoritmo se detiene cuando el residuo g_i es igual a 1.

Como ejemplo calcularemos el inverso de $b(x) = x^4 + x^2 + 1$ bajo el polinomio irreducible de AES $f(x) = x^8 + x^4 + x^3 + x + 1$. Como nosotros no necesitamos el coeficiente u , este puede ser omitido.

Fila (i)	Cociente (c_i)	Residuo (g_i)	Coeficiente v_i
0	—	$x^8 + x^4 + x^3 + x + 1$	0
1	—	$x^4 + x^2 + 1$	1
2	$x^4 + x^2 + 1$	$x^3 + x$	$x^4 + x^2 + 1$
3	x	1	$x^5 + x^3 + x + 1$

El polinomio resultante $x^5 + x^3 + x + 1$ es la inversa multiplicativa de $x^4 + x^2 + 1$

Comprobación

$$(x^4 + x^2 + 1) * (x^5 + x^3 + x + 1) = x^9 + x^5 + x^4 + x^2 + x + 1 \equiv 1 \pmod{f(x)}$$

Reemplazamos $x^9 \equiv x^5 + x^4 + x^2 + x$ al despejarlo de $f(x) \equiv 0 \pmod{f(x)}$, lo que hace que el producto nos de exactamente 1.

UNIDAD N°7 - CRIPTOGRAFIA

7.1. Definiciones. Introducción a la teoría de números. Variación de Entropía.

Criptografía: Rama de la matemática e informática que estudia métodos y técnicas para proteger un mensaje o archivo por medio de un algoritmo, usando una o más claves. La criptografía es clave para la mayoría de los pilares de la seguridad informática.

Los **pilares de la seguridad informática** son los principios fundamentales que guían la protección de datos y sistemas

- **Confidencialidad:** Garantiza que la información solo sea accesible por las personas autorizadas. Ejemplo: cifrado de datos, contraseñas seguras.
- **Integridad:** Asegura que los datos transmitidos o almacenados no sean alterados (De forma intencional o no). Ejemplo: firmas digitales, sumas de verificación (hashes).
- **Disponibilidad:** Los sistemas y datos deben estar accesibles cuando se necesiten. Ejemplo: redundancia de servidores, planes de recuperación ante desastres.
- **Autenticidad:** Verifica que los usuarios, mensajes o sistemas sean quienes dicen ser. Ejemplo: certificados digitales, autenticación multifactor.
- **No repudio:** Impide que alguien niegue haber realizado una acción (por ejemplo, enviar un mensaje o firmar un documento, como un contrato o transacción comercial), se divide en:
 - **No repudio de origen:** El emisor no puede negar que él envió el mensaje (ya que el mensaje lleva su firma digital única).
 - **No repudio de destino:** El receptor no puede negar que recibió el mensaje.

En el único pilar en la que la criptografía no interviene de forma directa es en la disponibilidad, y el no repudio no siempre es reconocido como un pilar de la seguridad informática, depende del autor.

Algunos mecanismos de seguridad son:

- **Cifrado:** Es el proceso de transformar un mensaje para convertirlo en un criptograma ininteligible. De modo que el mensaje original solo pueda ser recuperado por la persona autorizada (quien posee la clave correcta). Garantiza la confidencialidad.
- **Integridad de datos:** Asegura que el mensaje original no haya sido modificado de forma accidental ni intencionada. Se aplica principalmente mediante funciones Hash.
- **Firma digital (integridad de datos + cifrado):** Datos (como un Hash) cifrados que se añaden a un mensaje para protegerlo contra cualquier falsificación (integridad). Además, respalda la autoría del emisor (autenticidad, no repudio)

Criptoanálisis: Disciplina dedicada a estudiar como encontrar la clave utilizada o descifrar mensajes sin conocer la clave.

7.2. Sistema criptográfico

Un sistema criptográfico o criptosistema es un conjunto de elementos que permite cifrar y proteger la información que se transmite a través de un canal inseguro.

Cifrar es el proceso de aplicar transformaciones (algoritmos) sobre un mensaje (M) para convertirlo en un criptograma ininteligible (C). Esta transformación debe ser reversible, de modo que solo quien posee la clave correcta pueda realizar la transformación inversa de descifrado y recuperar el mensaje original.

Componentes de un criptosistema:

- **Espacio de textos en claro (M):** Conjunto de mensajes originales que se desea enviar.
- **Espacio de textos cifrados (C):** Conjunto de criptogramas, resultado del cifrado.
- **Espacio de claves (K):** Conjunto de parámetros matemáticos que determinan la transformación específica a usar.
- **Algoritmos de cifrado (E):** conjunto de transformaciones ($E_k: M \rightarrow C$) de mensaje en claro a criptograma usando la clave K .
- **Algoritmos de descifrado (D):** conjunto de transformaciones ($D_k: C \rightarrow M$) de criptograma a mensaje en claro usando la clave K .

Según Shannon un criptosistema es perfecto cuando conocer el texto cifrado no revela absolutamente ninguna información sobre el mensaje original. Es decir:

$$H(M) = H(M|C)$$

También estableció condiciones para que esto ocurra:

1. **Longitud de clave:** El espacio de claves debe ser igual o más grande que el espacio de mensajes, es decir que la clave debe ser al menos tan larga como el mensaje.
2. **Aleatoriedad:** Las claves deben ser elegidas de forma equiprobable.
3. **Uso único:** La clave debe usarse una sola vez y luego destruirse.

Seguridad computacional vs incondicional

- **Incondicional:** Seguridad demostrable matemáticamente, independiente de la capacidad computacional del atacante. Este sería el caso de un criptosistema perfecto.
- **Computacional:** Seguro contra adversarios con recursos computacionales limitados (práctico). La seguridad no es absoluta, pero traspasarla requeriría miles de años con los recursos actuales.

7.3. Sistemas criptográficos simétricos

También llamados de clave privada. Son aquellos donde **se emplea la misma clave para cifrar y para descifrar** (o se deriva fácilmente una de la otra), por lo cual esta debe mantenerse privada, pero para esto el emisor y el receptor deben encontrar una forma segura de intercambiarla antes de comunicarse.

Tienen la ventaja de ser muy rápidos y eficientes, su único problema es el intercambio de claves.

Confusión y difusión

- **Confusión:** Su objetivo es **ocultar la relación** estadística que existe entre el texto claro, el texto cifrado y la clave de cifrado. Se logra principalmente mediante la **sustitución**, que consiste en cambiar unos símbolos o bits por otros siguiendo una regla o función compleja. La confusión por sí sola podría ser suficiente para la seguridad total, pero requeriría tablas de sustitución astronómicamente grandes e inviables en la práctica.
- **Difusión:** Busca **dispersar la redundancia** y las propiedades estadísticas del lenguaje del texto claro a lo largo de todo el texto cifrado. Se implementa habitualmente a través de la **transposición o permutación**.

7.4. Algoritmos clásicos de cifrado

Los algoritmos clásicos agrupan a todos los sistemas de cifrado simétricos desarrollados antes de la Segunda Guerra Mundial y de la invención de las computadoras. Tienen como característica principal que son procedimientos que pueden ser ejecutados empleando lápiz y papel.

Notas:

- A partir de aquí se trabajará mucho con algebra modular, modulo N
- N es el tamaño del alfabeto objetivo, para los ejemplos usaremos el alfabeto $\{a, b, c, d, e, f, g, h, i, j, k, l, m, n, ñ, o, p, q, r, s, t, u, v, w, x, y, z\}$ tamaño 27
- Cuando se menciona el inverso multiplicativo x^{-1} de un numero x es un numero tal que: $x * x^{-1} \equiv 1 \pmod{N}$
- Una condición común para garantizar la existencia del inverso multiplicativo es que x y N sean coprimos (primos entre si), es decir que no tengan divisores comunes además del 1. Otra forma de garantizar esto es hacer que N sea primo, dado que no tiene divisores además del 1.
- En el alfabeto $N = 27$, los números 3, 9 y 18 no tienen inverso, por eso solo se usa de ejemplo.

7.3.1. Cifrados de sustitución

Estos métodos basan su seguridad en el concepto de **confusión**. Consisten en reemplazar los símbolos originales del mensaje por otros diferentes, manteniendo intacto el orden en que aparecen.

Monoalfabéticos: Emplean una única correspondencia de sustitución para todos los caracteres a lo largo del texto. Son los métodos más vulnerables ya que conservan las propiedades estadísticas del lenguaje, haciendo fácil su criptoanálisis mediante el estudio de frecuencias. Ejemplos:

- **Algoritmo de Cesar:** Consiste en desplazar cada letra tres posiciones hacia la derecha en el alfabeto.

$$C = (M + 3) \pmod{27}$$
$$M = (C - 3 + 27) \pmod{27}$$

Ej.:

$$M = \text{el patio de mi casa es particular}$$
$$C = \text{hñ sdwlr gh ol fdvd hv sduwlfxñdu}$$

En este caso el cifrado se aplica letra por letra, y se puede ver como cada letra se cifra siempre igual (todas las a se convierten en d por ej.).

- **Sustitución afín:** Es la generalización de cesar, el valor del carácter se multiplica por una clave a y luego se desplaza b espacios en el alfabeto:

$$C = (a * M + b) \% 27$$
$$M = (C - b) * a^{-1} \% 27$$

El factor a debe ser coprimo con b para poder garantizar

Sustitución homofónica: Buscan eliminar la redundancia estadística del texto claro utilizando un alfabeto de salida que posee más símbolos que el de entrada. A un mismo carácter de entrada se le pueden asignar múltiples símbolos de salida, con mayor probabilidad de aparición cuanto más frecuente sea la letra en el idioma original. Ejemplo:

$M = BANANA$

Mapeamos: $B: 17, 19, 23, 47, 64$ $A: 2, 18, 26, 35, 36, 42$ $N: 5, 8, 12, 27$ $E: 9, 14, 15, 21, 22$

$C = 17\ 2\ 5\ 18\ 8\ 26$

Aquí el símbolo A toma 3 valores distintos en el mensaje cifrado, lo que frustra su análisis estadístico.

Polialfabéticos: La regla de sustitución aplicada a un carácter cambia dependiendo de la posición que ocupe dentro del texto original. Esto equivale a aplicar cíclicamente una serie de cifrados monoalfabéticos.

Vigenère con clave K longitud l_K (K es una cadena o secuencia de números):

$$C_i = (M_i + K_{i \% l_K}) \% 27$$

$$M_i = (C_i - K_{i \% l_K} + 27) \% 27$$

Esto hace que una misma letra tenga representaciones distintas dependiendo de su posición, burlando en análisis estadístico.

7.3.2. Cifrados de transposición

En contraposición a los anteriores, estos algoritmos aplican el principio de **difusión**, manteniendo los mismos símbolos del mensaje original, pero cambiando su orden o posición. Un método histórico es el *escítalo* utilizado en la Antigua Grecia, que consistía en enrollar una tira de piel sobre un bastón cilíndrico para leer o escribir el texto a lo largo del mismo. De manera general, se pueden resolver escribiendo el texto en una tabla de columnas para luego leer las columnas en un orden distinto.

Ejemplo de transposición básica:

$M = AHIVALABOMBA$
 $K = \{3, 1, 4, 2\}$
 $C = IABAAOVBAHLM$

1	2	3	4
A	H	I	V
A	L	A	B
O	M	B	A

7.5. Cifrados por bloques

Los cifrados por bloques operan dividiendo la información a cifrar en subcadenas o bloques de un tamaño fijo. Sobre cada bloque aplican operaciones de confusión (como sustituciones) y difusión (permutaciones) iteradas varias veces, formando lo que se conoce como un **cifrado de producto**.

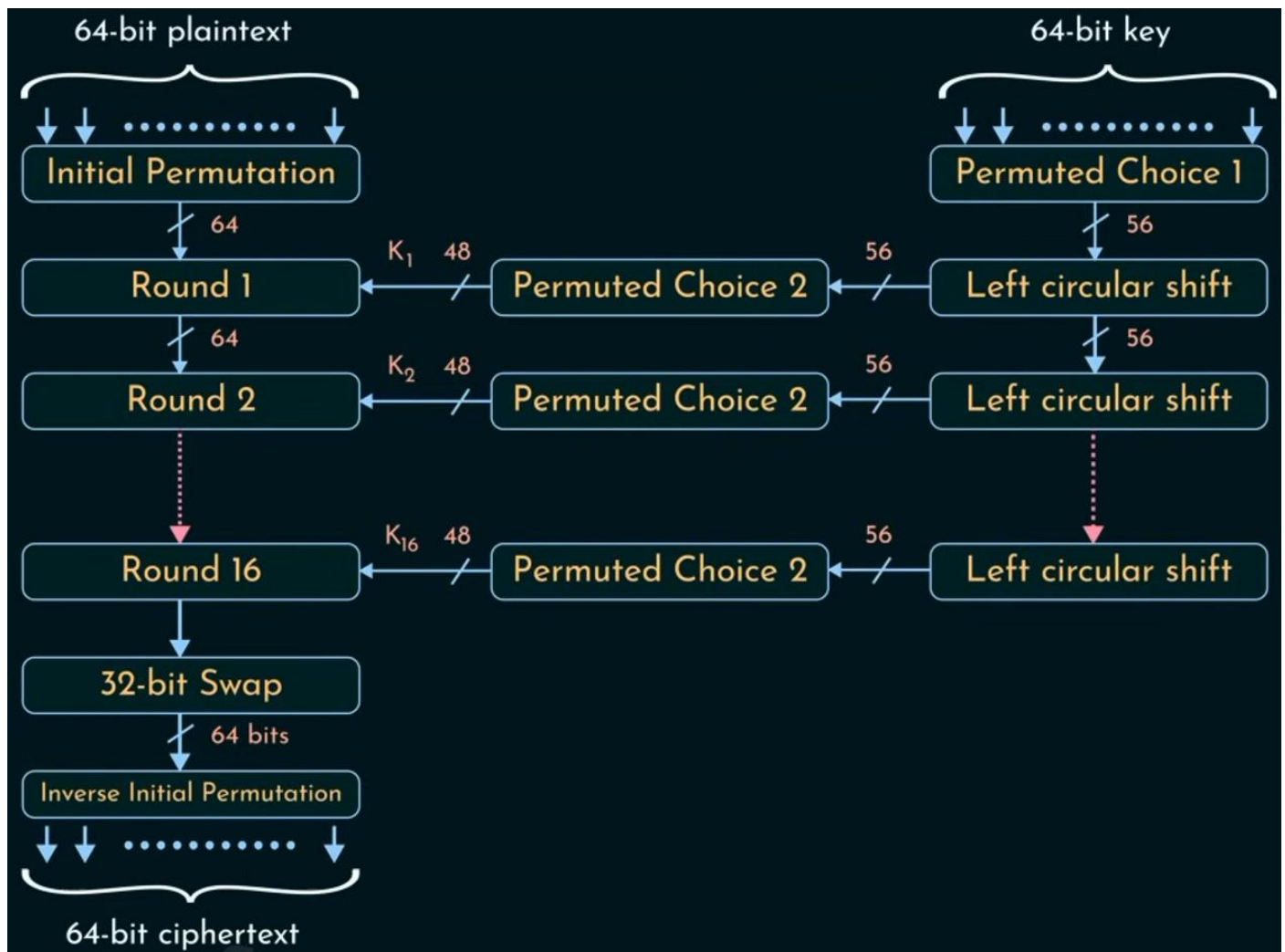
7.4.1. Algoritmo DES

El **Data Encryption Standard (DES)** es un algoritmo simétrico basado en la red de Feistel (16 rondas) que fue adoptado como estándar por el gobierno de EE.UU. en 1976.

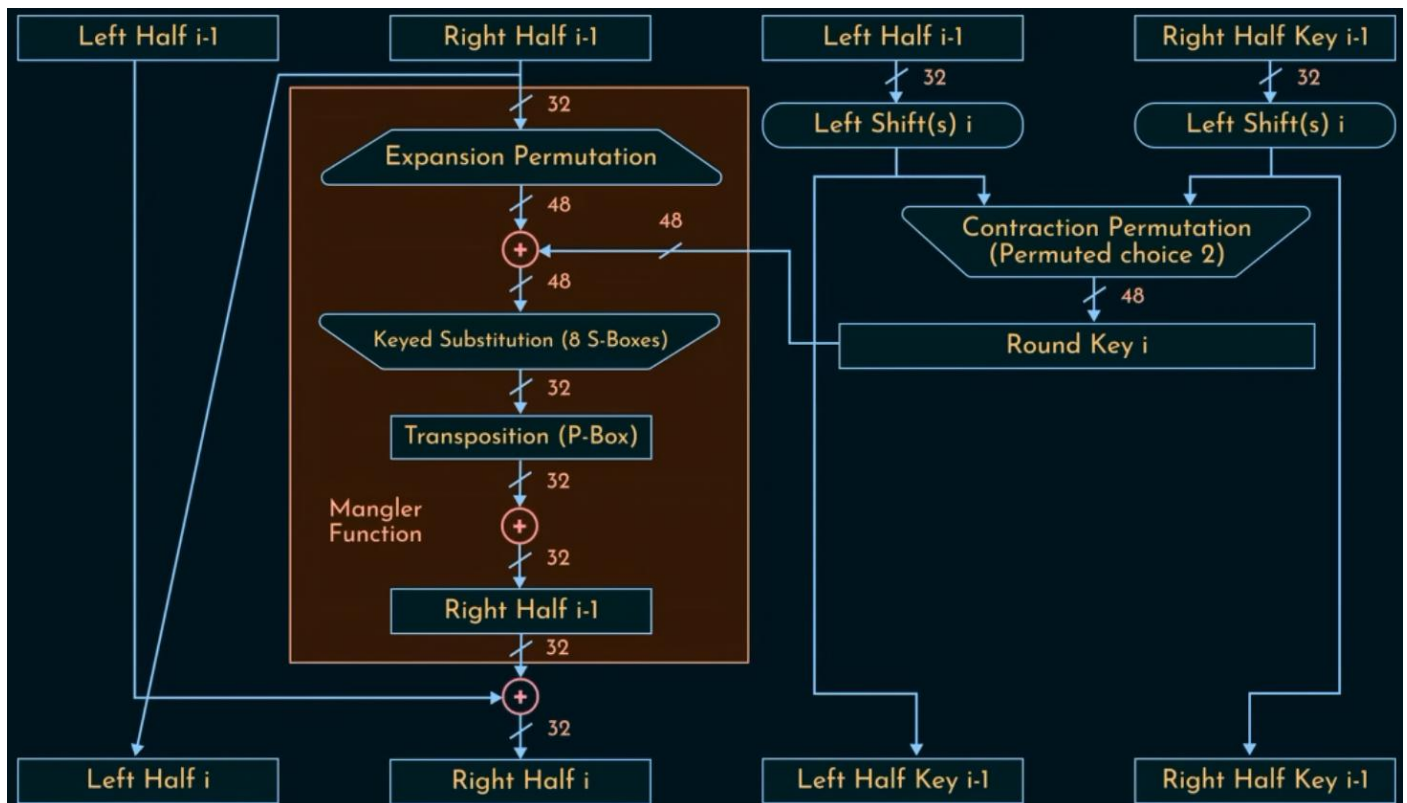
- Cifra bloques de 64 bits utilizando una **clave efectiva de 56 bits** (a partir de una original de 64 bits donde 8 bits son de paridad).
- Aplica permutaciones iniciales y finales sobre el bloque completo.
- Su función interna expande una de las mitades de 32 a 48 bits, la combina (XOR) con la subclave de la ronda y la hace pasar por **ocho S-Cajas de 6x4 bits**, las cuales le aportan robustez por su naturaleza no lineal.
- Aunque teóricamente sigue siendo robusto, en 1998 se demostró que **una máquina podía romperlo por fuerza bruta** en pocos días, debido a la corta longitud de su clave.

El algoritmo DES

Recibe un bloque de mensaje M (64 bits) y una clave K (64 bits), y su estructura es la siguiente:



En cada ronda tanto la clave como el mensaje se trabajan en mitades de la siguiente forma:



- **Expansión:** recibe los 32 bits de derecha y los expande a 48 bits mediante una permutación predefinida que duplica ciertos bits.
- **Sustitución (8 Cajas-S):** Los 48 bits se dividen en 8 grupos (B_1 a B_8) de 6 bits, donde a cada grupo le corresponde una tabla predefinida de tamaño 4×16 , luego de esos 6 bits ($b_1 b_2 b_3 b_4 b_5 b_6$), el número formado por b_1 y b_6 (entre 0 y 3) determina la fila, y los otros 4 bits determinan la columna. En las celdas de cada una de las tablas hay números de 4 bits, este valor sustituye el original de 6 bits, lo que devuelve el tamaño de 46 bits a 32 bits. Esta es la única etapa no lineal del algoritmo, y es su mayor fortaleza criptográfica.

Las permutaciones y tablas de las S-Boxes usadas a lo largo del algoritmo están predefinidas por un estándar oficial, no varían entre implementaciones. Cualquier variación de las permutaciones o tablas sería una variante de DES y no garantizaría interoperabilidad.

Nota: las imágenes son de videos de YouTube de Neso Academy

7.4.2. Variantes de DES

Ante la caída de DES, surgieron variantes para aumentar la seguridad reutilizando la tecnología existente:

- **DES Múltiple (o Triple-DES):** Consiste en encadenar el algoritmo DES varias veces. Típicamente codifica el mensaje con una clave k_1 , lo decodifica con una clave k_2 y lo vuelve a codificar con k_3 , resultando en una clave efectiva de 112 bits, lo que lo hace mucho más seguro.

$$C = E_{k_1}(D_{k_2}(E_{k_3}(M)))$$

$$M = D_{k_3}(E_{k_2}(D_{k_1}(C)))$$

- **DES con subclaves independientes:** En lugar de generar subclaves a partir de una de 56 bits, esta variante emplea 16 subclaves distintas de 48 bits totalmente independientes. Siendo una longitud total de clave de 768 bits. A pesar de la clave se ha demostrado que puede romperse mediante criptoanálisis diferencial por lo que no ofrece una ventaja sustancial.

- **DES Generalizado:** Usa n bloques en vez de solo 2 (izquierdo y derecho). Aumenta tanto la longitud de clave como el tamaño del mensaje a procesar. Se ha demostrado que gana poca seguridad y en algunos casos es más débil.
- **DES con S-Cajas alternativas:** Esta variante propone sustituir las ocho tablas de sustitución (S-Boxes) originales por otras diferentes. Pero no se han encontrado S-Cajas que superen las originales. Muchos años después se revelaron que las S-Cajas de la NSA tenían propiedades de resistencia contra técnicas de criptoanálisis que no fueron públicas hasta muchos años después.

7.4.3. Algoritmo AES

El **Advanced Encryption Standard (AES)**, originalmente llamado Rijndael (por sus autores Daemen y Rijmen), fue seleccionado en el año 2000 por el NIST para reemplazar a DES. A diferencia de su predecesor, el AES no utiliza una red de Feistel; en su lugar, procesa todo el bloque de datos de forma paralela en cada ronda.

Parámetros fundamentales:

- **Tamaño de bloque:** es fijo de 128 bits (organizados en una matriz de 4×4 bytes llamada “estado”)
- **Tamaño de clave y rondas:** Los tamaños de clave dictan la cantidad de rondas, el tamaño de estas suele ser 128, 192 o 256 bits y la cantidad de rondas 10, 12 y 14 respectivamente
- **Aritmética:** Usa operación en el cuerpo de Galois $GF(2^8)$ tratando cada byte como un polinomio.

AES es sumamente seguro, es **resistente al criptoanálisis lineal y diferencial**, y hace imposible en la práctica los ataques de fuerza bruta.

7.6. Cifrados de flujo

Son un tipo de algoritmo de cifrado simétrico que operan procesando el mensaje original elemento por elemento (típicamente bit a bit o byte a byte), a diferencia de los cifrados por bloques que agrupan la información.

El cifrado se realiza combinando el texto en claro M con una **secuencia cifrante K** (clave) mediante una función lógica simple y reversible, casi siempre la operación **XOR**. Gracias a la propiedad involutiva de la función XOR, el algoritmo de descifrado es exactamente el mismo que el de cifrado.

Ej: Si tengo una secuencia M de datos y una secuencia cifrante K , el cifrado y descifrado es una simple operación XOR bit a bit:

$$C_i = M_i \oplus K_i$$

$$M_i = C_i \oplus K_i$$

Aquí lo importante es la generación de la secuencia cifrante.

7.5.1. Secuencias pseudoaleatorias

Si recordamos la primera condición de Shannon para alcanzar un criptosistema perfecto es que la clave debe ser tan larga como el mensaje, pero intercambiar esta clave es inviable, si se dispusiera de un canal seguro para transmitir una clave tan larga como el mensaje, sería más eficiente transmitir el mensaje por ese canal directamente.

Por eso los cifrados de flujo modernos emplean un **generador pseudoaleatorio** de secuencias de bits, que solo necesitan una clave secreta inicial más corta (y a veces un vector de inicialización). Si

bien estos no producen secuencias infinitas ni perfectamente aleatorias, son una gran aproximación para lograr un criptosistema perfecto.

- Estos producen secuencias **criptográficamente aleatorias (o pseudoaleatorias)**. Esto significa que, a partir de una porción de la secuencia, debe ser computacionalmente intratable predecir el siguiente bit.
- La longitud o período de estas secuencias depende directamente del número de etapas o celdas (n) del generador y de la naturaleza de sus conexiones, pero a grandes rasgos generan secuencias de longitud 2^n o $2^n - 1$, para aplicaciones normales se usa entre 120 y 250 etapas, lo que permite obtener secuencias astronómicamente largas, garantizando que la clave nunca se repita.

Paradójicamente, la ventaja de estas secuencias es que **pueden ser reconstruidas en su totalidad** por el receptor legítimo siempre que conozca la semilla (clave) original que inicializó el generador.

Tipos de generadores de secuencia

Los generadores para cifrados de flujo se dividen en dos grandes familias, dependiendo de si utilizan o no fragmentos del mensaje cifrado previo para calcular los siguientes valores de la secuencia:

- **Generadores síncronos:** En estos generadores, la secuencia se calcula de manera totalmente independiente del texto en claro y del texto cifrado. Requieren que el emisor y el receptor estén perfectamente sincronizados; si se pierde un bit en la transmisión, el resto del mensaje se vuelve indescifrable.
- **Generadores asíncronos:** La generación de la secuencia depende de los últimos t bits del mensaje ya cifrado (o de la semilla al comienzo). Su ventaja es que, si se pierde la sincronía por un error de transmisión, el sistema se re-sincroniza automáticamente tras procesar de forma correcta los siguientes t bits.

7.5.2. Generador de congruencia lineal

Es un generador de números pseudoaleatorios, de la forma:

$$x_{i+1} = (a * x_i + b) \bmod n$$

- x_0 : es la semilla que inicia el proceso
- a, b, n : son los parámetros que caracterizan al generador y definen la secuencia (deben ser secretos junto con la semilla)
- Este genera valores entre 0 y $m - 1$

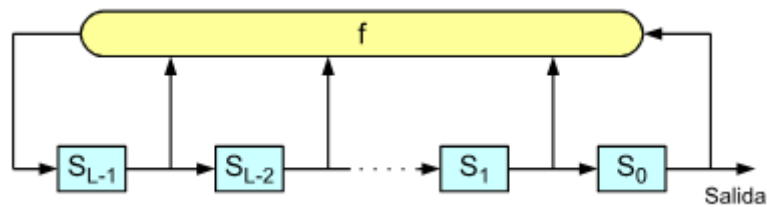
Problema: Son inútiles en la criptografía pues son fácilmente predecibles: ecuación lineal, parámetros fijos, relación lineal de valores (después los FSR tienen relación exponencial)

7.5.3. Registros de desplazamiento retroalimentados (FSR)

Luego surgen mejores formas para hacer la secuencia pseudoaleatoria para el cifrado en flujo mediante registros de desplazamiento con retroalimentación.

Son estructuras fundamentales tanto en la criptografía como en la teoría de códigos. Se utilizan de forma masiva como la base de los **generadores de secuencias síncronos para cifrados de flujo** y en la implementación física de **códigos cíclicos para la detección y corrección de errores**.

Su gran popularidad se debe a que son **extremadamente rápidos y muy fáciles de implementar mediante hardware** (circuitos electrónicos), requiriendo únicamente celdas de memoria biestables (como *Flip-Flops*) y compuertas lógicas (como XOR).



Funcionamiento básico: Están compuestos por un conjunto de L estados o celdas dispuestas en fila, donde cada una es capaz de almacenar un bit de información (S_i). Todo el sistema es controlado por un reloj electrónico que coordina el flujo de los datos. En cada ciclo o pulso de reloj ocurre lo siguiente:

1. El contenido de la celda de un extremo (usualmente llamado S_0) abandona el registro y se convierte en el **bit de salida**.
2. Los bits contenidos en el resto de las celdas se **desplazan una posición** hacia la celda adyacente.
3. El nuevo bit que ingresa para llenar la celda que quedó vacía se calcula mediante una **función de retroalimentación**, la cual evalúa el contenido que tenían las demás celdas en el estado anterior.

Dependiendo de si la función de retroalimentación es lineal o no, se dirá que los registros de desplazamiento son lineales (LFSR) o no (NLFSR).

Característica	Lineales	No lineales
Tipo de función	Lineal, consiste en operaciones XOR de un conjunto prefijado de los estados del registro	No lineal, puede ser cualquier función booleana basada en los estados del registro
Periodo de secuencia	$2^n - 1$	2^n
¿ Puede pasar por el estado donde todas las celdas valen 0 ?	No	Si
Facilidad de implementar (hardware)	Extremadamente fáciles	Muy fáciles
Facilidad de plantear matemáticamente	Extremadamente fáciles	Difícil (para secuencias largas)
Dificultad criptográfica	Baja, la secuencia puede ser predicha con una porción relativamente pequeña ($2n$) de la misma	Alta
Ejemplo	$S_0 = S_1 \oplus S_4$	$S_0 = ((S_1 \wedge S_2) \vee \neg S_3) \oplus S_4$

Dado que los LFSR son muy eficientes y rápidos en hardware, pero algebraicamente débiles, y los NLFSR son seguros, pero más complejos de plantear para secuencias largas, los diseñadores criptográficos optan por una solución híbrida.

Para obtener secuencias seguras y rápidas, se suelen combinar las salidas de n generadores lineales (LFSR) independientes a través de una función matemática no lineal. El resultado de esta combinación es el que se utiliza como cada bit de la secuencia cifrante final, logrando así lo mejor de ambos enfoques.

Un sistema híbrido con 3 LSFR de longitudes 11, 9 y 10 bits, combinados con una función booleana $Z = (X_1 \wedge X_2) \oplus X_3$, ya ofrece una secuencia pseudoaleatoria mucho mas robusta que un único registro. Otro ejemplo, usado en telefonía GSM es la función $Z = X_1 \oplus X_2 \oplus X_3$ combinando 3 LSFR.

7.6. Sistemas criptográficos asimétricos.

También llamados de clave pública, introducidos por Whitfield Diffie y Martin Hellman, revolucionaron la criptografía al usar un **par de claves diferentes**: una **pública** (que se comparte libremente) y una **privada** (que se mantiene en secreto y no se comparte JAMAS, ni de forma secreta). Lo que se cifra con una clave solo puede ser descifrado por la otra.

Tienen dos usos principales:

- **Protección de información:** El emisor cifra el mensaje usando la clave *pública* del destinatario, garantizando que solo el destinatario podrá leerlo usando su clave *privada*.
- **Firma digital / Autenticación:** El emisor cifra un resumen del mensaje con su clave *privada*. Cualquiera puede usar la clave *pública* del emisor para leerlo y comprobar que indudablemente provino de él.

Características:

- Velocidad: Lenta (cientos de veces más lenta que simétrica)
- Distribución de claves: Resuelta
- Ejemplos: RSA, Diffie-Hellman, ElGamal

Generar estas claves se logra mediante funciones unidireccionales: Función $f(x)$ fácil de calcular, pero computacionalmente inviable de invertir, es decir de encontrar x dado $f(x)$, ejemplos:

- Factorización de números:
 - Es fácil calcular $N = p \times q$ conociendo p y q
 - En cambio, encontrar p y q conociendo N es muy difícil de revertir
 - Siempre y cuando p y q sean suficientemente grandes
- Logaritmo discreto:
 - Es fácil calcular $a = b^x \% p$ conociendo b , x y p
 - Es un problema intratable encontrar x conociendo a , b y p
 - Siempre y cuando se usen números suficientemente grandes

A diferencia de los sistemas simétricos que solo pueden romperse con fuerza bruta, los asimétricos son vulnerables a análisis matemáticos (como la aparición de algoritmos de factorización mas eficientes, o reducir el espacio de búsqueda de claves entre muchos otros) por ello una clave simétrica de 128 bits se considera segura, pero una asimétrica requiere 2048 bits para ofrecer una protección equivalente.

Notas (otra vez):

- Cuando se menciona el inverso multiplicativo x^{-1} de un numero x es un numero tal que: $x * x^{-1} \equiv 1 \pmod{N}$
- Una condición común para garantizar la existencia del inverso multiplicativo es que x y N sean coprimos (primos entre si), es decir que no tengan divisores comunes además del 1. Otra forma de garantizar esto es hacer que N sea primo, dado que no tiene divisores además del 1.

7.6.1. Protocolo Diffie-Hellman

Este no es un criptosistema, sino un protocolo diseñado para el **intercambio seguro de claves** a través de un canal de comunicación inseguro, que fue lo que dio pie a los criptosistemas asimétricos. Su fortaleza radica en el problema matemático de los **logaritmos discretos**. **Funcionamiento:**

Dos interlocutores (por ejemplo, Alice y Bob) acuerdan de manera pública dos números: un número primo grande p (1024 bits mínimo) y un generador g de Z_p^* (grupo multiplicativo de un campo de Galois).

- El tamaño de p es lo que incrementa el tamaño máximo del espacio de búsqueda para ataques de fuerza bruta, el hecho de que sea primo impide factorizarlo y acortar el espacio de búsqueda.
- Que g sea generador de Z_p^* garantiza que la expresión g^x puede generar todos los valores desde 1 hasta $p - 1$, evitando acortar el espacio de búsqueda.
- Es recomendable que $p - 1$ tenga al menos un factor primo grande. Esto evita algoritmos que resuelven el logaritmo discreto de forma eficiente cuando los factores de $p - 1$ son pequeños.

Alice escoge un número secreto a y calcula un valor público $A = g^a \% p$.

De forma paralela, Bob elige un número secreto b y calcula $B = g^b \% p$

Ambos intercambian abiertamente sus resultados A y B . Finalmente, Alice toma el número recibido de Bob y calcula $B^a \% p$, mientras que Bob calcula $A^b \% p$

Gracias a las propiedades matemáticas, ambos cálculos dan exactamente $g^{ab} \% p$, obteniendo así una clave común sin que sus números a y b hayan viajado nunca por la red.

Luego la clave $g^{ab} \% p$ es usada en un cifrado simétrico.

Nota: Yo lo llamaría protocolo de generación o acuerdo de clave secreta, porque lo que intercambian son números secretos no claves, lo que hacen es calcular la clave en base a esos números, pero bueno, son mis pensamientos.

7.6.2. Algoritmo RSA

El algoritmo RSA (desarrollado por Rivest, Shamir y Adleman) basa su seguridad en la intratabilidad computacional del problema de factorizar números enteros muy grandes. Este es el más conocido y utilizado a nivel global. Su funcionamiento se estructura de la siguiente manera:

Generación de claves (se hace del lado receptor):

- Se eligen aleatoriamente dos números primos grandes p y q , y se calcula el módulo $n = p * q$
- Calcula la función de Euler para este módulo: $\phi(n) = (p - 1)(q - 1)$
- Se escoge un exponente público e que sea coprimo con $\phi(n)$. (es decir $\text{mcd}(e, \phi(n)) = 1$)
- Se calcula el exponente de descifrado d (usando algoritmos como el Extendido de Euclides), que es la inversa multiplicativa de e módulo $\phi(n)$, es decir: $d * e \% \phi(n) = 1$

La clave pública es el par (e, n) que se coloca en un directorio de acceso libre, mientras que la clave privada es el par (d, n) (o (d, p, q)) y debe mantenerse en estricto secreto.

Para cifrar un mensaje secreto m (convertido previamente a un número entre 1 y $n - 1$), el emisor utiliza la clave pública del destinatario calculando el criptograma:

$$C_i = (M_i)^e \text{ mod } n$$

El destinatario recupera el mensaje original aplicando su clave privada mediante la operación:

$$M_i = (C_i)^d \text{ mod } n$$

Seguridad: Para ser seguro, se recomienda que la longitud del módulo n sea de al menos 2048 bits. Además, presenta vulnerabilidades en su implementación si se usan valores de exponente e demasiado bajos o si múltiples usuarios comparten un módulo n común.

Para referencia: $2^{1024} = 179769313486231590772930519078902473361797697894230657273430081157732675805500963132708477322407536021120113879871393357658789768814416622492847430639474124377767893424865485276302219601246094119453082952085005768838150682342462881473913110540827237163350510684586298239947245938479716304835356329624224137216$

7.6.3. Firmas digitales

Es el análogo digital de la firma convencional en papel, una **firma digital** es una secuencia de bits que se añade a una pieza de información cualquiera, y que permite garantizar su autenticidad de forma independiente del proceso de transmisión.

En la práctica, se construye aplicando una función resumen al mensaje y cifrando ese resumen con la clave privada del emisor (cifrado asimétrico).

Una **función resumen** (también llamadas funciones **hash**) son algoritmos que transforman un mensaje de cualquier longitud en una secuencia de bits de **longitud fija** (el resumen o hash), que actúa como una **huella digital** del documento original. Pueden ser extremadamente simples como sumar los bytes con aritmética modular a algoritmos

Para que una función resumen sea considerada segura, debe cumplir con características fundamentales: es **fácil de calcular** a partir de un mensaje, pero es **computacionalmente imposible** recuperar el mensaje original desde el resumen o encontrar dos mensajes distintos que produzcan el mismo resultado (resistencia a colisiones).

Entonces quien recibe un documento firmado digitalmente busca la clave pública del emisor y descifra el resumen, luego debe saber que función resumen se utilizó y para verificar que la información transmitida no haya sido modificada vuelve calcular el resumen, si el valor resumen descifrado coincide, quiere decir que el documento no fue modificado y que el autor es quien dice ser, pues para cifrar el resumen correctamente necesita conocer la clave privada.

Esto garantiza la identidad del emisor y la integridad de los datos.

7.6.3. Algoritmo ElGamal

El **algoritmo de ElGamal** es un sistema criptográfico asimétrico (de clave pública) que fue diseñado en un principio para la producción de firmas digitales, pero posteriormente su uso se extendió también para el cifrado de mensajes.

Su seguridad matemática se fundamenta en la intratabilidad del problema de los **logaritmos discretos** y está íntimamente relacionado con el protocolo de Diffie-Hellman.

Para generar un par de claves, un usuario lleva a cabo los siguientes pasos:

1. Se escoge un número primo n grande
2. Se busca un generador g de Z_p^* (grupo multiplicativo de un campo de Galois).
3. Escoge un número aleatorio x menor que n
4. Se calcula $y = g^x \pmod{n}$

La **clave pública** queda constituida por el trío de valores (g, y, n) , mientras que la **clave privada** es el número x

Firmas digitales:

Para firmar un mensaje m , el emisor (el único que conoce x) lleva a cabo el siguiente proceso:

- Elige un número aleatorio secreto k , el cual debe ser coprimo con $(n - 1)$. Este número k debe ser distinto cada vez que se firme un documento.
- Calcula el resumen del mensaje $f(m)$
- Calcula dos valores matemáticos que conformaran la firma:
 - $a = p^k \pmod n$
 - $b = (f(m) - x * a) * k^{-1} \pmod{n - 1}$
- La firma digital es el par (a, b)

El receptor, para confirmar que la firma es auténtica, comprueba que se cumpla la siguiente igualdad matemática:

$$y^a * a^b \equiv p^{f(m)} \pmod n$$

Si la igualdad se cumple, la firma es legítima.

Notas:

- Las 3 condiciones sobre p (n en este caso) y g del protocolo Diffie-Hellman también aplican sobre ElGamal.
- k debe ser coprimo con $n - 1$ para garantizar la existencia de su inverso multiplicativo, esto permite que el valor de k se cancele algebraicamente.
- El número k introduce aleatoriedad al proceso, haciendo que un mismo mensaje firmado dos veces produzca firmas distintas lo que frustra su análisis, si k fuera un valor conocido o no estuviera ($k = 1$), es posible despejar la clave privada x :

$$x = (f(m) - b * k) * a^{-1}$$

Como el mensaje m la función resumen f , la firma (a, b) y el valor k son conocidos, cualquiera podría calcular x . (a^{-1} no es difícil de calcular)

- Si el valor k fuera reutilizado, teniendo dos mensajes (m_1 y m_2) con sus firmas (a_1, b_1) y (a_2, b_2) bastaría para despejar k , lo que nos lleva al problema anterior.

Este despeje se logra planteando $b_1 - b_2$, lo que nos lleva a que:

$$k = (m_1 - m_2)(b_1 - b_2)^{-1}$$

La extensión de ElGamal: Cifrado de mensajes

Para enviar un mensaje m , el emisor utiliza la clave pública (g, y, n) del destinatario de la siguiente manera:

- Escoge un número aleatorio k que sea primo relativo (coprimo) con $(n - 1)$. Este valor k debe mantenerse en absoluto secreto.
- Calcula dos valores matemáticos:

$$\begin{aligned} a &= g^k \pmod n \\ b &= y^k * m \pmod n \end{aligned}$$

- El criptograma resultante es el par (a, b) . Una de las principales desventajas de este algoritmo es que, al generar dos valores por cada bloque de mensaje, **el texto cifrado tiene el doble de longitud que el texto original**.

Descifrado de mensajes: El receptor legítimo, que es el único que conoce la clave privada x , recupera el mensaje original efectuando la siguiente operación matemática con los valores recibidos a y b :

$$m = b * a^{-x} \pmod{n}$$

Notas: las mismas que para las firmas

- Si k fuera fijo o no estuviera ($k = 1$): $m = b * (y^k)^{-1}$
- Si k se reutiliza en 2 mensajes distintos: $(a, b_1) = (g^k, y^k * m_1)$ y $(a, b_2) = (g^k, y^k * m_2)$, partiendo de $b_1 \div b_2$ podemos despejar fácilmente: $m_1 \div m_2$, si bien no se puede despejar la clave x , se expone la relación exacta entre dos mensajes. Si el atacante conociera o adivinara el contenido de un mensaje (como una cabecera de archivo o un handshake o protocolo estándar), puede calcular inmediatamente el otro mensaje.
- Notar que a diferencia de las firmas no es posible despejar la clave privada x en ningún caso, esto se debe a la estructura algebraica de las ecuaciones, en las firmas x es parte de una ecuación lineal, lo que lo hace fácilmente despejable sin la protección de k , pero en el cifrado permanece en su forma exponencial $y = g^x$, siendo imposible despejarla sin resolver el problema del logaritmo discreto.

7.7. Criptografía de Curva Elíptica (ECC)

Es una de las áreas más prometedoras para mejorar los algoritmos asimétricos clásicos

Como habíamos comentado antes, el algoritmo original de ElGamal basa su seguridad en la dificultad de resolver logaritmos discretos usando aritmética modular con números enteros gigantescos. Lo que los criptógrafos descubrieron es que este mismo algoritmo se puede "mudar" o redefinir sobre una estructura matemática diferente: las curvas elípticas.

¿Qué es lo de la curva elíptica? Matemáticamente, una curva elíptica es un conjunto de puntos en un plano (x, y) que cumplen con una ecuación específica (del tipo $y^2 = x^3 + ax + b$). En lugar de tomar números gigantes y multiplicarlos entre sí (como hace el ElGamal tradicional), en la curva elíptica se define una operación geométrica para "**sumar**" puntos. Si tomas un punto p de la curva y lo sumas consigo mismo k veces, obtendrás un nuevo punto q (lo que se anota como $q = kp$).

El truco de magia aquí es que se genera un nuevo **Problema de Logaritmo Discreto en Curvas Elípticas**: si a un atacante se le da el punto original p y el punto final q , no existe ningún algoritmo eficiente que le permita deducir el número de veces (k) que el punto fue sumado.

7.8. Funciones resumen (para firmas digitales)

Una **función resumen** (o *hash*) es un algoritmo matemático que toma un mensaje de cualquier tamaño y lo transforma en una secuencia de bits corta y de **longitud fija**. Funciona como una "huella digital" o "ADN" de los datos.

Para que se considere segura, debe cumplir estas cuatro propiedades:

- **Longitud fija:** El resumen siempre tiene el mismo tamaño, sin importar si el archivo original es de 1 KB o de 1 GB.
- **Fácil de calcular:** El sistema debe poder generar el resumen del mensaje de forma rápida.

- **Irreversible (Unidireccional):** Es imposible hacer el camino inverso; es decir, no se puede reconstruir el mensaje original teniendo únicamente su resumen.
- **Resistente a colisiones:** Es computacionalmente imposible encontrar dos mensajes distintos que, al pasar por la función, produzcan exactamente el mismo resumen.

Usos:

- **Verificación de integridad:** Sirven para comprobar matemáticamente que un documento, archivo o mensaje no ha sufrido ninguna modificación (intencionada o accidental). Un receptor puede recalcular el resumen en base a lo recibido para compararlo con el resumen calculado por el emisor, si estos coinciden quiere decir que la información no fue modificada.
- **Firmas Digitales:** Optimizan y hacen viable el proceso de firma. Debido a que el cifrado asimétrico es computacionalmente lento, no se cifra el documento completo; en su lugar, se calcula su función resumen y se cifra únicamente esa pequeña "huella digital" con la clave privada del emisor. Esto permite a los receptores del mensaje, tener una garantía de quien fue el emisor, y que el mensaje no fue modificado.

Algunas de las funciones resumen más famosas son:

MD5 (Message-Digest Algorithm): MD5 (y sus predecesores como MD4) es uno de los algoritmos más famosos e históricamente ha sido muy utilizado en la comunidad de software libre. Funciona procesando el mensaje de entrada en bloques de 512 bits y produce como resultado una huella o resumen de **128 bits**. Pero recientemente se encontraron colisiones prácticas dentro de este, así que no es recomendado donde se requiera alta seguridad. Funcionamiento:

1. **Relleno (Padding):** El algoritmo no procesa el archivo tal cual. Primero se lo rellena hasta que el tamaño total sea un múltiplo exacto de 512 bits. Añadiendo un bit "1" seguido de tantos ceros como sea necesario y al final añade 64 bits que indican la longitud original del mensaje.
2. **Procesamiento:** El mensaje se divide y procesa en bloques de **512 bits**.
3. **Operaciones (Rondas):** El algoritmo inicializa 4 registros o variables de 32 bits (A, B, C, D). Por cada bloque de 512 bits, ejecuta un bucle principal compuesto por **4 "rondas" de 16 operaciones** cada una. Estas operaciones mezclan los datos usando funciones lógicas no lineales complejas (XOR, AND, OR, NOT) y desplazamientos circulares (son muy complejas para poner por escrito aquí).
4. **Resultado:** Al finalizar con todos los bloques, se concatenan los 4 registros para obtener la huella final de **128 bits**.

SHA (Secure Hash Algorithms): Fueron desarrollados conjuntamente por el NIST (Instituto Nacional de Estándares y Tecnología de EE.UU.) y la NSA (Agencia de Seguridad Nacional).

- **SHA-0:** Fue la versión original lanzada por la NSA, pero fue rápidamente retirada y sustituida por el SHA-1 para corregir una debilidad en su diseño.
- **SHA-1:** Es el más representativo de esta familia clásica. Procesa los mensajes también en bloques de 512 bits, pero genera un resumen más largo y seguro que el MD5, concretamente de **160 bits**. Similar a MD5 se encontraron colisiones posteriormente.

- **SHA-256:** Mucho más seguro.

Funcionamiento: muy similar a MD5, relleno y separando en bloques de 512 bits, con la diferencia que usa 5 registros en vez de 4, lo que lleva a generar un resumen de 160 bits y que su ciclo principal es mas extenso, ejecutando 4 rondas de 20 operaciones matemáticas y lógicas para mezclar la información.

7.9. Comparacion métodos de seguridad

El cifrado asimétrico requiere cálculos matemáticos inmensos (exponenciaciones con números gigantes), lo que lo hace computacionalmente muy costoso y lento. De hecho, la criptografía simétrica es unas 4000 veces más rápida que la asimétrica. Por lo tanto, intentar cifrar un archivo de video, un documento grande o el tráfico continuo de una videollamada usando RSA sería inviable e ineficiente.

La solución en la práctica (Sistemas Híbridos) Para resolver esto, los sistemas modernos de comunicación estructuran el proceso en las siguientes fases:

- **Para los datos (Cifrado Simétrico):** Para enviar un mensaje largo o transmitir datos en volumen, el sistema genera aleatoriamente una **clave de sesión** (una clave simétrica de un solo uso) y cifra el documento usando un algoritmo simétrico rápido y robusto (como AES).
- **Para la clave de sesión (Cifrado Asimétrico):** Como el receptor necesita esa clave simétrica para descifrar el documento, esta se envía usando cifrado asimétrico, garantizando que solo el destinatario pueda leer la clave.
- **Para la integridad y autoría (Funciones Resumen + Asimétrico):** Para garantizar quien envió el mensaje y que nadie lo modificó, se usa el **Hash** (la función resumen) del documento y se cifra únicamente esa pequeña huella digital con la **clave privada del emisor** (Firma Digital).

Al llegar al destino, el receptor usa su clave privada para descubrir la clave de sesión simétrica, y con ella descifra de forma rápida todo el volumen de datos. Al mismo tiempo, usa la clave pública del emisor para descifrar el hash y verificar la firma.

Extra – Porque se usan tantos números primos/coprimos en criptografía (eliminar?)

Los números primos son principalmente para que las claves o números cifrados no se puedan “simplificar” o descomponer en factores mas pequeños que los hagan más manejables (mientras más grande los números más tiempo toma de descifrar, pero si se pudieran simplificar los números reduciría el tiempo necesario)

Y también sirven para las secuencias pseudoaleatorias, garantizar que sean más largas e impredecibles. Ya que garantizan usar todo el rango de valores disponibles antes de repetir la secuencia (como en las tablas hash), además de que los números normales empezarían a exhibir patrones.

Las condiciones de coprimalidad de un numero a es usada para garantizar que exista a^{-1} es decir su inverso multiplicativo y no solo eso, si no que exista uno solo, el cual es necesario para poder deshacer operaciones de cifrado por ejemplo sin ambigüedades.

UNIDAD N°8 - CONTROL DE ERROR

8.1. Errores

En cualquier sistema de comunicación, el canal introduce ruido que puede alterar los bits transmitidos, estos bits alterados son errores, existen 2 tipos:

- **Errores aleatorios o independientes:** son aquellos donde la causa de un error en un bit no guarda relación con un error en otro bit, con igual probabilidad de aparición en cualquier posición. Se asocian típicamente al **ruido blanco (ruido de ambiente)**.
- **Errores en ráfaga:** Son un conjunto de bits en una longitud corta que fueron afectados por una causa común (por ejemplo, una caída de tensión o una perturbación magnética).
 - **Longitud de ráfaga (L):** Número de bits desde el primer error hasta el último.

8.2. Introducción

Las metodologías de control de error se tratan de agregar bits de redundancia para que el receptor pueda detectar o corregir errores.

- Los códigos detectores permiten identificar cuando ha ocurrido un error, permitiendo realizar una acción como pedir retransmisión o descartar los datos.
- Los códigos correctores permiten detectar y corregir los errores, pero necesitan muchos más bits de redundancia en comparación de los códigos detectores.

Cabe destacar que nosotros solo trabajaremos con metodologías lineales de control de error, que mediante una transformación lineal agregan los bits de redundancia (existen no lineales).

8.3. Como funcionan los códigos de control de error

Para implementar una metodología de control de error, se debe hacer un análisis del canal para detectar la tasa y tipos de error, para poder seleccionar una metodología de control de error apropiada.

Luego, para proteger una palabra de k bits de información, las metodologías codifican esa información en una palabra código de n bits, agregando $n - k$ bits de redundancia.

Redundancia del código: $(n \div k) * 100$ porcentaje de bits de redundancia en el código, mientras más menor es, mejor.

Palabra válida (o palabra código): Es la cadena final de longitud n conformada por la adecuada combinación lineal de los k bits de información originales más los $n - k$ bits de redundancia.

Distancia de Hamming: Número de posiciones en las que dos palabras de igual longitud difieren.

$$d_H(u, v) = \sum_{i=1}^n (u_i \oplus v_i)$$

Ej.: $d_H(0111, 0101) = 1$ $d_H(1000, 1111) = 3$ $d_H(0110, 1001) = 4$

Distancia mínima: Menor distancia de Hamming (diferencias en bits) entre cualquier par de palabras válidas distintas en un código K_n .

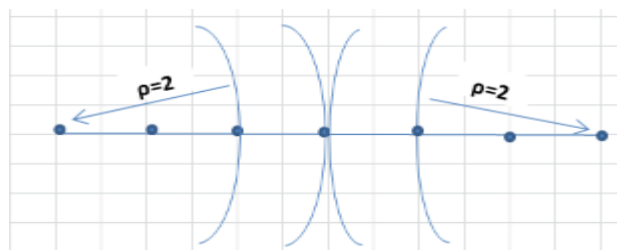
$$d_{min}(K_n) = \min \{d_H(a, b) \mid a, b \in K_n \wedge a \neq b\}$$

Para comprender por qué los bits de redundancia permiten diferenciar las palabras a pesar de los errores, podemos utilizar un análisis desde un punto de vista espacial. Imaginemos un espacio donde cada **palabra válida es el centro de una esfera**. La distancia mínima sería la separación que existe entre los centros de estas esferas.



Si tomamos el ejemplo de un código con una **distancia mínima igual a 6** entre sus palabras válidas:

- **Si funciona solo como detector:** El código es capaz de **detectar hasta 5 errores**.
Visualmente, cualquier salto de hasta 5 posiciones o "bits" no será suficiente para caer exactamente en el centro de otra esfera (otra palabra válida), por lo que el receptor siempre notará que el mensaje recibido es inválido.
Si ocurren 6 errores o más, aún pueden ser detectados, siempre y cuando no se imite otra palabra válida, pero el código no lo garantizaría.
- **Si funciona como corrector:** El código podrá **corregir hasta 2 errores y, simultáneamente, detectar un 3er error**.
 - **Para corregir:** Trazamos esferas de "radio 2" alrededor de cada palabra válida. Si el mensaje sufre 1 o 2 errores, caerá dentro de una de estas esferas y se aproximará (corregirá) al centro correspondiente.



- **Si ocurren 3 errores:** El mensaje caerá a la misma distancia de dos palabras válidas distintas. El sistema detectará la presencia del error, pero al estar en una zona ambigua, no sabrá a cuál centro aproximarlo.
- **Si ocurren 4 o más errores:** El mensaje erróneo invadirá la esfera de corrección de *otra* palabra válida, lo que provocará que el sistema realice una corrección equivocada hacia un mensaje que no fue el que originalmente se transmitió.

Aquí cabe destacar la importancia del análisis del canal de transmisión a la hora de seleccionar una metodología del control de error, dado que se debe buscar que los errores (según tipo y cantidad) que el canal presenta habitualmente sean manejables por la capacidad de la metodología elegida.

Nota: respecto a ver todo como un espacio o rango y esferas, es técnicamente incorrecto, en realidad es un hiperespacio e hiperesferas, de n dimensiones, donde cada dimensión o eje varía entre 0 y 1, así que es difícil imaginarlo correctamente pero sirve para orientarse.

8.4. Capacidad del código en base a su distancia mínima

La distancia mínima es el concepto que determina de forma intuitiva y matemática la capacidad de detección y corrección de un código:

- Un código detector de distancia mínima d_{min} puede detectar hasta $d_{min} - 1$ errores
- Un código corrector puede garantizar la corrección de hasta t errores si se cumple la relación:

$$d_{min} \geq 2t + 1 \quad \Rightarrow \quad t = \left\lfloor \frac{d_{min} - 1}{2} \right\rfloor$$

- Dicho de otra forma, un código corrector de distancia mínima d_{min} puede corregir hasta $\left\lfloor \frac{d_{min}-1}{2} \right\rfloor$ errores y detectar hasta $\left\lfloor \frac{d_{min}}{2} \right\rfloor$ errores.

Nota: $\lfloor x \rfloor$ es la parte entera de x , es decir es x truncado.

Ejemplos:

d_{min} del código	Usado como código detector	Usado como código corrector	
	Capacidad de detección	Capacidad de detección	Capacidad de corrección
2	1	1	0
3	2	1	1
4	3	2	1
5	4	2	2
6	5	3	2
7	6	3	3
8	7	4	3
...	...	...	...
16	15	8	7
17	16	8	8
18	17	9	8
...	...	...	...

Luego, la cota de Singleton establece: $d_{min} \leq n - k + 1$, es decir que la distancia mínima no puede ser mayor que la cantidad de bits de redundancia más uno.

De aquí se distingue:

- $d_{min} - 1 \leq n - k$, es decir la máxima cantidad de palabras invalidas entre 2 palabras validas es igual a la cantidad de bits de redundancia. Lo que nos lleva a que la capacidad de un código de detección como máximo es igual a la cantidad de bits de redundancia.
- $t \leq \left\lfloor \frac{n-k}{2} \right\rfloor$, es decir la cantidad máxima de errores que un código corrector puede corregir es la mitad de la cantidad de bits de redundancia (truncado).

8.5. Chequeadores de paridad simple.

Bit de paridad: Uno de los códigos de control de error más sencillo, consta en agregar un bit a un mensaje para detectar errores. Existen dos tipos:

- **Paridad par:** El bit extra tomará un valor tal que la cantidad total de unos en el mensaje final sea par. Es decir, se agrega un "1" si el número de unos en la información original es impar, y un "0" si ya era par.
- **Paridad impar:** El bit extra tomará un valor tal que la cantidad total de unos en el mensaje final sea impar.

El lado receptor al recibir el mensaje debería calcular por sí mismo la paridad, y si la paridad no coincide con la acordada (par o impar), detecta el error.

Ejemplo: Sea transmitir el mensaje de información: **001011** (tiene tres unos, cantidad impar).

- **Con paridad par:** Se le agrega un 1 al final para que la cantidad total de unos sea par (cuatro). Palabra válida final: **0010111**.
- **Con paridad impar:** Se le agrega un 0 al final para mantener la cantidad total impar. Palabra válida final: **0010110**.

¿Qué pasa si hay errores? Si enviamos 0010111 (paridad par) y el receptor recibe 0110111 (un error en la segunda posición), el receptor contará cinco unos (impar). Al no cumplirse la paridad par, solicitará la retransmisión. Si, en cambio, recibe 0111111 (dos errores), contará seis unos (par) y no detectará el error.

El problema de ese código es que solo detecta cantidades de errores impares, dado que si hubiera una cantidad de errores par (2, 4, 6, etc.), el bit de paridad daría el mismo resultado.

Dado que el código solo garantiza detectar de forma fiable 1 error, su distancia mínima es 2, es decir que con 2 cambios se podría transformar una palabra valida en otra palabra valida.

8.6. Códigos de números y letras, códigos pesados.

Para salir de la codificación estrictamente binaria, existen códigos orientados a errores tipográficos humanos (como confundir una 'O' con un '0', o intercambiar caracteres de lugar). Se los llama **códigos pesados**.

Un ejemplo es el **código base 37**, compuesto por 37 elementos: los 10 dígitos decimales (pesos 0-9), las 26 letras del abecedario (pesos A=10, B=11... Z=35) y el espacio en blanco (peso SP=36). Su protección consiste en agregar un símbolo de paridad extra tal que, al realizar una **suma de todos los símbolos pesados por la posición** que ocupan, el resultado sea un número divisible exactamente por 37 (módulo 37). Si el resto no es nulo en el receptor, se asume un mensaje erróneo y se pide retransmisión.

Es decir, siendo el mensaje $M = s_1 s_2 \dots s_n$ donde s_i es un símbolo del código, y sea sum la suma de los pesos de los símbolos del mensaje, se le agrega un símbolo de paridad S tal que:

$$sum + S \% 37 = 0$$

El peso de este se calcula como $S = 37 - (sum \% 37)$

8.6.1. Códigos ISBN.

El código ISBN (International Standard Book Number) es un **código pesado** utilizado para identificar libros a nivel mundial, Ej.: 0-13-065432-9. Su base de codificación es la **base 11**, contando con los elementos del 0 al 9 más el símbolo 'X'. La longitud de los códigos es de 10 caracteres:

- 1º dígito: Idioma en el que se escribe.
- 2º y 3º: Editorial que realiza la publicación.
- Sigüientes seis: Corresponden con el nombre y reseñas específicas.
- Última posición (10º): Símbolo de control.

Es decir, los dígitos 1 a 9 son la información, y decimo digito es la redundancia.

Este último se calcula para cumplir esta ecuación:

$$(x_1 * 10 + x_2 * 9 + x_3 * 8 + x_4 * 7 + x_5 * 6 + x_6 * 5 + x_7 * 4 + x_8 * 3 + x_9 * 2 + x_{10} * 1) \% 11 = 0$$

De la forma:

$$x_{10} = 11 - (x_1 * 10 + x_2 * 9 + x_3 * 8 + x_4 * 7 + x_5 * 6 + x_6 * 5 + x_7 * 4 + x_8 * 3 + x_9 * 2 \% 11)$$

Siendo los x_i los símbolos del código y los números dentro de la suma son los pesos de cada posición de símbolo, y el 11 es la base del código.

Lo que nos permite esta ecuación es asegurar la validez de la secuencia completa y detectar posibles alteraciones manuales o de transmisión.

Ejemplo: Deseamos encontrar el código de control del ISBN 0-13-065432-?, hacemos la suma basada en peso

$$0 * 10 + 1 * 9 + 3 * 8 + 0 * 7 + 6 * 6 + 5 * 5 + 4 * 4 + 3 * 3 + 2 * 2 = 123$$

Ahora debemos encontrar x tal que $(123 + x * 1) \% 11 = 0$

$$123 \% 11 = 2$$

$$x = 11 - 2 = 9$$

Entonces el código ISBN completo es **0-13-065432-9**, ahora cualquiera que reciba el código puede averiguar si un dígito cambio, esto es útil para, por ejemplo, asegurarse que el bibliotecario cargo el código correctamente, si el bibliotecario se equivocara en un carácter, el código lo detectaría, sin la necesidad de consultar una base de datos. En cambio, sí se equivoca en más de un carácter, la suma podría resultar de igual manera y el código no necesariamente detectaría los errores. ($d_{min} = 1$)

UNIDAD Nº9 - CÓDIGOS CORRECTORES DE ERROR

9.1. Introducción. Necesidad de la corrección de errores.

Los códigos correctores de errores aprovechan de la distancia mínima de un código para corregir palabras erróneas a la palabra correcta más similar, sin embargo, este necesita una redundancia mayor para ser capaz de corregir la misma cantidad de errores que un código detector puede detectar.

Estos son utilizados en ciertas situaciones donde retransmitir los datos es inviable (televisión digital, aplicaciones en tiempo real), o ineficiente (redes congestionadas, comunicación satelital), datos

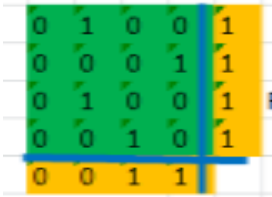
almacenados (dvd, bancos de datos?), por lo cual es conveniente transmitir con suficiente redundancia para que los errores puedan ser corregidos inmediatamente y sin retransmisión.

9.2. Arreglos gráficos: Código rectangular, triangular, cubico y N-dimensional.

Los **arreglos gráficos** son estrategias de corrección geométrica que consisten en disponer los bits de información en distintas dimensiones (conformando filas, columnas, o estructuras espaciales) e intercalar bits de paridad. **Al ocurrir un error, la intersección de los chequeadores (paridades) erróneos permite ubicar la posición exacta del bit afectado y corregirlo invirtiendo su valor.**

9.1.1. Código rectangular

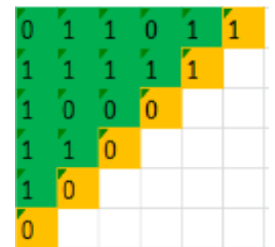
Consiste en ubicar los bits de información en un arreglo bidimensional formando N filas y M columnas. Por cada fila y columna se calcula un bit de paridad. Se presenta en dos variantes:

- **Sin bit esquina:** Su distancia mínima es tres, por lo que actúa como un **corrector simple** (detecta y corrige un error). Su redundancia se calcula mediante la fórmula $\frac{(M+1)*(N+1) - 1}{M*N}$. Para que la redundancia sea la menor posible, el arreglo debe aproximarse lo máximo posible a la forma de un cuadrado.
- 
- **Con bit esquina:** Se adiciona un bit de paridad extra que resulta de la suma lógica de la columna o de la fila de chequeadores (es decir, una paridad de las paridades). Si bien esto empeora la redundancia, la cual pasa a ser $\frac{(M+1)*(N+1)}{M*N}$, **la distancia mínima del código pasa a valer cuatro**. Esto significa que, además de funcionar como corrector, adquiere una mejor capacidad para la detección de errores.

2. Código Triangular

Esta metodología busca mejorar la eficiencia y tiene menor redundancia que cualquier arreglo rectangular (aproximadamente un 40% menos). Los bits de información se ordenan formando una especie de escalera invertida o triángulo. De este modo, a cada chequeador de paridad le corresponden bits tanto de una fila como de una columna (con excepción del primero y el último). Sigue siendo un código con distancia mínima de tres (corrector simple) y su redundancia está

dada por la expresión $\frac{M * \frac{M-1}{2} + M}{M * \frac{M-1}{2}}$ que simplificando es $1 + \frac{2}{M-1}$

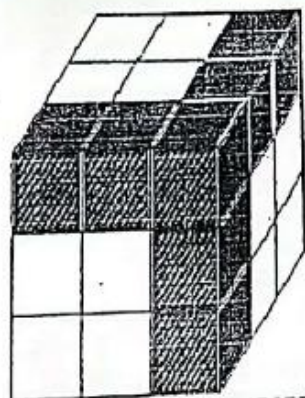


Nota: $M * \frac{M-1}{2}$ es la cantidad de elementos dentro del triángulo (lo verde) siendo M la longitud de la primera fila o columna. A su vez la cantidad de bits de paridad es M

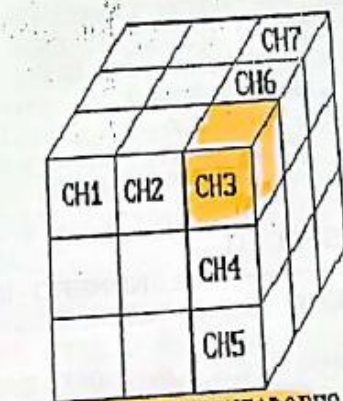
3. Código cúbico

La estrategia salta a la tercera dimensión, imaginando todos los bits de información y control **distribuidos en un arreglo espacial similar al formato de un cubo de Rubik**. En este caso, el control que efectúa cada chequeador consiste en ser la paridad lógica de todos los bits correspondientes a una "cara" del cubo. Al igual que en los arreglos 2D, la intersección geométrica de los chequeadores erróneos indica exactamente la posición del bit tridimensional a corregir. Su

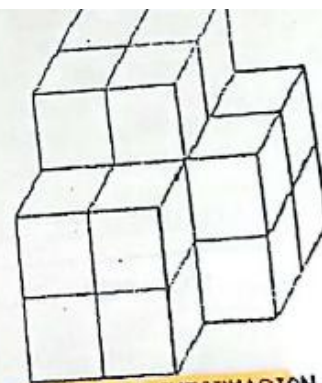
fórmula de redundancia es $\frac{N^3}{N^3 - (3N - 2)}$



POSICION DE CHEQUEADORES
EN EL ARREGLO.

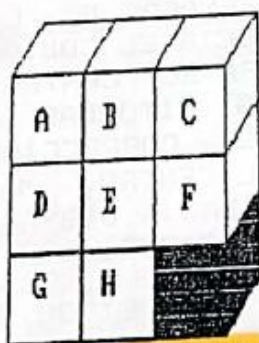


CANTIDAD DE CHEQUEADORES

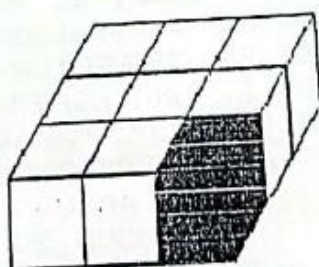


DIGITOS DE INFORMACION

EJEMPLO: $A+B+C+D+E+F+G+H = CH6 \text{ O } CH7$



DIGITOS QUE CONTROLAN
CH6 Y CH7



DIGITOS QUE CONTROLAN
CH4 Y CH5



DIGITOS QUE CONTROLAN
CH1 Y CH2

9.3. Aproximación geométrica

La **aproximación geométrica** aborda el problema de la corrección de errores desde una perspectiva analítica espacial. Su objetivo principal es responder a la pregunta: *¿Cuántos dígitos totales (N) son estrictamente necesarios para proteger una cantidad dada de dígitos de información (K)?*

Para resolverlo, se imagina un espacio constituido por la totalidad de combinaciones binarias posibles para una longitud N , cuyo volumen total es 2^N . Dentro de este espacio, se ubican las **palabras válidas**, que actuarán como el centro de "esferas" de protección. La cantidad de estas esferas es igual al número de combinaciones de información (2^K).

Para que el código corrija errores satisfactoriamente, **las esferas no deben superponerse**. Por lo tanto, el volumen total del espacio (2^N) dividido por el volumen que ocupa cada esfera debe ser mayor o igual al número total de esferas (2^K):

$$\frac{2^N}{\text{Volumen de la esfera}} \geq 2^K$$

El volumen de la esfera depende de la capacidad de corrección del código (no confundir con distancia mínima), para ser más específico, la capacidad de corrección t es el radio de cada esfera (en realidad son hiperesferas, es decir de más de 3 dimensiones), y el volumen de una esfera de radio P se calcula como: $\sum_{j=0}^t \binom{N}{j}$. Notar que el numero combinatorio $\binom{N}{j}$ representa la cantidad de

palabras de N bits con j bits alterados, lo que nos lleva a que el volumen de la esfera es la cantidad total de palabras modificadas que la capacidad de corrección P puede tratar correctamente.

Nota: $\binom{N}{j} = \frac{N!}{(N-j)! * j!}$

$$\binom{N}{0} = 1 \quad (\text{centro de la esfera, palabra de } N \text{ bits sin alterar})$$

$$\binom{N}{1} = \frac{N!}{(N-1)! * 1} = \frac{N * (N-1)!}{(N-1)!} = N \quad (\text{palabra de } N \text{ bits con } 1 \text{ alteracion})$$

$$\binom{N}{2} = \frac{N!}{(N-2)! * 2} = \frac{N * (N-1) * (N-2)!}{(N-2)! * 2} = \frac{N * (N-1)}{2} \quad (\text{palabra de } N \text{ bits con } 2 \text{ alteraciones})$$

Si buscamos un **corrector simple** ($t = 1$, un solo error), el volumen de la esfera resulta ser $1 + N$. La inecuación final queda: $\frac{2^N}{1+N} \geq 2^K$

De este análisis es de donde se desprende la cota de Singleton de la cual hablamos antes.

$$\frac{2^N}{\text{Volumen de la esfera}} \geq 2^K \Rightarrow \frac{2^N}{2^K} \geq \sum_{j=0}^t \binom{N}{j} \Rightarrow$$

muchos pasos que involucran series y numeros combinatorios despues ...

$$\Rightarrow 2^{N-K} > 2^{2t-1} \Rightarrow \dots \Rightarrow N - K + 1 \geq d_{\min}$$

9.4. Código de HAMMING

El **Código de Hamming** es una estrategia de codificación diseñada para lograr la corrección de un error simple, destacándose por ser la metodología que **menor redundancia agregada posee** entre los correctores simples.

Desde el punto de vista de la geometría analítica, el código de Hamming es catalogado como un **código perfecto**. En otras palabras, las esferas de protección trazadas alrededor de cada palabra válida ocupan todo el espacio posible sin solaparse y sin dejar "huecos" vacíos. Por ejemplo, para enviar 4 bits de información, reemplazamos en la fórmula: $\frac{2^N}{1+N} \geq 2^4 = 16$. Por prueba y error, se constata que el valor mínimo indispensable que satisface esta desigualdad es $N = 7$. Esto da origen al clásico **código Hamming (7,4)**, que utiliza 4 bits de datos y 3 de control ($7 - 4 = 3$), logrando la redundancia más baja posible. (Nota: su distancia mínima es 3)

Nota: después hay códigos de Hamming extendidos por eso a veces se menciona más de 3 bits de control y 4 de datos.

Su principio fundamental es lograr una **independencia absoluta entre los distintos dígitos chequeadores** (bits de paridad). Al repartir equitativamente la carga de trabajo, se asegura que la desaparición o fallo de un chequeador no deje posiciones sin controlar ni impida la corrección del error.

Hamming propuso una regla estricta para asegurar que todos los dígitos de control (chequeadores) trabajen la misma cantidad y sean totalmente independientes. Para lograrlo, las posiciones de la palabra transmitida se numeran en binario (1, 2, 3, 4, 5, etc.) y se distribuyen de la siguiente manera:

- **Posiciones de Control (Potencias de 2):** Los bits chequeadores se ubican exclusivamente en las posiciones 1, 2, 4, 8, 16, etc..
- **Posiciones de Información:** Los bits de datos originales se ubican en los huecos restantes (posiciones 3, 5, 6, 7, 9, etc.).

INFORMACIÓN					PALABRAS VÁLIDAS							
I1	I2	I3	I4		C1	C2	I1	C3	I2	I3	I4	
0	0	0	0		0	0	0	0	0	0	0	
0	0	0	1		1	1	0	1	0	0	1	
0	0	1	0		0	1	0	1	0	1	0	
0	0	1	1		1	0	0	0	0	1	1	
0	1	0	0		1	0	0	1	1	0	0	
0	1	0	1		0	1	0	0	1	0	1	
0	1	1	0		1	1	0	0	1	1	0	
0	1	1	1		0	0	0	1	1	1	1	
1	0	0	0		1	1	1	0	0	0	0	
1	0	0	1		0	0	1	1	0	0	1	
1	0	1	0		1	0	1	1	0	1	0	
1	0	1	1		0	1	1	0	0	1	1	
1	1	0	0		0	1	1	1	1	0	0	
1	1	0	1		1	0	1	0	1	0	1	
1	1	1	0		0	0	1	0	1	1	0	
1	1	1	1		1	1	1	1	1	1	1	

¿Qué controla cada chequeador? Cada chequeador es un bit de paridad par (que busca la paridad par) de conjunto de posiciones basándose en su representación binaria:

- **CH1 (Posición 1):** Controla todas las posiciones cuyo bit menos significativo sea un 1, posiciones impares: 1, 3, 5, 7, 9... (esto se incluye a si mismo) y busca que la paridad sea par, se calcula:

$$CH_1 = I_1 \oplus I_2 \oplus I_4$$

- **CH2 (Posición 2):** Posiciones cuyo segundo bit menos significativo sea un 1, posiciones 2, 3, 6, 7, 10, 11... (de 2 en 2, omitiendo 2).

$$CH_2 = I_1 \oplus I_3 \oplus I_4$$

- **CH3 (Posición 4):** Posiciones cuyo tercer bit sea un 1, posiciones 4, 5, 6, 7, 12, 13... (de 4 en 4 omitiendo 4).

$$CH_3 = I_2 \oplus I_3 \oplus I_4$$

Recordatorio del XOR, da 0 cuando recibe números iguales y da 1 cuando recibe números distintos. Aquí son las posibilidades de los cálculos de CH:

$$0 \oplus 0 \oplus 0 = 0 \quad 0 \oplus 0 \oplus 1 = 1 \quad 0 \oplus 1 \oplus 1 = 0 \quad 1 \oplus 1 \oplus 1 = 1$$

También puede ser vista como una suma normal (base 2) pero donde se elimina el overflow, tipo 10 (o 2 en decimal) es reemplazado por 0.

Si uno después ve los 4 números resultantes, siempre queda paridad par.

Posición Controlada	C3	C2	C1	
1			x	C1
2		x		C2
3		x	x	I1
4	x			C3
5	x		x	I2
6	x	x		I3
7	x	x	x	I4

9.4.1. Síndrome

Esto es lo que se usa después del lado receptor como comparación para saber si hubo error, se forma un vector (S_1, S_2, S_3) que se llama síndrome:

$$CH_1 \oplus I_1 \oplus I_2 \oplus I_4 = S_1$$

$$CH_2 \oplus I_1 \oplus I_3 \oplus I_4 = S_2$$

$$I_2 \oplus CH_3 \oplus I_3 \oplus I_4 = S_3$$

Si la transmisión fue correcta el síndrome debería ser $(0,0,0)$ dado al como se calcularon los dígitos de control para mantener la paridad par.

Por lo tanto si el síndrome no da $(0,0,0)$ sabemos que ocurrió un error, ahora, lo brillante del método de Hamming es que **el valor decimal del síndrome indica la posición exacta del bit erróneo**. Por ejemplo, si los chequeadores determinan un síndrome binario de $(1, 1, 0)$ (que es el número 6 en decimal), el receptor sabe que el error se encuentra en el sexto bit del mensaje recibido.

Si además vemos la tabla de las posiciones controladas el único error de un bit que puede modificar esos 2 cálculos es el de la posición 6. (Siempre y cuando sea un solo error)

Este es el brillante posicionamiento de los bits que Hamming diseño.

9.4.2. Código de Hamming extendido

Adicionando simplemente un bit de paridad de todas las posiciones (incluyendo los de control), podemos crear un nuevo **código de Hamming (8, 4)**, incrementando su distancia mínima a 4.

Comparación:

Código	d_{min}	Capacidad como detector	Capacidad como corrector	
			De detección	De corrección
Hamming (7,4)	3	2 errores	1	1
Hamming (8,4)	4	3 errores	2	1

9.5. Códigos equivalentes

Dos códigos se consideran **equivalentes** si es posible construir uno a partir del otro mediante operaciones básicas, principalmente reordenando o permutando las posiciones de los bits (es decir, intercambiando las columnas de su matriz) o permutando los símbolos en una posición fija.

La característica clave de los códigos equivalentes es que, a pesar de estas transformaciones visuales o de diseño, **conservan intactas todas sus propiedades geométricas y funcionales**. Es decir, ambos códigos mantienen exactamente la misma longitud de palabra, la misma cantidad de palabras válidas y, lo más importante, la misma **distancia mínima** entre ellas. Como resultado, dos códigos equivalentes tienen exactamente la misma capacidad para detectar y corregir errores.

9.6. Códigos de bloques lineales.

Reciben este nombre aquellos códigos de control de error en lo que los dígitos chequeadores son combinaciones lineales (multiplicación y luego suma) de los dígitos de información, por lo tanto, el código de Hamming pertenece a esta clase.

La idea de este tipo de códigos es generar una sola vez una matriz generadora $K \times N$ que nos permite calcular la palabra valida de forma fácil, sin tener que visitar un diccionario de: palabra de entrada -> palabra valida. Si los dígitos chequeadores no fueran combinaciones lineales de la palabra de información, no se podrían calcular mediante operaciones matriciales, por lo tanto, no se podrían representar en la matriz.

Ej: en Hamming el cálculo de cada bit de control se puede expresar como una simple multiplicación del vector de información por coeficientes dependiendo de que posiciones que controla cada bit de control (con las sumas hechas en base 2 para imitar el XOR):

$$CH_1 = I_1 \oplus I_2 \oplus I_4 = (1,1,0,1) * (I_1, I_2, I_3, I_4)$$

$$CH_2 = I_1 \oplus I_3 \oplus I_4 = (1,0,1,1) * (I_1, I_2, I_3, I_4)$$

$$CH_3 = I_2 \oplus I_3 \oplus I_4 = (0,1,1,1) * (I_1, I_2, I_3, I_4)$$

Recordar:

- N: longitud total de una palabra valida final
- K: dígitos de la información original
- N – K: cantidad de bits de control

Nota: los códigos de bloques lineales muchas veces van acompañados con la notación (n, k, d_{min})

9.6.1. Matriz generadora.

La matriz generadora G es la herramienta matemática que se utiliza en el extremo emisor para transformar el bloque de información original de K bits en una palabra código final N de bits. Se construye ensamblando una matriz identidad I (de dimensión $K \times K$) y una matriz de paridad P (de dimensión $K \times (N - K)$). Al presentarla en esta forma sistemática ($G = [I | P]$), se garantiza que los primeros bits de la palabra transmitida sean idénticos a los de la información original.

Tomando el ejemplo clásico de un código Hamming (7,4), la matriz generadora se ve así:

$$G = [I | P] = \begin{pmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{pmatrix}$$

La matriz P (paridad), es la representación matricial de **las combinaciones lineales utilizadas para el cálculo de los bits de paridad**. Es decir, son los vectores que se mencionó anteriormente, solo que colocados en la matriz, los cuales son usados para los cálculos de los bits de paridad.

Para calcular la palabra codificada se debe hacer:

$$F = (I_1, I_2, I_3, I_4) * G$$

Que es lo mismo que hacer:

$$F_1 = I_1 * 1 \oplus I_2 * 0 \oplus I_3 * 0 \oplus I_4 * 0 = I_1$$

$$F_2 = I_1 * 0 \oplus I_2 * 1 \oplus I_3 * 0 \oplus I_4 * 0 = I_2$$

$$F_3 = I_1 * 0 \oplus I_2 * 0 \oplus I_3 * 1 \oplus I_4 * 0 = I_3$$

$$F_4 = I_1 * 0 \oplus I_2 * 0 \oplus I_3 * 0 \oplus I_4 * 1 = I_4$$

$$F_5 = I_1 * 1 \oplus I_2 * 1 \oplus I_3 * 0 \oplus I_4 * 1 = I_1 \oplus I_2 \oplus I_4$$

$$F_6 = I_1 * 1 \oplus I_2 * 0 \oplus I_3 * 1 \oplus I_4 * 1 = I_1 \oplus I_3 \oplus I_4$$

$$F_7 = I_1 * 0 \oplus I_2 * 1 \oplus I_3 * 1 \oplus I_4 * 1 = I_2 \oplus I_3 \oplus I_4$$

Lo que hace que F resulte en $(I_1, I_2, I_3, I_4, CH_1, CH_2, CH_3)$. Notar que a diferencia de hamming los bits de control están al final y no entrelazados, lo que hace que se pierda la propiedad del código de Hamming donde el síndrome nos indica la posición del error. Pero todas las demás propiedades se mantienen, especialmente la distancia mínima, es decir que son códigos equivalentes.

Ahora, ¿que nos impide permutar las columnas de G para que este imite perfectamente el código de Hamming?, la respuesta es: nada realmente, simplemente $G = [I | P]$ es la forma generalizada, podríamos hacer que funcione de igual manera, aun que habría que tener cuidado al construir la matriz de chequeo de paridad H que se vera después, y además dificulta un poco la extracción de la información al tener que evitar los bits de control, mientras que de la forma generalizada simplemente sabemos que los bits de información son los primeros k bits y los demás se descartan.

La forma entrelazada facilita la corrección física de un error simple mediante la propiedad del síndrome (solo en el caso de hamming), pero la forma sistemática prioriza la velocidad de extracción de la información cuando no hay errores, el cual es el caso mas probable en su uso.

9.6.2. Matriz de chequeo de paridad.

En el extremo receptor, se necesita una forma rápida y eficiente de verificar si el mensaje recibido contiene errores. Para esto se utiliza la matriz chequeadora o de paridad (H). Esta matriz se deriva directamente de G y se conforma tomando la matriz de paridad transpuesta (P^T) y concatenándola con una nueva matriz identidad correspondiente al tamaño de los bits de control, otra vez vemos el ejemplo del código de Hamming:

$$H = [P^T | I] = \begin{pmatrix} 1 & 1 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 & 1 \end{pmatrix}$$

¿Cómo funciona? Se basa en la propiedad algebraica de que toda palabra válida del código es ortogonal a las filas de la matriz H (es decir $R * H^T = 0$ si R es una palabra valida). Por lo tanto, si el receptor multiplica el mensaje recibido (R) por la matriz transpuesta H^T , el resultado será el **síndrome** (S).

$$S = R * H^T$$

Que es lo mismo que (para el código de Hamming):

$$S_1 = R_1 * 1 \oplus R_2 * 1 \oplus R_3 * 0 \oplus R_4 * 1 \oplus R_5 * 1 \oplus R_6 * 0 \oplus R_7 * 0 =$$

$$R_1 \oplus R_2 \oplus R_4 \oplus R_5 = I_1 \oplus I_2 \oplus I_4 \oplus CH_1$$

$$S_2 = R_1 * 1 \oplus R_2 * 0 \oplus R_3 * 1 \oplus R_4 * 1 \oplus R_5 * 0 \oplus R_6 * 1 \oplus R_7 * 0 =$$

$$R_1 \oplus R_3 \oplus R_4 \oplus R_6 = I_1 \oplus I_3 \oplus I_4 \oplus CH_2$$

$$S_3 = R_1 * 0 \oplus R_2 * 1 \oplus R_3 * 1 \oplus R_4 * 1 \oplus R_5 * 0 \oplus R_6 * 0 \oplus R_7 * 1 =$$

$$R_2 \oplus R_3 \oplus R_4 \oplus R_7 = I_2 \oplus I_3 \oplus I_4 \oplus CH_3$$

- Si $S = (0,0,0)$ el mensaje llegó intacto, dado que todas las ecuaciones de paridad se cumplen.
- Si el ruido alteró algún bit, el vector síndrome arrojará un resultado distinto de ceros.

Síndrome y patrón de error

El síndrome es por definición matemática una **combinación lineal de las columnas de la matriz de chequeo de paridad H** .

La cantidad de columnas involucradas en esa combinación es la cantidad de errores, y además los índices de las columnas involucradas nos dan las ubicaciones de los errores.

- **Si ocurre 1 error:** El síndrome resultante es idéntico a la i -ésima columna de la matriz H . Lo cual nos indica que el error está en la posición i del mensaje.
- **Si ocurren 2 errores:** El síndrome es una suma modulo 2 (XOR) de las columnas i y j de la matriz H , lo cual nos indica que los errores están en la posición i y j .
- **Si ocurren ω errores:** El síndrome será la suma (combinación lineal) de las ω columnas correspondientes a las posiciones donde están los errores.

Obviamente hay que tener en cuenta de la capacidad de detección y corrección del código. Si ocurriera que la cantidad de errores supera la capacidad de detección, podría darse el caso que el síndrome da 0 a pesar de haber múltiples errores, aunque es muy poco probable, y si el síndrome no da 0, o se supera la capacidad de corrección, el síndrome simplemente es inútil pues no marca los errores reales.

Un **patrón o vector de error**, es la diferencia algebraica entre el vector o secuencia recibida por el receptor (R) y la palabra código enviada originalmente (V), es una máscara de los bits modificados:

$$e = R \oplus V$$

Por ej., si la palabra enviada fue 0110110 y la recibida fue 0010111, el patrón de error es: 0100001, donde los unos indican los bits modificados.

El “peso” de un síndrome, es la cantidad de 1 que tiene, es decir, la cantidad de errores que señala.

Los síndromes son los que nos ayudan a construir los patrones de error para corregir la palabra. Sin embargo, existen más patrones de error posibles (2^n) que síndromes (2^{n-k}), por eso para poder distinguir y corregir errores, necesitamos asociar cada patrón a un síndrome único, por lo tanto el número de patrones de error de peso $\leq t$ (capacidad de corrección) no puede superar el número total de síndromes disponibles. Esto nos da la célebre cota de Hamming:

$$\sum_{w=0}^t \binom{n}{w} \leq 2^{n-k}$$

9.6.3. Capacidad de corrección de un código lineal.

La cota de singleton establece que $d_{min} \leq n - k + 1$, si reemplazamos por la d_{min} por la capacidad de corrección de errores ($d_{min} \geq 2t + 1$) tenemos que $t \leq \frac{n-k}{2}$, es decir que la cantidad de errores que se pueden corregir es la mitad o menor a la cantidad de bits de paridad/control (en realidad es la mitad truncada)

En los códigos lineales, la **distancia mínima del código es siempre igual al peso mínimo (la cantidad de "unos") de cualquier palabra código no nula.**

Es decir, con buscar la palabra válida con menos unos (ignorando la que es puros ceros), encontramos la distancia mínima, y con la distancia mínima calculamos la capacidad de corrección del código: $t = \text{floor}\left(\frac{d_{\min}-1}{2}\right)$

En criollo:

La palabra nula (todos ceros) funciona como el "ancla" o el punto de origen del sistema. Como estamos trabajando con códigos lineales, el "mapa" de las palabras válidas es matemáticamente perfecto y simétrico. Esto significa que si te paras en cualquier palabra válida, las distancias hacia el resto de las palabras se ven exactamente igual que si te pararas en la palabra nula.

Entonces, en lugar de medir la distancia cruzada entre todas las combinaciones posibles de palabras para ver cuáles están más juntas (lo cual sería un trabajo enorme), aprovechamos esta simetría matemática: tomamos la palabra nula (se podría tomar cualquier palabra, pero se elige la nula por simplicidad) y simplemente buscamos a su vecina más cercana.

Como la palabra nula es puros ceros, su vecina más próxima será, lógicamente, la palabra válida que tenga menos "unos" (el peso mínimo). Gracias a la perfección de este mapa, esta distancia mínima calculada es exactamente la misma separación mínima que existe entre cualquier otro par de palabras vecinas en todo el sistema.

9.7. Códigos binarios cíclicos.

Son un subgrupo de códigos lineales, su definición formal se basa en una propiedad geométrica y algebraica muy sencilla: **un código es cíclico si, y solo si, cualquier desplazamiento cíclico (o rotación) de una palabra código válida da como resultado otra palabra código válida que también pertenece al mismo código.**

Es decir, si la secuencia de bits $c = (c_0, c_1, \dots, c_{n-1})$ es una palabra válida, entonces $(c_{n-1}, c_0, \dots, c_{n-2})$ también lo es. (Notar que esto aplica en ambos sentidos)

Nota: agregar a partir de aquí que trabajan con la aritmética de un campo de Galois $GF(2^m)$

Representación polinómica: Para trabajar analíticamente con los códigos cíclicos, se sustituye el uso clásico de vectores y matrices por **polinomios**. En esta notación, los bits de una secuencia se convierten en los coeficientes de un polinomio. Por ejemplo, una cadena de información (1,0,1,1) se representa algebraicamente como el polinomio $x^3 + x + 1$.

Nota: existe otra notación donde los coeficientes se consideran al revés, tipo (1,0,1,1) sería $x^3 + x^2 + 1$, si revisan los libros, algunos usan esta otra notación.

Matemáticamente, efectuar el desplazamiento cíclico de una palabra equivale a multiplicarla por x y tomar el resto de la división módulo $(x^n + 1)$ (esta división lo que hace es revisar si x^n existe y lo reemplaza por el término independiente 1, para que siga la rotación de bits como se mostró antes).

Ej: (1,1,0,1) rotado es (1,0,1,1), (la matemática lo rota hacia la izquierda porque si nos acordamos, a la izquierda están los términos de mayor grado, y como multiplicamos por x estamos agregando un grado a cada termino) esto en polinomios es:

$$(x^3 + x^2 + 1) * x = x^4 + x^3 + x \text{ y luego quitandamos } x^4 \text{ y agregamos 1: } x^3 + x + 1$$

El polinomio generador: Todo código cíclico de longitud n y dimensión k posee un único polinomio no nulo de grado mínimo que actúa como la base del código, denominado **polinomio generador**.

Este polinomio tiene la forma $g(X) = g_0 + g_1 * x + g_2 * x^2 + g_3 * x^3 + \dots + g_{n-k} * x^{n-k}$

Este polinomio cumple estrictamente con las siguientes propiedades:

- Es divisor o factor exacto de $x^n + 1$, es decir: $(x^n + 1) \% g(x) = 0$
- Su grado es exactamente $n - k$
- Debe tener coeficiente unitario (un "1") tanto en el término de mayor grado como en el término independiente (es decir $g_0 = 1$ y $g_{n-k} = 1$ siempre, pero en la notación esta de forma genérica para que se entienda)
- Toda palabra valida p del código es un múltiplo exacto del polinomio generador $g(x)$, es decir $p(x) \% g(x) = 0$

Ventajas principales: Los códigos cíclicos están fuertemente orientados hacia la implementación por hardware. Se destacan porque se pueden describir de manera muy compacta (el polinomio generador en vez de una tabla o matriz), son sumamente fáciles de almacenar y sus procesos de codificación y decodificación son simples de operar (electrónicamente).

¿Como se encuentra/construye un polinomio generador?

Se pueden hacer polinomios generadores mediante prueba y error, pero esto es viable solamente para códigos simples. Para hacer códigos complejos se usa magia negra, llamada campos de Galois que se van a ver después.

Un ejemplo del polinomio generador del código de Hamming (7,4) seria: $g(x) = x^3 + x + 1$

9.7.1. ¿Cómo se codifica el mensaje?

Existen dos metodologías matemáticas para generar la palabra código $P(x)$ a partir de la información $I(X)$ y el generador $g(X)$:

- 1. Forma no sistemática:** Es la más simple algebraicamente. Se obtiene **multiplicando directamente el polinomio de información por el polinomio generador:**

$$P(x) = I(x) * g(x)$$

Al ser el resultado una multiplicación directa, nos aseguramos de que la palabra sea un múltiplo de $g(X)$ y por tanto, válida. Aunque es rápido, su gran desventaja es que los bits de información original terminan mezclados, y no se distinguen a simple vista. Lo que dificulta mucho el trabajo al receptor.

Y para recuperar la información hay que dividir $P(x)$ en $g(x)$, donde el resto de la división será el síndrome.

Ejemplo en el código de hamming de la información $x^2 + 1$ (0101):

- Codificación:

$$P(x) = (x^2 + 1) * (x^3 + x + 1) = x^5 + x^2 + x + 1 \text{ (0100111)}$$

Si miramos, el mensaje original (0101), no se encuentra a simple vista en el resultado (0100111)

- Decodificación:

$$I(x) = \frac{P(x)}{g(x)} = x^2 + 1 \text{ (resto 0)}$$

Si el resto es 0 indica que no hubo error, si el resto no es 0, significa que hubo error,
CONTINUAR

2. Forma sistemática: Es la preferida en la práctica porque garantiza que los K bits de información se ubiquen intactos al principio del bloque, seguidos por los bits $N - K$ chequeadores al final. El procedimiento matemático es:

$$r(x) = (x^{n-k} * I(x)) \% g(x)$$

$$P(X) = x^{n-k} * I(x) + r(X)$$

Notar: $x^{n-k} * I(x)$ es simplemente la palabra de información desplazada $n - k$ posiciones, ejemplo: 0101 pasaría a ser 0101000, y luego $r(x)$ es el calculo de los bits de chequeo.

Ejemplo: $r(x) = (x^5 + x^3) \% (x^3 + x + 1) = x^2$ (100), luego $P(x) = x^5 + x^3 + x^2$ (0101100)

Y la decodificación en este caso constaría en agarrar los primeros k bits de la palabra para obtener la información, y los demás $n - k$ bits son los de chequeo

Para obtener el síndrome se debe hacer la división $\frac{P(x)}{g(x)}$ y observar el resto (el cociente aquí es “basura” algebraica, no tiene ningún valor y se descarta), si el resto es 0 quiere decir que no hubo error, si el resto no es 0 quiere decir que hubo error

Si hay error, igual que códigos lineales se compara con las columnas de la matriz H , ¿cómo se obtiene la matriz H ? Se obtiene el polinomio de chequeo $h(x) = \frac{x^{n+1}}{g(x)}$, Y luego la matriz se forma colocando los coeficientes de $h(x)$ en orden inverso y luego realizando desplazamientos cíclicos, también se pueden la i -ésima fila haciendo $\frac{x^{n-k+i}}{g(x)}$ donde i varía desde 0 hasta $k - 1$

9.7.2. Codificación con registro de desplazamiento de $(n-k)$, y de k estados.

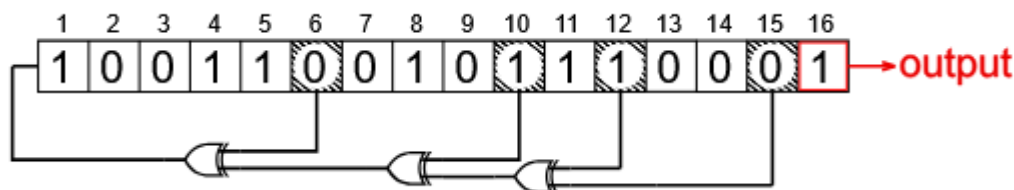
Como ya habíamos dicho, la gran ventaja de los códigos cíclicos sistemáticos es su extrema facilidad de implementación mediante circuitos electrónicos. Estos circuitos operan utilizando Registros de Desplazamiento con Retroalimentación Lineal (LFSR, por sus siglas en inglés) y compuertas lógicas.

Dependiendo del diseño y de los requerimientos de capacidad de corrección, existen dos arquitecturas principales para el codificador en el emisor: usando $n - k$ estados o usando k estados.

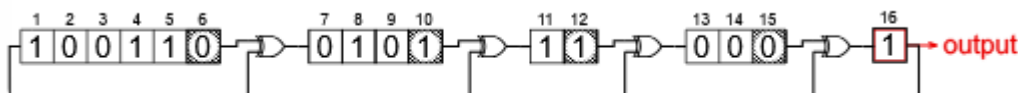
- La forma exacta en la que se conectan los cables de "realimentación" hacia las compuertas XOR está dictada directamente por los coeficientes "1" y "0" presentes en el polinomio generador $g(X)$
- Al inyectar los bits de información en este circuito, el mismo va calculando el resto de la división polinómica "al vuelo" pulso a pulso de reloj, obteniendo los bits chequeadores de forma instantánea.

Aquí hay que aclarar, hay 2 tipos de LSFR, en criptografía se usan los de Fibonacci, pero en control de error se usan los de Galois. Si se quieren usar los de Fibonacci hay que usar el polinomio inverso al polinomio generador.

Ejemplo de un LSFR de Fibonacci:



Ejemplo de uno de Galois:



Esto es algo que el profesor no aclara, y da lugar a mucha confusión, y si uno se guía por los gráficos en las presentaciones estos señalan al de Fibonacci, pero como ya dijimos, esto no funcionaria.

Aquí hay una pagina que explica las diferencias y además tiene una calculadora, la recomiendo:

<https://www.omnicalculator.com/math/linear-feedback-shift-register>

Pero en base a esos ejemplos, lo que uno tiene que pensar es que los bits se van moviendo a la derecha, y si los bloques están contiguos los bits se pasan como están, en cambio si hay una compuerta hay que hacer el cálculo.

Si nos vamos al ejemplo del código de hamming, con el polinomio $g(x) = x^3 + x + 1$ y hacemos el LSFR de k registros, usamos la "seed" o estado inicial 1000 (no puede ser 0000 o no cedería nada)

HACER

9.9. Introducción códigos BCH

Los **códigos BCH** son una de las familias más importantes y versátiles de **códigos de bloque cíclicos**, a diferencia de los códigos de Hamming pueden diseñarse para corregir cualquier combinación de t o más errores de forma óptima (Hamming se limita a 1 error).

Introdujeron el concepto de distancia de diseño, que permite asegurar matemáticamente la distancia mínima del código diseñado, lo que los hizo muy útiles para diseñar códigos capaces de soportar ráfagas de errores.

9.10. Códigos REED-SOLOMON.

Los códigos Reed-Solomon (RS) son un subconjunto de los códigos BCH.

A diferencia de los códigos clásicos que operan sobre bits individuales de ceros y unos, los códigos Reed-Solomon **operan sobre bloques de bits agrupados en "símbolos"**. Lo que los hace mucho mas eficientes y potentes contra errores en ráfagas, pues no importa cuantos bits de un simbolo sean modificados, se considera simplemente como 1 simbolo con error.

Parámetros del Código $RS(n, k)$:

- n (Longitud total): Es el número de símbolos a la salida del codificador. Por definición matemática en el campo de Galois, $n = q - 1$ (cantidad de elementos en el campo de Galois), donde $q = 2^s$ es el tamaño del campo. Por ejemplo, en $GF(2^8)$, la longitud del bloque suele ser de $2^8 - 1 = 255$ símbolos (bytes).
- k (Símbolos de información): Es la cantidad de símbolos de datos originales que se van a proteger.
- $2t = n - k$ (Símbolos de paridad): Es la redundancia agregada al final de los datos (forma sistemática).
- t (Capacidad de corrección): El decodificador puede corregir automáticamente hasta t símbolos defectuosos en la palabra de código.

Ejemplo clásico: El código **RS (255, 223)** con símbolos de 8 bits (usado ampliamente en televisión). Toma 223 bytes de datos, le añade 32 bytes de paridad ($2t = 32$), formando un bloque de 255 bytes. Este código puede corregir hasta $t = 16$ **bytes erróneos** en cualquier lugar del bloque.

Propiedad MDS (Maximum Distance Separable)

Los códigos RS son óptimos desde el punto de vista matemático porque son códigos de **Máxima Distancia Separable (MDS)**. Esto significa que alcanzan el límite teórico (Cota de Singleton), logrando que la distancia mínima entre dos palabras válidas sea la máxima posible: $d_{min} = n - k + 1$. Al tener la máxima distancia posible entre códigos, se maximiza la capacidad de detectar y corregir errores sin desperdiciar redundancia.

Su mayor poder: Los errores en ráfaga (Burst Errors)

La razón por la que operan con símbolos (bytes) y no con bits, es para hacerlos extremadamente robustos contra **errores en ráfaga** (cuando el ruido daña muchos bits consecutivos, como un rayón en un CD o un desvanecimiento temporal en la señal de radio o television).

Si el canal corrompe 8 bits consecutivos, en un código corrector binario esto representaría 8 errores catastróficos distintos. Sin embargo, para un código Reed-Solomon que agrupa de a 8 bits, esos 8 bits consecutivos dañados caen dentro de un mismo símbolo (o a lo sumo en dos símbolos adyacentes). **Al decodificador RS no le importa si un símbolo tiene 1 bit volteado o sus 8 bits destruidos; el esfuerzo y "costo" matemático para corregirlo es el mismo: cuenta como solo "1 símbolo erróneo".**

Fundamentos Matemáticos: Generación y Codificación

Para comprender la matemática detrás de los códigos Reed-Solomon, el proceso se debe dividir en dos grandes etapas: la construcción del "universo" numérico donde operan los símbolos (el Campo de Galois) y la definición del polinomio que se encargará de generar la palabra código.

1. El Campo de Galois y el Polinomio Irreducible Para obtener un código RS, primero hay que encontrar un polinomio de grado tal que no sea divisible por ningún otro polinomio de grado menor. A este polinomio se lo denomina **polinomio irreducible**. Es vital destacar que **este polinomio todavía no es un generador $g(x)$ del código RS**. Su única función es generar todos los elementos del campo de Galois $GF(q)$. Estos elementos del campo serán el 0, el 1 y además se contará con un **elemento primitivo α** . Las sucesivas potencias de este elemento α ($\alpha^1, \alpha^2, \alpha^3, \dots, \alpha^{q-1}$) generarán todos los símbolos distintos del campo hasta que comiencen a repetirse al llegar al valor de $\alpha^{q-1} = 1$.

2. Obtención del Polinomio Generador Reed-Solomon ($g_{rs}(x)$)

Una vez definido el Campo de Galois, se procede a construir el polinomio que añadirá la redundancia al mensaje. En general, un código RS de longitud n y dimensión k es un código cíclico generado por el siguiente polinomio matemático:

$$g_{rs}(x) = (x - \alpha)(x - \alpha^2) \dots (x - \alpha^{n-k})$$

Donde α es el elemento primitivo del campo de Galois.

Ejemplo práctico: Para un código RS(7, 3, 5) operando sobre $GF(8)$, tenemos $n - k = 4$ chequeadores (símbolos de paridad), y su polinomio sería:

$$g_{rs}(x) = (x - \alpha)(x - \alpha^2)(x - \alpha^3)(x - \alpha^4) = \alpha^3 + \alpha x + x^2 + \alpha^3 x^3 + x^4$$

9.10.1. Codificación sistemática

El procedimiento de codificación es el mismo que en códigos cíclicos en general (recordar que los códigos RS son una subclase de los BCH que son una subclase de los cíclicos):

$$r(x) = (x^{n-k} * I(x)) \% g(x)$$

$$P(X) = x^{n-k} * I(x) + r(X)$$

(multiplicar la información por x^{n-k} para desplazarla, dividirla por el polinomio generador y quedarnos con el resto como redundancia)

Sin embargo la forma de hacer los cálculos es distinto, dado que se operan bajo campos de Galois extendidos $GF(2^m)$,

En un código cíclico binario clásico (como Hamming o un BCH estándar), la división polinómica se hace bit a bit operando con aritmética sencilla en el campo base $GF(2)$.

En Reed-Solomon, esa mismísima división sintética se realiza procesando **bloques o símbolos enteros** (típicamente bytes) bajo las reglas aritméticas especiales del campo de galois de extensión $GF(2^m)$

9.11.2. Decodificación y síndrome.

Este es un proceso bastante más complejo que en códigos cíclicos normales.

COMPLETAR

9.12. Introducción a los códigos convolucionales.

A diferencia de los códigos de bloques tradicionales que segmentan la información, son esquemas de control de errores sin bloques que operan sobre secuencias o flujos continuos de datos binarios. El proceso de codificación incorpora **memoria**; esto significa que los bits de salida en un instante temporal específico están determinados no solo por los bits de información actuales, sino también de una cantidad fija de bits anteriores almacenados en un registro de desplazamiento.

9.13. Entrelazamiento o intercalado

La mayoría de los códigos de bloque (como los códigos Hamming) están diseñados y dimensionados para corregir errores **aleatorios e independientes** (aislados). Si un bloque de datos recibe una ráfaga que altera, por ejemplo, 4 bits seguidos, superará inmediatamente la capacidad de corrección del código (que suele ser de 1 o 2 errores), haciendo que el mensaje sea incorregible.

La idea del entrelazamiento no es cambiar el código corrector en sí, sino **desordenar temporalmente la secuencia de transmisión** para engañar al canal ruidoso. El proceso consta de los siguientes pasos:

1. **Codificación 1 (Código externo):** El emisor toma los datos originales y los codifica en bloques utilizando un código estándar (por ejemplo, Hamming o Reed-Solomon). Añade *bits de control 1*
2. **Entrelazamiento (Emisor):** La secuencia unificada resultante del paso anterior (datos + redundancia externa) ingresa al **intercalador/entrelazador** que altera de forma sistemática el orden físico de los bits o símbolos antes de enviarlos al canal, de modo que los bits que originalmente eran vecinos queden ampliamente separados entre sí.

El intercalador más simple es reordenar los bits a enviar en una matriz, donde generalmente se enviarían fila por fila, y enviar columna por columna. Ej.: La secuencia 123 456 789 se envía como 147 258 369.

3. **Codificación 2 (código interno):** La secuencia reordenada y mezclada ingresa al segundo codificador (generalmente un código convolucional o Hamming, diseñados contra errores independientes). Añade *bits de control 2*
4. **Transmisión por el canal con ruido:** Durante la transmisión por el canal, ocurre una perturbación que destruye un grupo de **bits consecutivos**.

5. **Primera decodificación (Decodificador interno):** Este decodificador utiliza los *bits de control* 2 de la secuencia para corregir errores sencillos. Sin embargo, la ráfaga de errores supera su capacidad, y quedan bloques de errores.
6. **Desentrelazamiento (Receptor):** Del paso anterior recibe la secuencia reordenada de *bits de información con errores + bits de control* 1, el receptor pasa los datos por un **desintercalador**, el cual realiza el proceso inverso y restaura el orden original de los bits. Al devolver cada bit a su posición original, la ráfaga de errores consecutivos se **dispersa**. Ahora, lo que era una ráfaga destructora concentrada en un único bloque se convierte en **pequeños errores aislados (de 1 bit)** distribuidos a lo largo de muchos bloques diferentes.
7. **Decodificación:** Al estar los errores ampliamente separados, cada bloque individual ahora contiene un número de errores que entra cómodamente dentro de la capacidad de corrección de los decodificadores, logrando recuperar la información de forma impecable.

Esta es llamada la **concatenación en serie**, donde los bits de control de ambos códigos y la información viajan mezclados entre ellos.

Pero existe la **concatenación en paralelo**, donde el intercalador solo reordena los bits de información, el segundo codificador jamás “ve” ni codifica los bits de paridad del primero, y la secuencia final es: bits de información originales + bits de redundancia 1 + bits de redundancia 2, lo cual da una secuencia mucho más limpia.

Aquí uno podría pensar que transmitir los bits originales sería mala idea, pero no es así:

- Los bits de redundancia 1, fueron hechos sobre la secuencia original, así que son susceptibles a los errores en ráfaga, estos solo sirven para arreglar errores simples.
- Los bits de redundancia 2, fueron hechos sobre la secuencia reordenada, en caso de que hubiera una ráfaga, estos serán los que sirven para arreglar los errores.

El receptor cuando recibe la secuencia, envía los bits de información originales al decodificador 1, para verificar/corregir errores simples. Al mismo tiempo entrelaza los bits originales antes de enviarlos al decodificador 2, lo cual hace que, si hubo errores en ráfaga, se dispersen, y el decodificador 2 soluciona múltiples errores simples, luego la secuencia corregida es desentrelazada para obtener la información original.

El problema aquí es: que pasa si las secuencias de cada decodificador son distintas, ¿con cuál nos quedamos?, la respuesta es: no es tan sencillo, en su lugar, los dos decodificadores **cooperan y se ayudan mutuamente para llegar a un consenso**, intercambiando información sobre probabilidades sobre cuál era el bit original, refinando de forma iterativa el resultado hasta converger en una palabra correcta.

9.13. Introducción a los códigos turbo. Límite de Shannon

Son un esquema avanzado de corrección de errores que se estructura mediante la **concatenación en paralelo** de dos o más codificadores convolucionales de baja complejidad acoplados a través de un **entrelazador pseudoaleatorio**.

Nota: un entrelazador pseudoaleatorio simplemente la estructura de mezcla no sigue un patrón geométrico simple como el que vimos, este genera una permutación una vez, y queda fija en la implementación del código.

Claude Shannon demostró teóricamente la capacidad máxima de un canal (el límite superior de velocidad a la que se puede transmitir información sin errores ante un nivel de ruido dado). Durante décadas, ningún código práctico lograba acercarse a este límite.

Shannon demostró que los mejores códigos para acercarse al límite del canal eran los códigos aleatorios (las palabras de código se eligen de forma **completamente aleatoria e independiente**), pero estos requerían tablas de búsqueda exponenciales y recursos inviables de implementar.

Los códigos Turbo, al combinar la concatenación con el entrelazado aleatorio y la decodificación iterativa por etapas, logran emular ese comportamiento óptimo "aleatorio" y operar muy cerca del límite de Shannon con una **complejidad de hardware perfectamente viable**.

UNIDAD Nº10 - RECONOCIMIENTO DE PATRONES, TRATAMIENTO DE IMÁGENES

10.1. Análisis digital y reconocimiento de patrones

10.1.1. Introducción

El **análisis digital de imágenes (ADI) y reconocimiento de patrones** se enfoca en la descripción, identificación de estructuras y clasificación del contenido presente en una imagen digital para categorizar los objetos detectados.

10.1.2. Ejemplos.

10.1.3. Proceso de reconocimiento. Clasificación.

10.2. Tratamiento digital de imágenes

10-2-1 Introducción:

Procesamiento Digital de Imágenes (PDI): Se ocupa de la transformación de una imagen a formato digital y de su posterior procesamiento mediante una computadora para mejorar su calidad visual, restaurarla o modificar sus características.

10-2-1-1 Tópicos tratados por el procesamiento digital de imagen.

10-2-1-2 Trabajos que necesitan edición de gráficos (aplicación).

10-2-2 Adquisición de imágenes:

La **adquisición de imágenes** es la primera etapa fundamental en el tratamiento digital de imágenes. Su objetivo es capturar la luz de una escena física continua y convertirla en una matriz de datos digitales numéricos que una computadora pueda procesar.

El proceso físico de adquisición se lleva a cabo mediante una cadena de bloques compuesta por tres elementos principales:

1. **Sistema Óptico:** Capta la señal óptica (los fotones de luz que provienen de la escena) y la enfoca.
2. **Sensor:** Transforma la señal óptica capturada en una señal eléctrica de naturaleza analógica.
3. **Digitalizador (Conversor A/D):** Transforma la señal eléctrica analógica continua en un flujo de datos digitales discretos.

Nota: Durante este proceso físico de conversión se producen inevitablemente pequeñas imperfecciones o **degradaciones** en la señal.

Una imagen continua se define como una función bidimensional de intensidad luminosa $f(x, y)$, donde x , y representan las coordenadas espaciales en el plano 2D. Puesto que la luz es una forma de energía finita y positiva, se cumple que $0 < f(x, y) < \infty$

Generalmente, la luz que captamos es reflejada por los objetos de la escena. El valor de brillo $f(x, y)$ se determina mediante el producto de dos componentes:

$$f(x, y) = i(x, y) * r(x, y)$$

- Iluminación $i(x, y)$: Es la cantidad de luz procedente de la fuente de energía que incide sobre la escena ($0 < i(x, y) < \infty$)
- Reflectancia $r(x, y)$: Es la proporción de luz que reflejan las superficies de los objetos presentes en la escena ($0 < r(x, y) < 1$)

10-2-2-1 Sistemas, representación, formación de la imagen digital.

Dado que la computadora opera de forma discreta, la función continua $f(x, y)$ debe convertirse en una **imagen digital** mediante dos pasos de discretización:

1. **Discretización espacial (Muestreo):** Subdivide el plano continuo (x, y) en una cuadrícula o matriz regular de $M \times N$ puntos discretos llamados **píxeles** (*picture elements*).
2. **Discretización en amplitud (Cuantización):** Mide la intensidad luminosa en cada punto y le asigna un valor numérico entero discreto proporcional a su brillo o nivel de gris. En imágenes monocromáticas se guarda un entero; en imágenes a color (como RGB) se guarda un vector de tres valores.

Matemáticamente, la imagen digitalizada resultante es una función o aplicación discreta de la forma:

$$f: \{0, \dots, M - 1\} \times \{0, \dots, N - 1\} \rightarrow \{0, \dots, G - 1\}$$

Donde $M \times N$ son las dimensiones de la imagen en píxeles y G es el número total de niveles discretos de intensidad disponibles (por ejemplo, $G = 256$ para cuantización de 8 bits por píxel)

10-2-3 Operaciones elementales.

Las operaciones sobre imágenes digitales se dividen conceptualmente en **operaciones individuales o puntuales** (donde la transformación de un píxel depende únicamente de su valor actual) y **operaciones grupales o espaciales** (donde la transformación depende del entorno o vecindario del píxel).

Operaciones puntuales e individuales

Generan una nueva imagen de salida q aplicando una función de transformación uno a uno sobre el valor del píxel $p(x, y)$ de la imagen de entrada p , sin considerar a los píxeles vecinos: $q(x, y) = f(p(x, y))$, muchas veces la transformación f toma información de otra imagen que denominamos h

- **Operaciones aritméticas:**
 - **Adición** ($q = p + h$): Se utiliza para la **reducción de ruido** en la imagen mediante el promediado de múltiples capturas.
 - **Sustracción** ($q = p - h$): Permite **eliminar la información de fondo** constante o resaltar diferencias temporales entre dos imágenes (por ejemplo, en medicina se resta una radiografía normal de una con contraste para dejar únicamente los vasos sanguíneos)
 - **Multiplicación y división:** Corrige variaciones indeseadas por **iluminación no uniforme** o respuesta desigual del sensor. Dividir por un valor menor a 1 ilumina la imagen, mientras que dividir por un valor mayor a 1 la oscurece. La multiplicación permite superponer estructuras (como imágenes de Rayos X sobre fotos originales).
- **Operaciones Lógicas:** Se aplican bit a bit entre imágenes o máscaras binarias: **AND**, **OR**, **XOR** y **NOT**. Se emplean para extraer (recortar) o enmascarar (tapar) objetos específicos dentro de una escena.
- **Transformaciones directas:**
 - **Identidad:** Genera una copia exacta de la imagen ($q = p$).
 - **Negativo:** Invierte la escala de tonos ($q = 255 - p$ para imágenes de 8 bits).
 - **Umbralización (Binarización):** Convierte una imagen de niveles de gris en una imagen binaria (valores de 0 o 255) en función de un valor de umbral de transición u :
$$q = \begin{cases} 0 & \text{si } p \leq u \\ 255 & \text{si } p > u \end{cases}$$
 - **Traslación:** Operaciones geométricas que desplazan la posición (i, j) de un píxel a $(i + i_d, j + j_d)$.

- **Rotación:** Aplica un giro de ángulo ϕ mediante las ecuaciones de transformación trigonométrica.

Operaciones grupales

Elementos: Ventanas, Máscaras y Convolución

- **Ventana:** Subconjunto o pequeña porción de píxeles de la imagen.
- **Máscara:** Matriz de coeficientes o pesos que asigna una importancia numérica a cada punto de la ventana.
- **Convolución espacial:** Cualquier calculo que se base en una combinación lineal entre los píxeles de una ventana y los pesos fijados en la máscara, generalmente la combinación es una multiplicación del pixel por el peso de la matriz, en caso de querer hacer restas se usan pesos negativos.

Filtrado espacial

El filtrado espacial se aplica típicamente para la eliminación del ruido o para realizar algún tipo de mejora de la imagen. Operan en los datos de imagen considerando pequeños vecindarios en una imagen, como 3x3, 5x5, y devuelven un resultado moviéndose secuencialmente por toda la imagen.

Existen distintos tipos de filtros espaciales: paso bajo, paso alto y paso banda.

Métodos de suavizado y eliminación de ruido (filtros paso bajo)

Los filtros paso bajo atenúan o eliminan las componentes de alta frecuencia espacial, dejando inalteradas las frecuencias bajas. Como las altas frecuencias caracterizan a los bordes y detalles muy marcados, el efecto visual primario es emborronar o suavizar la imagen.

- **Filtro de la media:** Sustituye el valor del píxel central por el promedio numérico de los valores en una ventana $N \times N$ (usando una mascara con coeficientes $\frac{1}{N^2}$). A mayor tamaño de máscara, mayor mitigación de ruido a costa de un mayor desenfoque.
- **Filtro Gaussiano:** Realiza una convolución basada en la distribución gaussiana (recordar la campana de gauss de la distribución normal en estadística), valorando la contribución de cada píxel de la ventana según su distancia al centro (mientras mas lejano el pixel, menos peso). Suaviza la imagen emborronando menos que el filtro de la media.
- **Filtro de la mediana:** Algoritmo no lineal que ordena de menor a mayor los niveles de gris de los píxeles de la ventana y reemplaza el valor central por la **mediana** estadística. Es el método más eficaz para eliminar el **ruido impulsional** ("sal y pimienta" o "salpicaduras").

Métodos de extracción de bordes y afilado (filtros paso alto y gradiente)

Los métodos de paso alto o realzado buscan destacar detalles finos, contornos y discontinuidades (altas frecuencias), es decir, buscan mejorar la imagen.

- **Paso alto básico:** Emplea máscaras con coeficientes positivos en el centro y negativos en la periferia. La suma de todos los coeficientes de la máscara es igual a **cero**, para que en regiones de intensidad constante o suave la respuesta sea nula. Mientras que sobre bordes

genera picos de intensidad, lo que realzaría los píxeles en los bordes, haciendo mas fácil distinguirlos.

- **Filtro high-boost:** Utiliza una máscara con una constante en el coeficiente central (como 0) para enfatizar las componentes de alta frecuencia manteniendo parte de las bajas frecuencias, evitando que la imagen pierda su fondo original.
- **Filtros paso banda:** Los filtros de paso de banda se especifican mediante dos frecuencias de corte, un corte bajo y un corte alto. Eliminando las frecuencias que están fuera del rango, estos se utilizan principalmente para restaurar imágenes. Por ej, en una imagen dañada por ruido, si es posible aislar las frecuencias del ruido, estas se pueden eliminar siempre y cuando no dañe mas la imagen, y aun que lo hiciera después se empareja con otras tecnicas.
- **Filtrado homomórfico:** Es una técnica de mejorado bastante útil en los casos en los que la imagen fuente es perturbada por ruido multiplicativo, interferencias o iluminación poco uniforme. El ruido multiplicativo se caracteriza porque las partes de la imagen que son mas claras se ven mas afectadas por el ruido que las mas oscuras. Las técnicas de eliminación de ruido anteriores no pueden ser utilizadas porque están diseñadas para limpiar ruido aditivo (sumado a la imagen). ¿¿¿Esto se modela como: la imagen original $Img(x, y)$ es el producto de una imagen sin perturbación y una matriz de inteferencia $Img(x, y) = A(x, y) * I(x, y)$, aplicando logaritmos podemos modelarlo como ruido aditivo denuevo.???

Mejoras de Contraste (Operaciones de Histograma)

El **histograma** representa la cantidad de píxeles que poseen cada nivel de gris.

- **Estiramiento (Histogram Stretching):** Expande la distribución del histograma a todo el rango dinámico , incrementando el contraste de la imagen.
- **Contracción (Histogram Shrinking):** Comprime los niveles de gris a un rango más estrecho, disminuyendo el contraste.
- **Desplazamiento (Histogram Slide):** Suma o resta un valor constante (*offset*) a todos los píxeles para aclarar u oscurecer la imagen manteniendo las proporciones de gris.
- **Ecuálización del Histograma:** Transforma la imagen para obtener un **histograma con una distribución uniforme de píxeles** en todos los niveles de gris. Se ejecuta en 4 pasos:
 1. Calcular la suma acumulada de los valores del histograma.
 2. Normalizar dividiendo por el total de píxeles.
 3. Multiplicar por el nivel de gris máximo y redondear al entero más cercano.
 4. Mapear los niveles de gris originales a los nuevos valores.

10-2-3-2 Método de realzado de imágenes, detección de bordes mediante Laplaciano.

10-2-3-3 Imágenes color: método RGB, color indexado, sistemas de representación del color.

Representación de imágenes en color: Cada píxel en una imagen a color se define mediante un **vector de tres componentes**.

- **Método RGB:** Define cada píxel combinando tres canales de colores primarios aditivos: **Rojo (Red), Verde (Green) y Azul (Blue)**. En una representación típica de 24 bits (llamada *True Color*), cada canal (color) utiliza 8 bits (valores de 0 a 255), lo que permite representar 2^{24} ($\approx$ 16.7 millones) colores
- **Color indexado (paleta de colores):** En lugar de almacenar los tres valores RGB completos para cada píxel, la imagen almacena una **tabla o paleta de colores** y la matriz de la imagen guarda únicamente un **índice entero por píxel** que apunta a la posición correspondiente de la paleta, reduciendo significativamente el tamaño del archivo. Es el mecanismo utilizado en formatos gráficos como **GIF, PCX y BMP de 8 bits**.

Otros sistemas de representación del color: Espacios de Luminancia/Crominancia (YC_bC_r o YIQ)

Separan la componente de **Luminancia** (brillo o intensidad de la luz: Y) de las componentes de **Crominancia** (información de color: C_b, C_r o I, Q)

Fundamento: El ojo humano posee muchos más receptores para percibir el brillo (bastones) que para el color (conos). Por ello, el sistema visual es sensible a pequeños cambios de luminancia, pero tolera pérdidas sustanciales en la crominancia.

Aplicación en compresión: Formatos como **JPEG y MPEG** convierten las imágenes RGB a YC_bC_r para realizar **submuestreo de crominancia**, reduciendo la cantidad de datos sin una degradación visual perceptible.

Extra

Simbolos generales

- S alfabeto de una fuente
- s_i símbolo de una fuente
- q numero de símbolos de una fuente
- S^n extensión de orden n del alfabeto de una fuente
- a^i símbolo de la extensión de orden n del alfabeto de una fuente
- $\bar{S}$ afín de la fuente S
- P_i probabilidad del símbolo s_i de una fuente

- X alfabeto código
- x_i símbolo de un código
- r numero de símbolos de un código (también, numero de símbolos de entrada en un canal)
- X_i palabra de un código (secuencia de x_i 's) correspondiente a s_i
- l_i numero de símbolos de la palabra código correspondiente a s_i
- λ^i numero de símbolos de la palabra código correspondiente a a^i
- L longitud media de una palabra de un código del alfabeto S
- L_n longitud media de la apalabra de un código del alfabeto S^n

A alfabeto de entrada de un canal
 a_i símbolo de entrada en un canal
 r numero de símbolos de entrada de un canal (también, numero de símbolos de un código)
 A^n extensión de orden n del alfabeto de entrada de un canal
 α_i símbolo de la extensión de orden n del alfabeto de entrada de un canal

B alfabeto de salida de un canal
 b_j símbolo de salida en un canal
 s numero de símbolos de salida de un canal
 B^n extensión de orden n del alfabeto de salida de un canal
 β_j símbolo de la extensión de orden n del alfabeto de salida de un canal

m orden de una fuente de markov
 P_{ij} elemento de la matriz de un canal; probabilidad de recibir b_j al enviar a_i
 p probabilidad de error en un BSC ($\bar{p} = 1 - p$)
 C capacidad de un canal
 P_E probabilidad de error
 M numero de mensajes de un código
 R velocidad de información
 D distancia de Hamming
 $d(b_j)$ regla de decisión

Expresiones de la entropía

$$I(s_i) = \log \frac{1}{P(s_i)}$$

Cantidad de información obtenida al recibir s_i (fuente de memoria nula)

$$I(s_i|s_j) = \log \frac{1}{P(s_i|s_j)}$$

Cantidad de información obtenida al recibir sucesivamente s_j y s_i (fuente de markov de primer orden)

$$H(S) = \sum_s P(s_i) * \log \frac{1}{P(s_i)}$$

Entropía de la fuente de memoria nula S

$$H(S|s_j) = \sum_{i=1}^q P(s_i|s_j) * \log \frac{1}{P(s_i|s_j)}$$

Entropía condicional de una fuente de markov de primero orden S

$$H(S) = \sum_{s^2} P(s_i, s_j) * \log \frac{1}{p(s_i|s_j)}$$

Entropía de una fuente de markov de primer orden

$$H_r(S) = \frac{H(S)}{\log r}$$

Entropía, medida en unidades r -arias

$$H(\omega) = \omega * \log \frac{1}{\omega} + (1 - \omega) * \log \frac{1}{1 - \omega}$$

Función entropía

$$H(A) = \sum_A P(a) * \log \frac{1}{P(a)}$$

Entropía condicional de A (entropía a posteriori)

$$H(A|b_j) = \sum_A P(a|b_j) * \log \frac{1}{P(a|b_j)}$$

Entropía del alfabeto de entrada A (entropía a priori)

$$H(A, B) = \sum_{A, B} P(a, b) * \log \frac{1}{P(a, b)}$$

Entropía afín de A y B

$$I(A; B) = H(A) - H(A|B)$$

Información mutua de A y B

$$I(a; B) = \sum_B P(b|a) * \log \frac{P(b|a)}{P(b)}$$

Información mutua condicional

$$H(A, B|C) = \sum_{A, B, C} P(a, b, c) * \log \frac{1}{P(a, b|c)}$$

Equivocación de A y B con respecto a C

$$H(A|B, C) = \sum_{A, B, C} P(a, b, c) * \log \frac{1}{P(a|b, c)}$$

Equivocación de A con respecto a B y C

$$I(A; B|C) = H(A|C) - H(A|B, C)$$

Información mutua de A y B conocido C

$$I(A; B; C) = H(A; B) - H(A; B|C)$$

Información mutua de A , B y C